

C# - podstawy programowania 2026/2027

Zwarty PDF testowy na podstawie materiałów kursu

Autor: Mariusz Smółka
Wygenerowano: 2026-07-01

Spis treści

1. Strona główna	31
C# - podstawy programowania 2026/2027	31
Od czego zacząć	31
Jak korzystać z kursu?	31
Start	31
2. Licencja	32
Licencja	32
3. Organizacja kursu	33
Organizacja kursu	33
Cel działu	33
Jak korzystać z kursu	33
Co będzie dalej	33
4. Status kursu	34
5. Wprowadzenie	35
01 - Podstawy	35
Lekcje	35
Po tym dziale	35
6. Pierwszy program	36
Pierwszy program w C# i Console.WriteLine()	36
Cel lekcji	36
1. Czym jest program konsolowy	36
2. Pierwszy program w C#	36
3. Klasyczna struktura programu	36
4. Dlaczego używamy Main()	37

5. Skrócony zapis w nowszym C#	37
6. Krótko o namespace	37
7. Rola Console.WriteLine()	38
8. Tekst w cudzysłowie a kod programu	38
9. Komentarze	38
10. Proste przykłady wypisywania tekstu	39
Typowe błędy	40
Zapamiętaj	40
Ćwiczenia	40
7. Console.WriteLine	41
Console.WriteLine	41
Cel lekcji	41
Krótkie wyjaśnienie	41
Przykłady kodu	41
Najważniejsze elementy	41
Typowe błędy	41
Ćwiczenia kontrolne	42
8. Zmienne	43
Zmienne i podstawowe typy danych w C#	43
Cel lekcji	43
1. Czym jest zmienna	43
2. Deklaracja zmiennej	43
3. Przypisanie wartości	44
4. Zmiana wartości zmiennej	44
5. Podstawowe typy danych	44
6. Różnica między tekstem a liczbą	45
7. Wypisywanie wartości zmiennych	45

8. Łączenie tekstu z wartością zmiennej	46
9. Nazwy zmiennych	46
10. C# jest językiem statycznie typowanym	47
Typowe błędy	47
Zapamiętaj	47
Ćwiczenia	47
9. Typy danych	49
Typy danych	49
Cel lekcji	49
Krótkie wyjaśnienie	49
Przykład kodu	49
Najważniejsze elementy	49
Typowe błędy	49
Ćwiczenia kontrolne	49
10. Komentarze	51
Komentarze, czytelność kodu i konwencje nazewnictwa	51
Cel lekcji	51
1. Po co stosuje się komentarze	51
2. Komentarz jednoliniowy	51
3. Komentarz wieloliniowy	52
4. Kiedy komentarz pomaga	52
5. Kiedy komentarz jest zbędny	52
6. Czytelne nazwy zmiennych	53
7. Konwencje nazewnictwa	53
8. Przykłady gorszych nazw	54
9. Podstawowe zasady formatowania kodu	55
Typowe błędy	56

Zapamiętaj	56
Ćwiczenia	56
11. Operatory	57
Operatory w C#	57
Cel lekcji	57
1. Czym jest operator	57
2. Operatory arytmetyczne	57
3. Dzielenie całkowite i rzeczywiste	58
4. Operator przypisania	58
5. Skrócone operatory przypisania	59
6. Inkrementacja i dekrementacja	59
7. Operatory porównania	60
8. Wynik porównania jako bool	60
9. Operatory logiczne	61
10. Kolejność wykonywania działań	61
11. Proste przykłady z Console.WriteLine()	62
Typowe błędy	62
Zapamiętaj	62
Ćwiczenia	62
12. Wprowadzenie	64
02 - Wejście i wyjście	64
Lekcje	64
Po tym dziale	64
13. Console.ReadLine	65
Wczytywanie tekstu z konsoli - Console.ReadLine()	65
Cel lekcji	65
1. Czym jest wejście programu	65

2. Czym jest wyjście programu	65
3. Przypomnienie Console.WriteLine()	65
4. Wczytywanie danych za pomocą Console.ReadLine()	65
5. Console.ReadLine() zwraca string	66
6. Zapisanie wczytanego tekstu do zmiennej	66
7. Wypisanie wczytanej wartości	66
8. Prosty dialog z użytkownikiem	67
9. Komunikat dla użytkownika a wczytanie odpowiedzi	67
10. Console.WriteLine() i Console.ReadLine()	68
Typowe błędy	68
Zapamiętaj	68
Ćwiczenia	69
 14. Konwersja typów	 70
Konwersja typów po Console.ReadLine()	70
Cel lekcji	70
1. Przypomnienie: Console.ReadLine() zwraca string	70
2. Dlaczego nie można od razu liczyć na tekście	70
3. Konwersja tekstu na int	71
4. Krótszy zapis z int.Parse()	71
5. Dodawanie dwóch liczb	71
6. Konwersja tekstu na double	72
7. Konwersja za pomocą Convert	72
8. Parse i Convert	73
9. Typowe błędy przy konwersji	73
10. TryParse jako bezpieczniejszy sposób	73
11. Proste obliczenia po konwersji	73
Typowe błędy	74

Zapamiętaj	74
Ćwiczenia	74
15. Formatowanie wyników	76
Formatowanie wyników w konsoli	76
Cel lekcji	76
1. Dlaczego sposób wypisywania wyniku jest ważny	76
2. Przypomnienie Console.WriteLine()	76
3. Łączenie tekstu i zmiennych operatorem +	76
4. Interpolacja napisów	77
5. Dlaczego interpolacja jest czytelniejsza	77
6. Wypisywanie kilku zmiennych w jednym zdaniu	78
7. Wynik obliczenia w czytelnym formacie	78
8. Formatowanie liczb rzeczywistych	78
9. Wartość zmiennej a sposób wypisania	79
10. Średnia dwóch liczb	79
11. Prosty przykład z polem prostokąta	80
Typowe błędy	80
Zapamiętaj	80
Ćwiczenia	80
16. Podsumowanie działu	82
Podsumowanie działu - wejście i wyjście w konsoli	82
Cel lekcji	82
1. Console.WriteLine()	82
2. Console.ReadLine()	82
3. Console.ReadLine() zwraca string	82
4. Konwersja na int	83
5. Konwersja na double	83

6. TryParse	84
7. Interpolacja napisów	84
8. Formatowanie liczb	84
9. Przykład całościowy	84
10. Typowe błędy	85
Najważniejsze do zapamiętania	86
Minićwiczenia powtórkowe	86
17. Wprowadzenie	87
03 - Warunki	87
Lekcje	87
Po tym dziale	87
18. if	88
Instrukcja warunkowa if	88
Cel lekcji	88
1. Krótkie wprowadzenie	88
2. Składnia if	88
3. Pierwszy przykład	88
4. Przykład z warunkiem niespełnionym	89
5. Warunki z operatorami	89
6. Zgadnij wynik warunku	90
7. Przykład z danymi z klawiatury - liczba dodatnia	91
8. Przykład z parzystością	91
9. Przykład z progiem punktów	91
10. Przykład z temperaturą	92
11. Typowe błędy	92
12. Zapowiedź kolejnej lekcji	93
Zapamiętaj	93

Ćwiczenia	93
19. if else	95
Instrukcja warunkowa if else	95
Cel lekcji	95
1. Krótkie przypomnienie if	95
2. Składnia if else	95
3. Pierwszy przykład - pełnoletniość	95
4. Liczba dodatnia albo niedodatnia	96
5. Liczba parzysta albo nieparzysta	97
6. Próg punktów	97
7. Proste hasło	97
8. Zgadnij, który blok się wykona	98
9. If bez klamerek	99
10. Operator warunkowy ?:	100
11. Typowe błędy	100
12. Zapowiedź kolejnej lekcji	101
Zapamiętaj	101
Ćwiczenia	101
20. else if	103
Instrukcja else if - wiele warunków	103
Cel lekcji	103
1. Krótkie przypomnienie	103
2. Składnia else if	103
3. Liczba dodatnia, ujemna albo zero	103
4. Próg punktów	104
5. Temperatura	105
6. Prosty przykład z zakresem i operatorem &&	106

7. Proste menu tekstowe	106
8. Zgadnij, który blok się wykona	107
9. Typowe błędy	108
10. Zapowiedź kolejnej lekcji	109
Zapamiętaj	109
Ćwiczenia	110
 21. switch	 111
Instrukcja switch	111
Cel lekcji	111
1. Krótkie przypomnienie else if	111
2. Składnia switch	111
3. Pierwszy przykład - menu	112
4. Dzień tygodnia	112
5. Ocena szkolna	113
6. Wybór działania kalkulatora	114
7. Switch a if / else if	115
8. Zgadnij, który case się wykona	115
9. Typowe błędy	117
10. Krótka wzmianka o nowoczesnym switch	118
11. Zapowiedź kolejnej części	118
Zapamiętaj	118
Ćwiczenia	118
 22. Podsumowanie działu	 119
Podsumowanie działu - instrukcje warunkowe	119
Cel lekcji	119
1. Po co są instrukcje warunkowe	119
2. Warunek	119

3. Najważniejsze konstrukcje	119
4. if	120
5. if else	120
6. else if	120
7. switch	121
8. Operatory często używane w warunkach	121
9. Różnica między = i ==	122
10. Kilka if a if / else if / else	122
11. Kiedy switch, a kiedy if / else if	122
12. Przykład całościowy	123
Najczęstsze błędy	124
Zgadnij, co wypisze program	124
Najważniejsze do zapamiętania	125
Minićwiczenia powtórkowe	125
23. Wprowadzenie	127
04 - Pętle	127
Lekcje	127
Po tym dziale	127
24. while	128
Pętla while	128
Cel lekcji	128
1. Krótkie wprowadzenie	128
2. Składnia while	128
3. Pierwszy przykład - liczby od 1 do 5	128
4. Warunek od początku fałszywy	129
5. Liczby parzyste	130
6. Suma liczb od 1 do 5	130

7. Wczytywanie kilku liczb	130
8. Zgadnij, ile razy wykona się pętla	131
9. Typowe błędy	132
10. Zapowiedź kolejnej lekcji	133
Zapamiętaj	133
Ćwiczenia	133
25. do while	134
Pętla do while	134
Cel lekcji	134
Przypomnienie pętli while	134
Składnia pętli do while	134
Przykład: liczby od 1 do 5	134
Warunek fałszywy od początku	135
while a do while	135
Wyjaśnienie	136
Proste menu konsolowe	136
Powtarzanie prostego obliczenia	137
Ile razy wykona się pętla?	138
Typowe błędy	139
Kiedy używać do while?	140
Zapamiętaj	140
Ćwiczenia	141
26. for	142
Pętla for	142
Cel lekcji	142
Krótkie przypomnienie pętli	142
Składnia pętli for	142

Pierwszy przykład: liczby od 1 do 5	143
Przykład: liczby od 0 do 4	143
Przykład: odliczanie w dół	144
Przykład: liczby parzyste	144
Wyjaśnienie	144
Przykład: suma liczb od 1 do 10	145
Przykład: wczytanie 5 liczb i obliczenie sumy	145
Przykład: średnia z kilku liczb	146
Zgadnij, ile razy wykona się pętla	146
Porównanie for i while	147
Typowe błędy	147
Zapowiedź kolejnej lekcji	148
Zapamiętaj	149
Ćwiczenia	149
27. Tablice jednowymiarowe - podstawy	150
Tablice jednowymiarowe - podstawy	150
Cel lekcji	150
Krótkie wprowadzenie	150
Czym jest tablica	150
Deklaracja i utworzenie tablicy	150
Indeksy zaczynają się od 0	151
Inicjalizacja tablicy gotowymi wartościami	151
Tablice różnych typów	151
Zmiana wartości elementu tablicy	152
Właściwość Length	152
Wyjaśnienie	152
Typowe błędy	153

Zapowiedź kolejnej lekcji	153
Zapamiętaj	153
Ćwiczenia	154
28. Pętla for po tablicy	155
Pętla for po tablicy	155
Cel lekcji	155
Krótkie przypomnienie	155
Problem bez pętli	155
Pętla for po tablicy	155
Dlaczego używamy <code>i < tablica.Length</code>	156
Wypisywanie indeksu i wartości	156
Zmiana elementów tablicy w pętli	157
Wczytywanie danych do tablicy	157
Indeks w tablicy a numer dla użytkownika	158
Wyjaśnienie	158
Typowe błędy	159
Zapowiedź kolejnej lekcji	160
Zapamiętaj	160
Ćwiczenia	160
29. Suma i średnia elementów tablicy	161
Suma i średnia elementów tablicy	161
Cel lekcji	161
Krótkie przypomnienie	161
Suma elementów tablicy	161
Przebieg działania krok po kroku	161
Średnia elementów tablicy	162
Dlaczego używamy <code>Length</code>	162

Wczytywanie liczb do tablicy i obliczanie średniej	163
Suma i średnia liczb rzeczywistych	163
Wyjaśnienie	164
Typowe błędy	164
Zgadnij, jaki będzie wynik	165
Zapowiedź kolejnej lekcji	166
Zapamiętaj	166
Ćwiczenia	166
30. Minimum i maksimum elementów tablicy	167
Minimum i maksimum elementów tablicy	167
Cel lekcji	167
Krótkie wprowadzenie	167
Dlaczego zaczynamy od pierwszego elementu	167
Znalezienie minimum	167
Przebieg szukania minimum krok po kroku	168
Znalezienie maksimum	168
Minimum i maksimum w jednej pętli	168
Wczytanie liczb do tablicy i znalezienie minimum oraz maksimum	169
Wyjaśnienie	170
Typowe błędy	170
Zgadnij, jaki będzie wynik	171
Zapowiedź kolejnej lekcji	172
Zapamiętaj	172
Ćwiczenia	172
31. Zliczanie elementów tablicy spełniających warunek	174
Zliczanie elementów tablicy spełniających warunek	174
Cel lekcji	174

Krótkie wprowadzenie	174
Licznik	174
Akumulator	174
Licznik a akumulator: różnica	175
Przykład: zliczanie liczb parzystych	175
Przykład: suma podanych liczb	176
Przykład: suma i średnia	176
Przykład: licznik i akumulator razem	177
Wyjaśnienie	177
Typowe wartości początkowe	177
Typowe błędy	178
Zgadnij, co będzie wynikiem	178
Zapowiedź kolejnej lekcji	179
Zapamiętaj	179
Ćwiczenia	180
32. Pętla foreach	181
Pętla foreach	181
Cel lekcji	181
Krótkie przypomnienie for po tablicy	181
Składnia foreach	181
Pierwszy przykład foreach	181
Nazwa zmiennej w foreach	182
foreach z tablicą tekstów	182
Sumowanie za pomocą foreach	182
Średnia za pomocą foreach	183
Zliczanie elementów tablicy spełniających warunek	183
Kiedy for, a kiedy foreach	184

Przykład, w którym lepszy jest for	184
Przykład zmiany elementów tablicy	184
Wyjaśnienie	185
Typowe błędy	185
Zgadnij, co wypisze program	186
Zapowiedź kolejnej lekcji	186
Zapamiętaj	187
Ćwiczenia	187
33. break i continue	188
Instrukcje break i continue	188
Cel lekcji	188
Krótkie wprowadzenie	188
Instrukcja break	188
Instrukcja continue	188
Różnica między break i continue	189
Wyjaśnienie	190
break przy szukaniu elementu w tablicy	190
break i zmienna bool: czy znaleziono	190
continue przy pomijaniu elementów tablicy	191
continue przy sumowaniu tylko wybranych wartości	191
break w pętli while	192
continue w pętli while: ostrożnie	192
Zgadnij, co wypisze program	193
Typowe błędy	194
Kiedy używać break, a kiedy continue	195
Zapowiedź podsumowania działu	195
Zapamiętaj	195

Ćwiczenia	195
34. Wprowadzenie	196
05 - Metody	196
35. Po co są metody	197
Po co są metody	197
Problem: cały kod w Main	197
Metoda jako nazwany fragment kodu	197
Jak działa wywołanie metody	198
Metoda może być wywołana wiele razy	198
Dlaczego to jest przydatne	199
Ważne pojęcia	199
Ćwiczenia	199
Podsumowanie	199
36. Pierwsza metoda void	200
Pierwsza metoda void	200
Co oznacza void	200
Składnia metody void	200
Pierwszy przykład	200
Jak program przechodzi przez metodę	201
Metoda z kilkoma instrukcjami	201
Kilka wywołań tej samej metody	201
Nazwy metod	202
Najczęstsze błędy	202
Ćwiczenia	202
Podsumowanie	203
37. Parametry metody	204

Parametry metody	204
Po co są parametry	204
Parametr i argument	204
Jak dane trafiają do metody	204
Metoda z jednym parametrem	205
Typ parametru	205
Metoda z kilkoma parametrami	205
Kilka wywołań z różnymi argumentami	206
Najczęstsze błędy	206
Ćwiczenia	207
Podsumowanie	207
38. Metoda zwracająca wartość	208
Metoda zwracająca wartość	208
Od void do wartości zwracanej	208
Instrukcja return	208
Pierwszy przykład: metoda zwracająca int	208
Jak działa return	209
Metoda z parametrami i wartością zwracaną	209
Użycie wyniku bez zmiennej	210
Metoda zwracająca double	210
Metoda zwracająca string	210
Najczęstsze błędy	211
Ćwiczenia	211
Podsumowanie	211
39. void a return	213
void a return	213
Dwa rodzaje metod	213

Metoda void	213
Metoda zwracająca wartość	214
Najważniejsza różnica	214
Jak wybrać: void czy return	214
Diagram decyzyjny	215
Console.WriteLine to nie return	215
Częsty błąd: próba zapisania wyniku metody void	215
Ćwiczenia	216
Podsumowanie	216
40. Metody i tablice	217
Metody i tablice	217
Po co przekazywać tablicę do metody	217
Tablica jako parametr metody	217
Pierwszy przykład: wypisanie tablicy	217
Jak tablica trafia do metody	218
Metoda zwracająca sumę elementów tablicy	218
Metoda zwracająca średnią	219
Metoda zwracająca minimum	219
Porządkowanie programu	220
Najczęstsze błędy	221
Ćwiczenia	221
Podsumowanie	221
41. Ćwiczenia	222
Ćwiczenia — metody	222
Jak pracować z ćwiczeniami	222
Ćwiczenia podstawowe — metody void	222
Ćwiczenia z parametrami	222

Ćwiczenia z return	222
Ćwiczenia z tablicami	223
Zadania przekrojowe	223
Kontrola samodzielna	224
Podsumowanie	224
42. Wprowadzenie	225
06 - Zakres zmiennych	225
43. Blok kodu i zakres zmiennej	226
Blok kodu i zakres zmiennej	226
Co to jest blok kodu	226
Zmienna lokalna	226
Zakres zmiennej	226
Diagram: gdzie istnieje zmienna	226
Zmienna w instrukcji if	227
Zmienna zadeklarowana przed blokiem	228
Zmienna w pętli for	228
Dlaczego to jest ważne	229
Najczęstsze błędy	229
Ćwiczenia	229
Podsumowanie	229
44. Zmienne lokalne	230
Zmienne lokalne	230
Co to jest zmienna lokalna	230
Zmienna lokalna przechowuje dane tymczasowe	230
Zmienna lokalna w metodzie pomocniczej	230
Diagram: zmienne lokalne w metodzie	231
Zmienna lokalna w if	231

Zmienna lokalna w pętli	231
Dobre nazwy zmiennych lokalnych	232
Zmienna lokalna a parametr metody	232
Najczęstsze błędy	232
Ćwiczenia	233
Podsumowanie	233
45. Parametry metod a zmienne lokalne	234
Parametry metod a zmienne lokalne	234
Przypomnienie: parametr	234
Przypomnienie: zmienna lokalna	234
Podobieństwo	234
Różnica	234
Osobne zakresy metod	235
Diagram: argument i parametr	235
Zmienna z Main nie jest automatycznie widoczna w innej metodzie	236
Czy można użyć innej nazwy parametru	236
Najczęstsze błędy	237
Ćwiczenia	237
Podsumowanie	237
46. Zmienne w pętlach	238
Zmienne w pętlach	238
Dlaczego zakres zmiennych w pętli jest ważny	238
Licznik pętli for	238
Zmienna pomocnicza wewnątrz pętli	239
Akumulator przed pętlą	239
Diagram: zmienna w pętli a zmienna przed pętlą	239
Licznik spełnionych warunków	240

Błąd: akumulator zadeklarowany w pętli	240
Zmienna w pętli while	241
Najczęstsze błędy	241
Ćwiczenia	242
Podsumowanie	242
47. Przesłanianie zmiennych	243
Przesłanianie zmiennych	243
O co chodzi z nazwami	243
Czym jest przesłanianie	243
Przykład: pole klasy i zmienna lokalna	243
Diagram: która nazwa jest bliżej	243
Dlaczego to może być mylące	244
C# nie pozwala na wszystko	244
Parametr i zmienna lokalna o tej samej nazwie	245
Dobre praktyki	245
Najczęstsze błędy	246
Ćwiczenia	246
Podsumowanie	246
48. Najczęstsze błędy	247
Najczęstsze błędy — zakres zmiennych	247
Po co analizować błędy	247
Błąd 1: użycie zmiennej poza blokiem if	247
Błąd 2: użycie licznika for po pętli	248
Błąd 3: akumulator zadeklarowany wewnątrz pętli	248
Błąd 4: użycie zmiennej przed deklaracją	249
Błąd 5: metoda próbuje użyć zmiennej lokalnej z Main	250
Błąd 6: parametr i zmienna lokalna o tej samej nazwie	251

Błąd 7: mylące przesłanianie pola klasy	251
Jak unikać błędów zakresu	252
Ćwiczenia	252
Podsumowanie	252
49. Wprowadzenie	254
Lekcje	254
50. Tablice dwuwymiarowe	255
Tablice dwuwymiarowe	255
Po co są tablice dwuwymiarowe	255
Tablica jako tabela	255
Deklaracja tablicy dwuwymiarowej	255
Inicjalizacja tablicy wartościami	255
Dostęp do elementu tablicy	256
Diagram: wiersz i kolumna	256
Liczba wierszy i kolumn	256
Przejsięcie po tablicy dwiema pętlami	257
Suma wszystkich elementów	257
Suma jednego wiersza	258
Suma jednej kolumny	258
Najczęstsze błędy	258
Ćwiczenia	259
Podsumowanie	259
51. Tablice wielowymiarowe	260
Tablice wielowymiarowe	260
Od tablicy jednowymiarowej do wielowymiarowej	260
Wymiar tablicy	260

Tablica trójwymiarowa	260
Dostęp do elementu	260
Diagram: trzy indeksy	261
GetLength w tablicy trójwymiarowej	261
Przechodzenie po tablicy trójwymiarowej	262
Kiedy to może mieć sens	262
Dlaczego trzeba uważać	262
Najczęstsze błędy	262
Ćwiczenia	263
Podsumowanie	263
52. List	264
List	264
Po co jest List	264
Potrzebny using	264
Utworzenie listy i dodawanie elementów	264
Diagram: lista rośnie	265
Odczyt elementu przez indeks	265
Przejdźcie po liście pętlą for	265
Przejdźcie po liście pętlą foreach	266
Usuwanie elementów	266
Sprawdzenie, czy lista zawiera element	266
Suma elementów listy	266
Tablica a lista	267
Najczęstsze błędy	267
Ćwiczenia	267
Podsumowanie	267
53. List	268

List	268
Lista tekstów	268
Potrzebny using	268
Dodawanie tekstów do listy	268
Diagram: dodawanie tekstów	269
Odczyt elementu przez indeks	269
Wypisywanie listy pętlą for	269
Wypisywanie listy pętlą foreach	270
Sprawdzanie, czy lista zawiera tekst	270
Usuwanie tekstów z listy	270
Przykład: prosta lista obecności	270
Najczęstsze błędy	271
Ćwiczenia	271
Podsumowanie	271
54. List a tablica	272
List a tablica	272
Dwa sposoby przechowywania wielu wartości	272
Tablica ma ustalony rozmiar	272
Lista może rosnać	272
Diagram: stały rozmiar i rosnąca lista	273
Dostęp przez indeks	273
Przejsięcie pętlą for	273
Przejsięcie pętlą foreach	274
Kiedy użyć tablicy	274
Kiedy użyć listy	274
Tabela porównawcza	274
Najczęstsze błędy	275

Ćwiczenia	275
Podsumowanie	275
55. Przekazywanie parametrów przez wartość	276
Przekazywanie parametrów przez wartość	276
Przypomnienie: parametr i argument	276
Co znaczy „przez wartość”	276
Przykład: zmiana parametru nie zmienia zmiennej w Main	276
Diagram: kopia wartości	277
Dlaczego tak się dzieje	277
Jak poprawnie zwrócić zmienioną wartość	277
Zmiana parametru a zwrócenie wyniku	278
Przykład z dwoma liczbami	278
Uwaga o tablicach i listach	279
Przygotowanie do ref i out	279
Najczęstsze błędy	279
Ćwiczenia	279
Podsumowanie	279
56. ref i out	281
ref i out	281
Przypomnienie: zwykle przekazywanie przez wartość	281
Po co jest ref	281
Przykład ref	281
Diagram: ref zmienia zmienną z Main	282
Zamiana wartości dwóch zmiennych	282
Po co jest out	283
Przykład out	283
out i wynik dodatkowy	283

ref a out	284
Kiedy używać ref i out	284
Najczęstsze błędy	284
Ćwiczenia	284
Podsumowanie	285
57. Ćwiczenia	286
Ćwiczenia — tablice, listy i przekazywanie danych	286
Jak pracować z ćwiczeniami	286
Ćwiczenia: tablice dwuwymiarowe	286
Ćwiczenia: tablice wielowymiarowe	286
Ćwiczenia: List	286
Ćwiczenia: List	287
Ćwiczenia: tablica czy lista	287
Ćwiczenia: przekazywanie przez wartość	287
Ćwiczenia: ref	287
Ćwiczenia: out	287
Zadania przekrojowe	288
Kontrola samodzielna	288
Podsumowanie	289
58. Wprowadzenie	290
08 - Wstęp do obiektowości	290
59. Po co są obiekty	291
Po co są obiekty	291
Problem: wiele zmiennych opisuje jedną rzecz	291
Gdy danych jest więcej	291
Pomysł: potraktuj ucznia jako jedną całość	291
Obiekt jako jedna całość	292

Dane i czynności	292
Klasa i obiekt — bardzo wstępnie	292
Pierwszy bardzo prosty przykład	292
Diagram: od wielu zmiennych do obiektu	293
Dlaczego obiekty pomagają	293
Czego jeszcze nie robimy	294
Najczęstsze błędy na początku	294
Ćwiczenia	294
Podsumowanie	294
60. Klasa i obiekt	295
Klasa i obiekt	295
Przypomnienie z poprzedniej lekcji	295
Klasa jako opis	295
Prosta klasa Uczeń	295
Obiekt jako egzemplarz klasy	295
Dostęp do pól przez kropkę	296
Kompletny przykład	296
Diagram: klasa i obiekt	296
Kilka obiektów tej samej klasy	297
Klasa to nie obiekt	297
Najczęstsze błędy	297
Ćwiczenia	298
Podsumowanie	298
61. Pola obiektu	299
Pola obiektu	299
Cel lekcji	299
Co to jest pole	299

Pole a zmienna lokalna	299
Dostęp do pola przez kropkę	300
Kompletny przykład z polami	300
Diagram: obiekt i jego pola	300
Każdy obiekt ma własne pola	301
Zmiana wartości pola	301
public jako uproszczenie	302
Najczęstsze błędy	302
Ćwiczenia	302
Podsumowanie	303
62. Wprowadzenie	304
09 - Projekty	304
Planowane projekty	304

1. Strona główna

index.md

C# - podstawy programowania 2026/2027

Autor: Mariusz Smółka

To jest strona główna kursu C# dla początkujących. Materiały dotyczą głównie programowania konsolowego i podstaw potrzebnych na początku nauki INF.04.

Od czego zacząć

Przejdź do pierwszej lekcji (01-podstawy/01-pierwszy-program.md)

Jak korzystać z kursu?

1. Zaczynaj od pierwszej lekcji.
2. Korzystaj ze spisu treści po lewej stronie.
3. Na dole każdej lekcji używaj przycisków „Poprzednia lekcja” i „Następna lekcja”.
4. Na telefonie spis treści jest dostępny pod przyciskiem „Spis treści”.

Start

Rozpocznij kurs od pierwszej lekcji (./01-podstawy/01-pierwszy-program.html)

2. Licencja

licencja.md

□--- layout: default title: Licencja ---

Licencja

Materiały w tym repozytorium są udostępniane na licencji Creative Commons:

CC BY-NC-SA 4.0

Oznacza to, że możesz korzystać z materiałów, udostępniać je i modyfikować, jeżeli:

- podasz autora,
- nie użyjesz materiałów komercyjnie,
- udostępnisz zmienione materiały na tej samej licencji.

3. Organizacja kursu

00-organizacja/README.md

Organizacja kursu

Cel działu

Ten dział opisuje układ kursu i pomaga szybko sprawdzić, które materiały są już przygotowane.

Jak korzystać z kursu

Lekcje są ułożone od najprostszych tematów do trudniejszych. Najlepiej czytać je po kolei i przepisywać przykłady do projektu konsolowego w Visual Studio.

Co będzie dalej

Najpierw powstają krótkie lekcje teoretyczne z przykładami. Większe materiały praktyczne będą przygotowane w osobnych etapach.

4. Status kursu

00-organizacja/status-kursu.md

Status kursu

Statusy:

- OK - materiał jest przygotowany,
- BRAK - materiału jeszcze nie ma,
- DO POPRAWY - materiał wymaga zmian,
- ROBOCZE - materiał jest wstępny.

Element	Status
---	---
Strona główna	OK
Licencja	OK
00-organizacja	OK
01-podstawy	ROBOCZE
02-wejście-wyjście	ROBOCZE
03-warunki	ROBOCZE
04-pętle	ROBOCZE
05-metody	ROBOCZE
06-zakres-zmiennych	ROBOCZE
07-objekty	ROBOCZE
08-projekty	ROBOCZE

5. Wprowadzenie

01-podstawy/README.md

01 - Podstawy

W tym dziale poznasz pierwsze elementy języka C# potrzebne do pisania programów konsolowych.

Lekcje

- Pierwszy program (01-pierwszy-program.md)
- Console.WriteLine (02-console-writeline.md)
- Zmienne (03-zmienne.md)
- Typy danych (04-typy-danych.md)
- Komentarze (05-komentarze.md)
- Operatory (06-operatory.md)

Po tym dziale

Uczeń powinien umieć uruchomić prosty program, wypisać tekst w konsoli, utworzyć zmienne i wykonać podstawowe obliczenia.

6. Pierwszy program

01-podstawy/01-pierwszy-program.md

Pierwszy program w C# i Console.WriteLine()

Cel lekcji

Poznasz pierwszy program konsolowy w C#, klasyczną strukturę programu z metodą `Main()` oraz instrukcję `Console.WriteLine()`.

Po tej lekcji powinieneś umieć:

- rozpoznać miejsce, od którego program zaczyna działanie,
- wypisać prosty tekst w konsoli,
- odróżnić tekst w cudzysłowie od kodu programu,
- dodać prosty komentarz do kodu.

1. Czym jest program konsolowy

Program konsolowy to program, który działa w oknie konsoli. Użytkownik widzi tekst, może wpisywać dane z klawiatury, a program może wypisywać wyniki.

Na początku kursu będziemy pisać właśnie takie programy, ponieważ są proste i dobrze pokazują podstawy programowania.

W kolejnych lekcjach nauczysz się pobierać dane, tworzyć zmienne i wykonywać obliczenia. W tej lekcji skupiamy się tylko na pierwszym programie i wypisywaniu tekstu.

2. Pierwszy program w C#

W tym kursie jako podstawowy wariant programu stosujemy klasyczną strukturę z metodą `Main()`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Witaj w C#!");
    }
}
```

Po uruchomieniu programu w konsoli pojawi się tekst:

Kod: csharp

```
Witaj w C#!
```

3. Klasyczna struktura programu

Przyjrzyjmy się najważniejszym elementom programu.

Kod: csharp

```
using System;
```

Ta linia pozwala korzystać z elementów przestrzeni `System`, na przykład z `Console`.

Kod: csharp

```
class Program
{
}
```

`class Program` to kontener na kod programu. Na tym etapie nie musisz jeszcze znać szczegółów programowania obiektowego. Wystarczy zapamiętać, że klasa pomaga uporządkować kod.

Kod: csharp

```
static void Main()
{
}
```

`Main()` to miejsce startu programu. Instrukcje zapisane wewnątrz `Main()` wykonują się po uruchomieniu programu.

Kod: csharp

```
Console.WriteLine("Witaj w C#!");
```

Ta instrukcja wypisuje tekst w konsoli i przechodzi do nowej linii.

4. Dlaczego używamy `Main()`

W tym kursie używamy wersji z `Main()`, ponieważ:

- uczeń widzi strukturę programu,
- łatwo wskazać, gdzie program zaczyna działanie,
- od początku pojawia się pojęcie klasy i metody na poziomie organizacyjnym,
- kod jest czytelny i uporządkowany,
- później łatwiej przejść do metod, klas i większych programów.

Nie omawiamy jeszcze dokładnie klas i metod. Na razie wystarczy wiedzieć, że `Main()` jest punktem startowym programu.

5. Skrócony zapis w nowszym C#

W nowszych wersjach C# można napisać najprostszy program krócej:

Kod: csharp

```
Console.WriteLine("Witaj w C#!");
```

Taki zapis nazywa się często `top-level statements`. Kompilator tworzy część struktury programu automatycznie.

To wygodne, ale w tym kursie świadomie używamy wersji z `class Program` i `Main()`, ponieważ lepiej pokazuje budowę programu.

6. Krótko o namespace

`namespace` to przestrzeń nazw. Pomaga porządkować większe programy, szczególnie gdy projekt ma wiele plików i klas.

Na początku kursu `namespace` nie jest konieczny. Wrócimy do niego przy organizacji większych projektów.

Pełniejszy wariant programu może wyglądać tak:

Kod: csharp

```
using System;
```

```
namespace MojPierwszyProgram
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("Witaj w C#!");
        }
    }
}
```

Na razie nie używamy tego wariantu jako podstawowego.

7. Rola Console.WriteLine()

Console.WriteLine() służy do wypisywania informacji w konsoli.

Przykład:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("To jest mój pierwszy program.");
        Console.WriteLine("Uczę się języka C#.");
        Console.WriteLine("Programowanie konsolowe jest dobrym początkiem.");
    }
}
```

Każde Console.WriteLine() wypisuje tekst i przechodzi do następnej linii.

8. Tekst w cudzysłowie a kod programu

Tekst, który ma zostać wypisany, zapisujemy w cudzysłowie.

Kod: csharp

```
Console.WriteLine("Cześć!");
```

W tym przykładzie tekstem jest:

Kod: csharp

```
Cześć!
```

Natomiast Console.WriteLine, nawiasy, cudzysłowy i średnik są częścią kodu programu.

Porównaj:

Kod: csharp

```
Console.WriteLine("C#");
Console.WriteLine("Console.WriteLine");
```

Pierwsza instrukcja wypisze tekst C#. Druga instrukcja wypisze tekst Console.WriteLine, bo znajduje się on w cudzysłowie.

9. Komentarze

Komentarz to tekst w kodzie, który nie jest wykonywany przez program. Służy do krótkiego wyjaśnienia kodu.

Komentarz jednowierszowy zapisujemy po //.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        // Wypisanie powitania w konsoli.
        Console.WriteLine("Dzień dobry!");
    }
}
```

Komentarz wielowierszowy zapisujemy między /* i */.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        /*
            Ten program wypisuje
            trzy linie tekstu.
        */
        Console.WriteLine("Linia 1");
        Console.WriteLine("Linia 2");
        Console.WriteLine("Linia 3");
    }
}
```

Komentarze powinny pomagać w czytaniu kodu. Nie trzeba komentować każdej prostej instrukcji.

10. Proste przykłady wypisywania tekstu

Wypisanie jednej linii:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Mam na imię Jan.");
    }
}
```

Wypisanie kilku linii:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Szkoła");
        Console.WriteLine("Klasa");
        Console.WriteLine("Przedmiot: programowanie");
    }
}
```

Wypisanie pustej linii:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Pierwsza linia");
        Console.WriteLine();
        Console.WriteLine("Trzecia linia");
    }
}
```

Typowe błędy

- Brak średnika ; na końcu instrukcji.
- Brak cudzysłowu przy tekście.
- Wpisanie console małą literą zamiast Console.
- Usunięcie jednej z klamer { albo }.
- Umieszczenie instrukcji poza metodą Main().
- Mylenie tekstu w cudzysłowie z kodem programu.

Zapamiętaj

- Program konsolowy działa w oknie konsoli.
- W tym kursie używamy klasycznej struktury z class Program i Main().
- Main() to miejsce startu programu.
- Instrukcje wewnątrz Main() wykonują się po uruchomieniu programu.
- Console.WriteLine() wypisuje tekst i przechodzi do nowej linii.
- Tekst do wypisania zapisujemy w cudzysłowie.
- Komentarze // i /* ... */ nie są wykonywane przez program.
- Skrócony zapis bez Main() istnieje, ale w tym kursie używamy wersji klasycznej.

Ćwiczenia

1. Wypisz swoje imię.
2. Wypisz nazwę szkoły.
3. Wypisz trzy linie tekstu.
4. Dodaj komentarz do programu.
5. Zmień tekst wypisywany przez program.
6. Uruchom program i wskaż, która instrukcja została wykonana jako pierwsza w Main().
7. Porównaj wariant klasyczny z krótkim zapisem i wskaż, którego będziemy używać w kursie.

7. Console.WriteLine

01-podstawy/02-console-writeline.md

Console.WriteLine

Cel lekcji

Nauczysz się wypisywać tekst i wartości w konsoli.

Krótkie wyjaśnienie

`Console.WriteLine()` wypisuje dane i przechodzi do następnej linii. To jedna z najczęściej używanych instrukcji na początku nauki C#.

Przykłady kodu

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Cześć!");
        Console.WriteLine("Uczę się C#.");
        Console.WriteLine(2026);
    }
}
```

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine(2 + 3);
        Console.WriteLine("2 + 3");
    }
}
```

Najważniejsze elementy

- Tekst zapisujemy w cudzysłowie.
- Liczby można wypisywać bez cudzysłowu.
- `2 + 3` daje wynik 5.
- `"2 + 3"` jest zwykłym tekstem.

Typowe błędy

- Brak cudzysłowu przy tekście.
- Mylenie tekstu `"5"` z liczbą 5.
- Pisanie `WriteLine` bez `Console..`

Ćwiczenia kontrolne

1. Wypisz swoje imię i nazwisko w dwóch liniach.
2. Wypisz wynik działania $10 - 4$.
3. Wypisz tekst $10 - 4$ bez obliczania.

8. Zmienne

01-podstawy/03-zmienne.md

Zmienne i podstawowe typy danych w C#

Cel lekcji

Nauczysz się tworzyć zmienne, przypisywać im wartości, zmieniać te wartości oraz używać podstawowych typów danych w C#.

Po tej lekcji powinieneś umieć:

- wyjaśnić, czym jest zmienna,
- zadeklarować zmienną,
- przypisać wartość do zmiennej,
- wypisać wartość zmiennej w konsoli,
- rozpoznać podstawowe typy danych: `int`, `double`, `char`, `string`, `bool`.

1. Czym jest zmienna

Zmienna to nazwane miejsce w pamięci programu. Można zapisać w niej wartość, a potem używać tej wartości w kolejnych instrukcjach.

Przykład:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 16;

        Console.WriteLine(wiek);
    }
}
```

W tym programie zmienna `wiek` przechowuje wartość 16.

2. Deklaracja zmiennej

Deklaracja zmiennej oznacza utworzenie zmiennej i podanie jej typu oraz nazwy.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek;
    }
}
```

W tej instrukcji:

- `int` oznacza typ zmiennej,

- `wiek` oznacza nazwę zmiennej,
- średnik `;` kończy instrukcję.

3. Przypisanie wartości

Przypisanie wartości oznacza zapisanie konkretnej wartości w zmiennej.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek;
        wiek = 16;

        Console.WriteLine(wiek);
    }
}
```

Można też połączyć deklarację i przypisanie w jednej instrukcji.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 16;

        Console.WriteLine(wiek);
    }
}
```

Operator `=` oznacza przypisanie wartości. Czytamy to jako: do zmiennej `wiek` wpisz wartość 16.

4. Zmiana wartości zmiennej

Wartość zmiennej można zmienić w dalszej części programu.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 16;
        Console.WriteLine(wiek);

        wiek = 17;
        Console.WriteLine(wiek);
    }
}
```

Program najpierw wypisze 16, a potem 17.

Nie podajemy drugi raz typu `int`, gdy zmienna już istnieje. Typ podajemy przy deklaracji.

5. Podstawowe typy danych

W C# każda zmienna ma określony typ. Typ mówi, jakiego rodzaju wartość można przechowywać w zmiennej.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 16;
        double wzrost = 1.75;
        char ocena = 'A';
        string imie = "Anna";
        bool czyAktywny = true;

        Console.WriteLine(wiek);
        Console.WriteLine(wzrost);
        Console.WriteLine(ocena);
        Console.WriteLine(imie);
        Console.WriteLine(czyAktywny);
    }
}
```

Najważniejsze typy na początku kursu:

- `int` przechowuje liczby całkowite, na przykład 16, 0, -5.
- `double` przechowuje liczby rzeczywiste, na przykład 1.75, 3.14, -2.5.
- `char` przechowuje jeden znak zapisany w apostrofach, na przykład 'A'.
- `string` przechowuje tekst zapisany w cudzysłowie, na przykład "Anna".
- `bool` przechowuje wartość logiczną `true` albo `false`.

6. Różnica między tekstem a liczbą

Liczba i tekst to nie to samo.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 16;
        string tekst = "16";

        Console.WriteLine(liczba);
        Console.WriteLine(tekst);
    }
}
```

W konsoli oba wyniki wyglądają podobnie, ale dla programu są to różne wartości:

- 16 bez cudzysłowu jest liczbą,
- "16" w cudzysłowie jest tekstem.

Dlatego tekst zapisujemy w cudzysłowie, a liczby zapisujemy bez cudzysłowu.

7. Wypisywanie wartości zmiennych

Wartość zmiennej można wypisać za pomocą `Console.WriteLine()`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        int wiek = 16;

        Console.WriteLine(imie);
        Console.WriteLine(wiek);
    }
}
```

Jeżeli nazwa zmiennej nie jest w cudzysłowie, program wypisze jej wartość.

8. Łączenie tekstu z wartością zmiennej

Można połączyć tekst z wartością zmiennej za pomocą operatora +.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        int wiek = 16;

        Console.WriteLine("Imię: " + imie);
        Console.WriteLine("Wiek: " + wiek);
    }
}
```

Wynik w konsoli:

Kod: csharp

```
Imię: Anna
Wiek: 16
```

Na tym etapie wystarczy znać proste łączenie tekstu za pomocą +. Bardziej wygodne sposoby formatowania wyników pojawiają się w kolejnych lekcjach.

9. Nazwy zmiennych

Nazwy zmiennych powinny być krótkie, czytelne i związane z przechowywaną wartością.

Dobre nazwy:

- imie,
- wiek,
- wzrost,
- czyAktywny,
- ocena.

Słabsze nazwy:

- x,
- a1,

- cos,
- dane.

W C# często stosuje się zapis, w którym pierwsze słowo zaczyna się małą literą, a kolejne słowa wielką literą, na przykład czyAktywny.

10. C# jest językiem statycznie typowanym

C# jest językiem statycznie typowanym. Oznacza to, że typ zmiennej jest znany już podczas pisania i kompilowania programu.

Jeżeli utworzysz zmienną typu `int`, to nie możesz później wpisać do niej tekstu.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 16;

        Console.WriteLine(wiek);
    }
}
```

Taki zapis jest poprawny. Natomiast próba wpisania tekstu do zmiennej typu `int` byłaby błędem.

Typowe błędy

- Brak średnika ; na końcu instrukcji.
- Zapisanie tekstu bez cudzysłowu.
- Zapisanie wartości typu `char` w cudzysłowie zamiast w apostrofach.
- Próba przypisania tekstu do zmiennej typu `int`.
- Ponowne podawanie typu przy zmianie wartości istniejącej zmiennej.
- Literówka w nazwie zmiennej.
- Mylenie wartości `true` i `false` z tekstem `"true"` i `"false"`.

Zapamiętaj

- Zmienna to nazwane miejsce na wartość.
- Deklaracja zmiennej podaje jej typ i nazwę.
- Operator `=` przypisuje wartość do zmiennej.
- Wartość zmiennej można zmienić.
- `int` służy do liczb całkowitych.
- `double` służy do liczb rzeczywistych.
- `char` służy do pojedynczego znaku w apostrofach.
- `string` służy do tekstu w cudzysłowie.
- `bool` przechowuje `true` albo `false`.
- C# jest językiem statycznie typowanym.

Ćwiczenia

1. Utwórz zmienną `imie` i wypisz jej wartość.
2. Utwórz zmienną `wiek` typu `int` i wypisz jej wartość.
3. Utwórz zmienną `srednia` typu `double` i wypisz jej wartość.
4. Utwórz zmienną `czyUczen` typu `bool`.
5. Wypisz zdanie złożone z tekstu i wartości zmiennej.
6. Zmień wartość jednej zmiennej i wypisz ją ponownie.
7. Sprawdź, co się stanie, gdy tekst zapiszesz bez cudzysłowu.

9. Typy danych

01-podstawy/04-typy-danych.md

Typy danych

Cel lekcji

Poznasz podstawowe typy danych: int, double, string, bool i char.

Krótkie wyjaśnienie

Typ danych określa, jaką wartość może przechowywać zmienna. C# jest językiem silnie typowanym, więc typ zmiennej jest ważny.

Przykład kodu

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 10;
        double temperatura = 21.5;
        string imie = "Ola";
        bool czyPelnoletni = false;
        char znak = 'A';

        Console.WriteLine(liczba);
        Console.WriteLine(temperatura);
        Console.WriteLine(imie);
        Console.WriteLine(czyPelnoletni);
        Console.WriteLine(znak);
    }
}
```

Najważniejsze elementy

- int przechowuje liczby całkowite.
- double przechowuje liczby z częścią dziesiętną.
- string przechowuje tekst.
- bool przechowuje wartość true albo false.
- char przechowuje jeden znak w apostrofach.

Typowe błędy

- Zapis liczby 21,5 zamiast 21.5 w kodzie.
- Zapis znaku char w cudzysłowie zamiast w apostrofach.
- Przypisanie tekstu do zmiennej liczbowej.
- Mylenie string i char.

Ćwiczenia kontrolne

1. Utwórz zmienną typu `double` z ceną produktu.
2. Utwórz zmienną typu `bool`, która przechowuje informację, czy pada deszcz.
3. Utwórz zmienną typu `char` z pierwszą literą swojego imienia.

10. Komentarze

01-podstawy/05-komentarze.md

Komentarze, czytelność kodu i konwencje nazewnictwa

Cel lekcji

Nauczysz się stosować komentarze, pisać czytelniejszy kod i nazywać zmienne w sposób zrozumiały dla człowieka.

Po tej lekcji powinieneś umieć:

- dodać komentarz jednoliniowy,
- dodać komentarz wieloliniowy,
- rozpoznać komentarz pomocny i zbędny,
- dobrać czytelną nazwę zmiennej,
- stosować podstawową konwencję camelCase dla zmiennych lokalnych.

1. Po co stosuje się komentarze

Komentarz to tekst w kodzie, który nie jest wykonywany przez program. Komentarz jest przeznaczony dla człowieka, który czyta kod.

Komentarze pomagają wyjaśnić sens fragmentu programu. Nie powinny jednak przepisywać oczywistych instrukcji.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 16; // wiek ucznia

        Console.WriteLine(wiek);
    }
}
```

2. Komentarz jednoliniowy

Komentarz jednoliniowy zaczyna się od //. Wszystko po // do końca linii jest komentarzem.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        // Wypisanie imienia ucznia.
        string imie = "Anna";

        Console.WriteLine(imie);
    }
}
```

```
}
```

Komentarz jednoliniowy jest dobry do krótkich wyjaśnień.

3. Komentarz wieloliniowy

Komentarz wieloliniowy zaczyna się od `/*` i kończy na `*/`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        /*
         * Ten program wypisuje
         * podstawowe informacje
         * o uczniu.
         */

        string imie = "Anna";
        int liczbaPunktow = 25;

        Console.WriteLine(imie);
        Console.WriteLine(liczbaPunktow);
    }
}
```

Komentarz wieloliniowy przydaje się, gdy trzeba krótko opisać większy fragment kodu.

4. Kiedy komentarz pomaga

Komentarz pomaga, gdy wyjaśnia sens kodu.

Dobry komentarz:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        // Liczba punktów zdobytych przez ucznia w krótkim ćwiczeniu.
        int liczbaPunktow = 25;

        Console.WriteLine(liczbaPunktow);
    }
}
```

Taki komentarz dopowiada, co oznacza wartość w programie.

5. Kiedy komentarz jest zbędny

Komentarz jest zbędny, gdy opisuje coś oczywistego.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
```

```

        // Utworzenie zmiennej liczbaPunktow i przypisanie wartości 25.
        int liczbaPunktow = 25;

        // Wypisanie zmiennej liczbaPunktow.
        Console.WriteLine(liczbaPunktow);
    }
}

```

Ten komentarz głównie powtarza kod. Lepiej użyć czytelnej nazwy zmiennej i dodać komentarz tylko wtedy, gdy naprawdę pomaga.

6. Czytelne nazwy zmiennych

Nazwy zmiennych powinny mówić, co przechowują.

Gorszy przykład:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int x = 25;
        int y = 5;

        Console.WriteLine(x);
        Console.WriteLine(y);
    }
}

```

Lepszy przykład:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int liczbaPunktow = 25;
        int liczbaZadan = 5;

        Console.WriteLine(liczbaPunktow);
        Console.WriteLine(liczbaZadan);
    }
}

```

W kursie preferujemy czytelne nazwy, nawet jeśli są trochę dłuższe.

7. Konwencje nazewnictwa

Konwencja nazewnictwa to umówiony sposób nazywania elementów programu. Dzięki niej kod jest bardziej spójny i czytelny.

Dla zmiennych lokalnych w C# zwykle stosujemy camelCase.

camelCase oznacza, że:

- pierwsze słowo zaczynamy małą literą,
- kolejne słowa zaczynamy wielką literą,
- nie używamy podkreślników między słowami.

Przykłady nazw w camelCase:

- liczbaPunktow,
- imieUcznia,
- sredniaOcen,
- czyAktywny.

Poprawny przykład:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczbaPunktow = 25;
        string imieUcznia = "Anna";
        double sredniaOcen = 4.75;
        bool czyZaliczyl = true;

        Console.WriteLine(liczbaPunktow);
        Console.WriteLine(imieUcznia);
        Console.WriteLine(sredniaOcen);
        Console.WriteLine(czyZaliczyl);
    }
}
```

Wariant angielski jako informacja porządkowa:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int pointsCount = 25;
        string studentName = "Anna";
        double averageGrade = 4.75;
        bool hasPassed = true;

        Console.WriteLine(pointsCount);
        Console.WriteLine(studentName);
        Console.WriteLine(averageGrade);
        Console.WriteLine(hasPassed);
    }
}
```

W profesjonalnym kodzie często stosuje się nazwy angielskie. W kursie szkolnym można stosować nazwy polskie, jeśli pomagają uczniowi zrozumieć program.

Najważniejsza jest konsekwencja. Nie należy bez potrzeby mieszać polskiego i angielskiego w jednej nazwie.

8. Przykłady gorszych nazw

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
```

```

        int LiczbaPunktow = 25;
        int liczba_punktow = 25;
        int lp = 25;
        string imieucznia = "Anna";
        int liczbaPoints = 25;

        Console.WriteLine(LiczbaPunktow);
        Console.WriteLine(liczba_punktow);
        Console.WriteLine(lp);
        Console.WriteLine(imieucznia);
        Console.WriteLine(liczbaPoints);
    }
}

```

Te nazwy są gorsze, ponieważ:

- `LiczbaPunktow` nie jest zgodne z camelCase dla zmiennych lokalnych,
- `liczba_punktow` używa podkreślnika zamiast camelCase,
- `lp` jest zbyt krótkim i nieczytelnym skrótem,
- `imieucznia` nie rozdziela słów,
- `liczbaPoints` miesza język polski i angielski.

9. Podstawowe zasady formatowania kodu

Kod powinien być czytelny nie tylko dla komputera, ale też dla człowieka.

Podstawowe zasady:

- stosuj wcięcia wewnątrz klamerek,
- używaj pustych linii do oddzielania fragmentów kodu,
- zapisuj nawiasy klamrowe w sposób spójny,
- zapisuj jedną instrukcję w jednej linii.

Mniej czytelny zapis:

Kod: csharp

```

using System;
class Program
{
    static void Main()
    {
        string imie="Anna";int liczbaPunktow=25;Console.WriteLine(imie);Console.WriteLine(liczbaPunktow);
    }
}

```

Bardziej czytelny zapis:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        int liczbaPunktow = 25;

        Console.WriteLine(imie);
        Console.WriteLine(liczbaPunktow);
    }
}

```

Oba przykłady mogą oznaczać podobne instrukcje, ale drugi jest znacznie łatwiejszy do czytania.

Typowe błędy

- Komentowanie każdej oczywistej instrukcji.
- Brak zakończenia komentarza wieloliniowego.
- Używanie nazw zmiennych, które nic nie mówią.
- Mieszanie różnych stylów nazewnictwa.
- Mieszanie polskiego i angielskiego w jednej nazwie bez potrzeby.
- Zapisywanie wielu instrukcji w jednej linii.
- Brak wcięć wewnątrz klamerek.

Zapamiętaj

- Komentarz nie jest wykonywany przez program.
- `//` oznacza komentarz jednoliniowy.
- `/* ... */` oznacza komentarz wieloliniowy.
- Dobry komentarz wyjaśnia sens kodu.
- Zbędny komentarz tylko powtarza oczywistą instrukcję.
- Nazwa zmiennej powinna mówić, co zmienna przechowuje.
- Dla zmiennych lokalnych w C# zwykle stosujemy camelCase.
- W kursie można używać polskich nazw, ale trzeba robić to konsekwentnie.
- Czytelny kod jest ważny, bo kod czyta człowiek, nie tylko komputer.

Ćwiczenia

1. Dodaj komentarz jednoliniowy do programu.
2. Dodaj komentarz wieloliniowy przed fragmentem kodu.
3. Zmień nazwę zmiennej `x` na bardziej czytelną.
4. Popraw formatowanie przykładowego kodu.
5. Wskaż, który komentarz jest zbędny.
6. Napisz program z trzema zmiennymi o czytelnych nazwach.
7. Zmień nazwę zmiennej `liczba_punktow` na zgodną z camelCase.
8. Wskaż, która nazwa jest lepsza: `lp` czy `liczbaPunktow`.
9. Napisz trzy przykłady nazw zmiennych w camelCase.
10. Sprawdź, czy w jednym przykładzie nie mieszasz bez potrzeby nazw polskich i angielskich.

11. Operatory

01-podstawy/06-operatory.md

Operatory w C#

Cel lekcji

Poznasz podstawowe operatory w C# i nauczysz się używać ich w prostych programach konsolowych.

Po tej lekcji powinieneś umieć:

- wykonać proste działania arytmetyczne,
- odróżnić przypisanie = od porównania ==,
- obliczyć resztę z dzielenia,
- zwiększyć lub zmniejszyć wartość zmiennej,
- sprawdzić wynik porównania jako wartość true albo false.

1. Czym jest operator

Operator to znak lub zapis, który wykonuje działanie na wartościach.

Przykłady operatorów:

- + dodaje wartości,
- = przypisuje wartość do zmiennej,
- == sprawdza, czy dwie wartości są równe,
- > sprawdza, czy jedna wartość jest większa od drugiej.

Operatory są bardzo często używane w obliczeniach, porównaniach i warunkach.

2. Operatory arytmetyczne

Operatory arytmetyczne służą do wykonywania działań matematycznych.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int a = 10;
        int b = 3;

        Console.WriteLine(a + b);
        Console.WriteLine(a - b);
        Console.WriteLine(a * b);
        Console.WriteLine(a / b);
        Console.WriteLine(a % b);
    }
}
```

Znaczenie operatorów:

- + dodawanie,
- - odejmowanie,

- * mnożenie,
- / dzielenie,
- % reszta z dzielenia.

Operator % zwraca resztę z dzielenia. Dla $10 \% 3$ wynikiem jest 1, ponieważ 10 podzielone przez 3 daje 3 i resztę 1.

3. Dzielenie całkowite i rzeczywiste

W C# wynik dzielenia zależy od typu danych.

Jeżeli dzielimy liczby typu `int`, wynik też jest liczbą całkowitą.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int a = 10;
        int b = 3;

        Console.WriteLine(a / b);
    }
}
```

Wynik:

Kod: csharp

```
3
```

Program nie wypisze 3.3333, ponieważ `a` i `b` są typu `int`.

Jeżeli chcemy wynik rzeczywisty, możemy użyć typu `double`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        double x = 10.0;
        double y = 3.0;

        Console.WriteLine(x / y);
    }
}
```

Wtedy wynik zawiera część dziesiętną.

4. Operator przypisania

Znak `=` oznacza przypisanie wartości do zmiennej.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int punkty = 10;
    }
}
```

```
        Console.WriteLine(punkty);
    }
}
```

Instrukcję `int punkty = 10;` czytamy jako: utwórz zmienną `punkty` i wpisz do niej wartość 10.

Ważne: `=` nie oznacza porównania. Do porównania równości używamy `==`.

5. Skrócone operatory przypisania

Skrócone operatory przypisania pozwalają krócej zapisać zmianę wartości zmiennej.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int punkty = 10;

        punkty += 5;
        Console.WriteLine(punkty);

        punkty -= 3;
        Console.WriteLine(punkty);

        punkty *= 2;
        Console.WriteLine(punkty);

        punkty /= 4;
        Console.WriteLine(punkty);
    }
}
```

Znaczenie zapisów:

- `punkty += 5;` oznacza `punkty = punkty + 5;`,
- `punkty -= 3;` oznacza `punkty = punkty - 3;`,
- `punkty *= 2;` oznacza `punkty = punkty * 2;`,
- `punkty /= 4;` oznacza `punkty = punkty / 4;`.

6. Inkrementacja i dekrementacja

Inkrementacja zwiększa wartość zmiennej o 1.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int licznik = 5;

        licznik++;

        Console.WriteLine(licznik);
    }
}
```

Dekrementacja zmniejsza wartość zmiennej o 1.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int licznik = 5;

        licznik--;

        Console.WriteLine(licznik);
    }
}
```

Znaczenie:

- `licznik++` oznacza zwiększenie o 1,
- `licznik--` oznacza zmniejszenie o 1.

7. Operatory porównania

Operatory porównania sprawdzają relację między wartościami.

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        int a = 10;
        int b = 3;

        Console.WriteLine(a == b);
        Console.WriteLine(a != b);
        Console.WriteLine(a > b);
        Console.WriteLine(a < b);
        Console.WriteLine(a >= b);
        Console.WriteLine(a <= b);
    }
}
```

Znaczenie operatorów:

- `==` równe,
- `!=` różne,
- `>` większe,
- `<` mniejsze,
- `>=` większe lub równe,
- `<=` mniejsze lub równe.

Do porównania równości używamy `==`, a nie `=`.

8. Wynik porównania jako bool

Porównania zwracają wartość typu `bool`, czyli `true` albo `false`.

Kod: **csharp**

```
using System;

class Program
{
```

```

static void Main()
{
    int wiek = 18;

    Console.WriteLine(wiek >= 18);
    Console.WriteLine(wiek < 18);
    Console.WriteLine(5 == 5);
    Console.WriteLine(5 != 5);
}
}

```

Wartość `true` oznacza, że porównanie jest prawdziwe. Wartość `false` oznacza, że porównanie jest fałszywe.

9. Operatory logiczne

Operatory logiczne będą dokładniej używane w dziale o warunkach. Na razie wystarczy znać ich znaczenie.

- `&&` oznacza logiczne „i”.
- `||` oznacza logiczne „lub”.
- `!` oznacza zaprzeczenie.

Krótki przykład:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        bool maBilet = true;
        bool maLegitymacje = false;

        Console.WriteLine(maBilet && maLegitymacje);
        Console.WriteLine(maBilet || maLegitymacje);
        Console.WriteLine(!maBilet);
    }
}

```

To jest tylko zapowiedź. Do operatorów logicznych wrócimy przy instrukcjach warunkowych.

10. Kolejność wykonywania działań

C# stosuje znaną z matematyki kolejność wykonywania działań.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine(2 + 3 * 4);
        Console.WriteLine((2 + 3) * 4);
    }
}

```

Najpierw wykonywane jest mnożenie, a potem dodawanie. Nawiasy pozwalają zmienić kolejność działań.

Warto używać nawiasów, gdy zapis ma być bardziej czytelny.

11. Proste przykłady z Console.WriteLine()

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int a = 10;
        int b = 3;

        Console.WriteLine("Suma: " + (a + b));
        Console.WriteLine("Różnica: " + (a - b));
        Console.WriteLine("Iloczyn: " + (a * b));
        Console.WriteLine("Iloraz całkowity: " + (a / b));
        Console.WriteLine("Reszta z dzielenia: " + (a % b));
        Console.WriteLine("Czy a jest większe od b?");
        Console.WriteLine(a > b);
    }
}
```

Nawiasy w zapisie "Suma: " + (a + b) sprawiają, że najpierw zostanie obliczona suma, a dopiero potem wynik zostanie połączony z tekstem.

Typowe błędy

- Użycie = zamiast == przy porównaniu.
- Oczekiwanie wyniku rzeczywistego przy dzieleniu dwóch liczb typu int.
- Mylenie operatora % z dzieleniem.
- Brak nawiasów w dłuższych działaniach.
- Próba użycia ++ albo -- na wartości, która nie jest zmienną.
- Mylenie wartości true i false z tekstem "true" i "false".

Zapamiętaj

- Operator wykonuje działanie na wartościach.
- +, -, *, /, % to operatory arytmetyczne.
- % zwraca resztę z dzielenia.
- 10 / 3 dla typu int daje wynik całkowity.
- 10.0 / 3.0 dla typu double daje wynik rzeczywisty.
- = oznacza przypisanie.
- == oznacza porównanie równości.
- +=, -=, *=, /= skracają zapis zmiany wartości zmiennej.
- ++ zwiększa zmienną o 1, a -- zmniejsza ją o 1.
- Porównania zwracają true albo false.
- Nawiasy mogą zmienić kolejność wykonywania działań.

Ćwiczenia

1. Utwórz dwie zmienne typu int i wypisz ich sumę.
2. Wypisz różnicę, iloczyn i iloraz dwóch liczb.
3. Sprawdź resztę z dzielenia liczby przez 2.

4. Zwiększ wartość zmiennej o 1 za pomocą ++.
5. Zwiększ wartość zmiennej o 10 za pomocą +=.
6. Sprawdź, czy jedna liczba jest większa od drugiej.
7. Sprawdź, jaki wynik daje porównanie `5 == 5`.
8. Sprawdź, czym różni się `=` od `==`.

12. Wprowadzenie

02-wejscie-wyjście/README.md

02 - Wejście i wyjście

W tym dziale poznasz pobieranie danych od użytkownika i przygotowanie wyników do wyświetlenia.

Lekcje

- Console.ReadLine (01-console-readline.md)
- Konwersja typów (02-konwersja-typow.md)
- Formatowanie wyników (03-formatowanie-wynikow.md)
- Podsumowanie działu (04-podsumowanie.md)

Po tym dziale

Uczeń powinien umieć pobrać tekst z klawiatury, zamienić go na liczbę i wypisać czytelny wynik.

13. Console.ReadLine

02-wejscie-wyjscie/01-console-readline.md

Wczytywanie tekstu z konsoli - Console.ReadLine()

Cel lekcji

Nauczysz się wczytywać tekst wpisany przez użytkownika w konsoli za pomocą `Console.ReadLine()`.

Po tej lekcji powinieneś umieć:

- odróżnić wejście programu od wyjścia programu,
- wypisać komunikat dla użytkownika,
- wczytać tekst z konsoli,
- zapisać wczytany tekst do zmiennej typu `string`,
- wypisać wczytaną wartość.

1. Czym jest wejście programu

Wejście programu to dane, które program otrzymuje z zewnątrz.

W programie konsolowym wejściem może być tekst wpisany przez użytkownika z klawiatury.

Przykład: użytkownik wpisuje swoje imię, nazwę miasta albo nazwę szkoły.

2. Czym jest wyjście programu

Wyjście programu to dane, które program pokazuje użytkownikowi.

W programie konsolowym wyjściem jest najczęściej tekst wypisany w konsoli.

Do wypisywania danych używamy `Console.WriteLine()`.

3. Przypomnienie Console.WriteLine()

`Console.WriteLine()` wypisuje dane w konsoli i przechodzi do nowej linii.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("To jest komunikat dla użytkownika.");
    }
}
```

W tej lekcji `Console.WriteLine()` będzie często używane do pokazania użytkownikowi, co ma wpisać.

4. Wczytywanie danych za pomocą Console.ReadLine()

`Console.ReadLine()` wczytuje tekst wpisany przez użytkownika.

Program czeka, aż użytkownik:

- wpisze tekst,

- naciśnięcie Enter.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj swoje imię:");
        string imie = Console.ReadLine();

        Console.WriteLine("Witaj, " + imie + "!");
    }
}
```

W tym przykładzie:

- program najpierw wypisuje komunikat,
- użytkownik wpisuje imię,
- `Console.ReadLine()` wczytuje wpisany tekst,
- tekst zostaje zapisany w zmiennej `imie`,
- program wypisuje powitanie.

5. `Console.ReadLine()` zwraca `string`

`Console.ReadLine()` zawsze zwraca tekst typu `string`.

Nawet jeśli użytkownik wpisze:

Kod: csharp

```
123
```

program na tym etapie traktuje to jako tekst "123", a nie jako liczbę.

Do obliczeń liczbowych potrzebna będzie konwersja. Konwersję omówimy w kolejnej lekcji.

6. Zapisanie wczytanego tekstu do zmiennej

Wczytany tekst najczęściej zapisujemy do zmiennej typu `string`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj nazwę szkoły:");
        string szkola = Console.ReadLine();

        Console.WriteLine("Twoja szkoła to: " + szkola);
    }
}
```

Zmienna `szkola` przechowuje tekst wpisany przez użytkownika.

7. Wypisanie wczytanej wartości

Po wczytaniu tekstu można go wypisać w konsoli.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj ulubiony język programowania:");
        string jezyk = Console.ReadLine();

        Console.WriteLine("Wybrany język: " + jezyk);
    }
}
```

Wartość zmiennej jezyk pochodzi od użytkownika.

8. Prosty dialog z użytkownikiem

Program może prowadzić prosty dialog z użytkownikiem.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj swoje imię:");
        string imie = Console.ReadLine();

        Console.WriteLine("Podaj nazwę miasta:");
        string miasto = Console.ReadLine();

        Console.WriteLine("Imię: " + imie);
        Console.WriteLine("Miasto: " + miasto);
    }
}
```

To nadal są dane tekstowe. Program nie wykonuje na nich obliczeń.

9. Komunikat dla użytkownika a wczytanie odpowiedzi

Komunikat dla użytkownika i wczytanie odpowiedzi to dwie różne instrukcje.

Mniej czytelny przykład:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        string imie = Console.ReadLine();

        Console.WriteLine("Witaj, " + imie + "!");
    }
}
```

Ten program zadziała, ale użytkownik może nie wiedzieć, co ma wpisać.

Lepszy przykład:

Kod: csharp

```
using System;
```

```

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj swoje imię:");
        string imie = Console.ReadLine();

        Console.WriteLine("Witaj, " + imie + "!");
    }
}

```

Jeżeli przed `Console.ReadLine()` nie wypiszemy komunikatu, użytkownik może nie wiedzieć, co ma wpisać.

10. Console.WriteLine() i Console.ReadLine()

Najważniejsza różnica:

- `Console.WriteLine()` wypisuje dane,
- `Console.ReadLine()` wczytuje tekst.

Przykład:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj ulubiony przedmiot:");
        string przedmiot = Console.ReadLine();

        Console.WriteLine("Ulubiony przedmiot: " + przedmiot);
    }
}

```

Pierwsze `Console.WriteLine()` pokazuje pytanie. `Console.ReadLine()` czeka na odpowiedź. Drugie `Console.WriteLine()` wypisuje wynik.

Typowe błędy

- Brak komunikatu przed `Console.ReadLine()`.
- Mylenie `Console.WriteLine()` z `Console.ReadLine()`.
- Oczekiwanie, że `Console.ReadLine()` od razu zwraca liczbę.
- Próba wykonywania obliczeń na tekście bez konwersji.
- Zapomnienie o zapisaniu wyniku `Console.ReadLine()` do zmiennej.

Zapamiętaj

- Wejście programu to dane podane przez użytkownika.
- Wyjście programu to dane pokazane użytkownikowi.
- `Console.WriteLine()` wypisuje dane.
- `Console.ReadLine()` wczytuje tekst.
- `Console.ReadLine()` czeka na wpisanie tekstu i naciśnięcie Enter.
- Wynik `Console.ReadLine()` ma typ `string`.
- Nawet wpisane 123 jest na tym etapie tekstem.

- Do obliczeń liczbowych potrzebna będzie konwersja, którą omówimy w następnej lekcji.
- Przed wczytaniem danych warto wypisać komunikat dla użytkownika.

Ćwiczenia

1. Wczytaj imię użytkownika i wypisz powitanie.
2. Wczytaj nazwę miasta i wypisz ją w zdaniu.
3. Wczytaj nazwę szkoły i wypisz komunikat.
4. Wczytaj ulubiony przedmiot użytkownika.
5. Wczytaj dwie informacje tekstowe i wypisz je w dwóch liniach.
6. Sprawdź, co się stanie, gdy wpiszesz liczbę zamiast tekstu.
7. Dodaj komunikat przed każdym `Console.ReadLine()`.
8. Popraw program, w którym użytkownik nie wie, co ma wpisać.

14. Konwersja typów

02-wejscie-wyjscie/02-konwersja-typow.md

Konwersja typów po Console.ReadLine()

Cel lekcji

Nauczysz się zamieniać tekst wczytany z konsoli na liczby, aby można było wykonać proste obliczenia.

Po tej lekcji powinieneś umieć:

- wyjaśnić, dlaczego `Console.ReadLine()` zwraca `string`,
- zamienić tekst na liczbę całkowitą za pomocą `int.Parse()`,
- zamienić tekst na liczbę rzeczywistą za pomocą `double.Parse()`,
- użyć `Convert.ToInt32()` i `Convert.ToDouble()`,
- wykonać proste obliczenia na liczbach wczytanych z konsoli.

1. Przypomnienie: `Console.ReadLine()` zwraca `string`

`Console.ReadLine()` zawsze zwraca tekst typu `string`.

Nawet jeśli użytkownik wpisze:

Kod: `csharp`

```
123
```

program najpierw traktuje tę wartość jako tekst "123", a nie jako liczbę 123.

Jeżeli chcemy wykonać obliczenia, musimy przekonwertować tekst na odpowiedni typ liczbowy.

2. Dlaczego nie można od razu liczyć na tekście

Tekst "123" i liczba 123 to dla programu różne wartości.

Przykład błędu logicznego bez konwersji:

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        string a = "10";
        string b = "20";

        Console.WriteLine(a + b);
    }
}
```

Program wypisze:

Kod: `csharp`

```
1020
```

To nie jest dodawanie liczb. To tekstowe sklejenie wartości "10" i "20".

Aby otrzymać wynik 30, potrzebne są liczby, a nie teksty.

3. Konwersja tekstu na int

`int.Parse()` zamienia poprawny tekst liczbowy na liczbę całkowitą typu `int`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj swój wiek:");
        string tekst = Console.ReadLine();

        int wiek = int.Parse(tekst);

        Console.WriteLine("Twój wiek to: " + wiek);
    }
}
```

W tym przykładzie są dwa etapy:

- wczytanie tekstu do zmiennej `tekst`,
- konwersja tekstu na liczbę `int`.

Na początku kursu taki zapis jest czytelny, bo pokazuje dokładnie, co dzieje się w programie.

4. Krótszy zapis z `int.Parse()`

Można też zapisać wczytanie i konwersję w jednej instrukcji.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        Console.WriteLine("Podana liczba: " + liczba);
    }
}
```

Ten zapis jest poprawny, ale na początku kursu dłuższy zapis bywa czytelniejszy, bo pokazuje dwa etapy:

- wczytanie tekstu,
- konwersję tekstu na liczbę.

5. Dodawanie dwóch liczb

Po konwersji można wykonywać działania matematyczne.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj pierwszą liczbę:");
    }
}
```

```

        int a = int.Parse(Console.ReadLine());

        Console.WriteLine("Podaj drugą liczbę:");
        int b = int.Parse(Console.ReadLine());

        int suma = a + b;

        Console.WriteLine("Suma: " + suma);
    }
}

```

Zmienne a i b są typu int, dlatego operator + dodaje liczby.

6. Konwersja tekstu na double

`double.Parse()` zamienia poprawny tekst liczbowy na liczbę rzeczywistą typu `double`.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj wzrost w metrach:");
        double wzrost = double.Parse(Console.ReadLine());

        Console.WriteLine("Wzrost: " + wzrost);
    }
}

```

Typ `double` stosujemy do liczb z częścią dziesiętną, na przykład wzrostu, ceny albo średniej.

Uwaga praktyczna: w zależności od ustawień systemu liczby rzeczywiste mogą wymagać przecinka albo kropki jako separatora dziesiętnego. Na polskich ustawieniach regionalnych często używany jest przecinek, na przykład 1,75.

7. Konwersja za pomocą Convert

Do konwersji można użyć także klasy `Convert`.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę punktów:");
        int punkty = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("Liczba punktów: " + punkty);
    }
}

```

Drugi przykład:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {

```



```

        Console.WriteLine("Podaj cenę produktu:");
        double cena = Convert.ToDouble(Console.ReadLine());

        Console.WriteLine("Cena produktu: " + cena);
    }
}

```

Najważniejsze metody:

- `Convert.ToInt32()` zamienia wartość na liczbę całkowitą,
- `Convert.ToDouble()` zamienia wartość na liczbę rzeczywistą.

8. Parse i Convert

`Parse` i `Convert` służą do konwersji danych.

Na tym etapie możesz zapamiętać proste rozróżnienie:

- `int.Parse()` i `double.Parse()` jasno pokazują, na jaki typ zamieniamy tekst,
- `Convert.ToInt32()` i `Convert.ToDouble()` są alternatywnym sposobem konwersji.

W prostych programach konsolowych oba sposoby mogą być używane. Ważne jest, aby rozumieć, że przed obliczeniami tekst z `Console.ReadLine()` trzeba zamienić na liczbę.

9. Typowe błędy przy konwersji

Program może zakończyć się błędem, jeśli użytkownik wpisze dane w złym formacie.

Typowe sytuacje:

- użytkownik wpisze tekst zamiast liczby, na przykład `a1a`,
- użytkownik zostawi pustą wartość i naciśnie `Enter`,
- użytkownik wpisze liczbę rzeczywistą z separatorem niezgodnym z ustawieniami systemu.

Przykład:

Kod: `csharp`

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        Console.WriteLine("Liczba: " + liczba);
    }
}

```

Jeśli użytkownik wpisze `abc`, program nie będzie mógł zamienić tego tekstu na `int`.

10. TryParse jako bezpieczniejszy sposób

`TryParse` pozwala bezpieczniej sprawdzić, czy konwersja się udała.

Na razie traktujemy `TryParse` tylko jako zapowiedź. Szczegółowo wrócimy do niego później, gdy będziemy omawiać sprawdzanie poprawności danych.

11. Proste obliczenia po konwersji

Po zamianie tekstu na liczby można wykonać obliczenia.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj długość boku kwadratu:");
        int bok = int.Parse(Console.ReadLine());

        int obwod = 4 * bok;

        Console.WriteLine("Obwód kwadratu: " + obwod);
    }
}
```

W tym przykładzie zmienna bok jest liczbą typu int, dlatego można obliczyć obwód kwadratu.

Typowe błędy

- Wykonywanie obliczeń na tekście zamiast na liczbach.
- Zapominanie, że Console.ReadLine() zwraca string.
- Wpisanie tekstu tam, gdzie program oczekuje liczby.
- Wpisanie pustej wartości.
- Użycie złego separatora dziesiętnego przy liczbie typu double.
- Mylenie int.Parse() z double.Parse().
- Oczekiwanie, że Console.ReadLine() samodzielnie rozpozna typ liczby.

Zapamiętaj

- Console.ReadLine() zawsze zwraca string.
- Tekst "123" nie jest tym samym co liczba 123.
- int.Parse() zamienia poprawny tekst liczbowy na int.
- double.Parse() zamienia poprawny tekst liczbowy na double.
- Convert.ToInt32() i Convert.ToDouble() są alternatywnym sposobem konwersji.
- Jeśli użytkownik wpisze dane w złym formacie, program może zakończyć się błędem.
- TryParse pozwala bezpieczniej sprawdzić, czy konwersja się udała, ale dokładniej omówimy to później.
- Przed wykonaniem obliczeń dane tekstowe trzeba przekonwertować na liczby.

Ćwiczenia

1. Wczytaj wiek użytkownika jako int i wypisz go.
2. Wczytaj dwie liczby całkowite i wypisz ich sumę.
3. Wczytaj dwie liczby całkowite i wypisz ich różnicę.
4. Wczytaj długość boku kwadratu i oblicz jego obwód.
5. Wczytaj cenę produktu jako double i wypisz ją.
6. Wczytaj dwie liczby typu double i wypisz ich średnią.
7. Sprawdź, co się stanie, gdy zamiast liczby wpiszesz tekst.
8. Sprawdź, co wypisze program, gdy połączysz string "10" i string "20".

9. Przepisz jeden przykład z `int.Parse()` na `Convert.ToInt32()`.
10. Wskaż, które dane trzeba przekonwertować przed wykonaniem obliczeń.

15. Formatowanie wyników

02-wejscie-wyjscie/03-formatowanie-wynikow.md

Formatowanie wyników w konsoli

Cel lekcji

Nauczysz się czytelnie wypisywać dane w konsoli, łączyć tekst ze zmiennymi oraz formatować liczby rzeczywiste.

Po tej lekcji powinieneś umieć:

- połączyć tekst z wartością zmiennej za pomocą operatora +,
- użyć interpolacji napisów,
- wypisać kilka zmiennych w jednym zdaniu,
- ograniczyć liczbę miejsc po przecinku przy wypisywaniu wyniku,
- odróżnić wartość zmiennej od sposobu jej pokazania na ekranie.

1. Dlaczego sposób wypisywania wyniku jest ważny

Program powinien wypisywać wyniki tak, aby użytkownik wiedział, co oznaczają.

Mniej czytelny wynik:

Kod: csharp

```
17
```

Bardziej czytelny wynik:

Kod: csharp

```
Liczba punktów: 17
```

Drugi komunikat jest lepszy, ponieważ od razu wiadomo, czego dotyczy liczba.

2. Przypomnienie Console.WriteLine()

Console.WriteLine() wypisuje dane w konsoli i przechodzi do nowej linii.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Wynik programu:");
        Console.WriteLine(25);
    }
}
```

W tej lekcji skupiamy się na tym, jak przygotować czytelny tekst do wypisania.

3. Łączenie tekstu i zmiennych operatorem +

Operator + może łączyć tekst z wartością zmiennej.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        int wiek = 16;

        Console.WriteLine("Imię: " + imie);
        Console.WriteLine("Wiek: " + wiek);
        Console.WriteLine("Uczeń: " + imie + ", wiek: " + wiek);
    }
}
```

Ten zapis jest poprawny, ale przy dłuższych zdaniach może stać się mało czytelny.

4. Interpolacja napisów

Interpolacja napisów pozwala wygodnie wstawiać wartości zmiennych do tekstu.

Interpolowany napis zaczyna się od znaku \$ przed cudzysłowem.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        int wiek = 16;

        Console.WriteLine($"Uczeń: {imie}, wiek: {wiek}");
    }
}
```

Zmienne w interpolowanym tekście zapisujemy w nawiasach klamrowych {}.

5. Dlaczego interpolacja jest czytelniejsza

Porównaj dwa zapisy.

Łączenie operatorem +:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        int wiek = 16;

        Console.WriteLine("Uczeń: " + imie + ", wiek: " + wiek);
    }
}
```

Interpolacja napisów:

Kod: csharp

```
using System;

class Program
```

```

{
    static void Main()
    {
        string imie = "Anna";
        int wiek = 16;

        Console.WriteLine($"Uczeń: {imie}, wiek: {wiek}");
    }
}

```

Drugi zapis jest zwykle czytelniejszy, bo zdanie wygląda prawie tak samo jak tekst, który zobaczy użytkownik.

6. Wypisywanie kilku zmiennych w jednym zdaniu

Interpolacja dobrze sprawdza się, gdy chcemy wypisać kilka wartości naraz.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        string szkola = "Technikum";
        int punkty = 25;

        Console.WriteLine($"{imie} uczy się w szkole: {szkola}.");
        Console.WriteLine($"{imie} zdobyła {punkty} punktów.");
    }
}

```

W nawiasach klamrowych można umieszczać nazwy zmiennych.

7. Wynik obliczenia w czytelnym formacie

Możemy wypisać działanie i jego wynik w jednym zdaniu.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int a = 10;
        int b = 3;

        int suma = a + b;

        Console.WriteLine($"{a} + {b} = {suma}");
    }
}

```

Dzięki temu użytkownik widzi nie tylko wynik, ale też działanie.

8. Formatowanie liczb rzeczywistych

Liczby typu double mogą mieć wiele miejsc po przecinku.

Kod: csharp

```

using System;

```

```

class Program
{
    static void Main()
    {
        double wynik = 10.0 / 3.0;

        Console.WriteLine($"Wynik: {wynik}");
        Console.WriteLine($"Wynik do 2 miejsc po przecinku: {wynik:F2}");
        Console.WriteLine($"Wynik do 1 miejsca po przecinku: {wynik:F1}");
        Console.WriteLine($"Wynik bez miejsc po przecinku: {wynik:F0}");
    }
}

```

Znaczenie zapisów:

- {wynik} wypisuje wartość w domyślny sposób,
- {wynik:F2} wypisuje liczbę z dwoma miejscami po przecinku,
- {wynik:F1} wypisuje liczbę z jednym miejscem po przecinku,
- {wynik:F0} wypisuje liczbę bez miejsc po przecinku.

9. Wartość zmiennej a sposób wypisania

Formatowanie nie musi zmieniać samej wartości zmiennej. Zmienia tylko sposób pokazania tej wartości na ekranie.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        double wynik = 10.0 / 3.0;

        Console.WriteLine($"Wynik dokładny: {wynik}");
        Console.WriteLine($"Wynik sformatowany: {wynik:F2}");
        Console.WriteLine($"Ta sama zmienna ponownie: {wynik}");
    }
}

```

Zmienna wynik nadal przechowuje swoją wartość. Format F2 wpływa na wypisanie, a nie na samą zmienną.

10. Średnia dwóch liczb

Poniższy przykład wczytuje dwie liczby typu double, oblicza średnią i wypisuje wynik z dokładnością do dwóch miejsc po przecinku.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj pierwszą liczbę:");
        double a = double.Parse(Console.ReadLine());

        Console.WriteLine("Podaj drugą liczbę:");
        double b = double.Parse(Console.ReadLine());

        double srednia = (a + b) / 2;
    }
}

```

```

        Console.WriteLine($"Średnia wynosi: {srednia:F2}");
    }
}

```

W zależności od ustawień regionalnych systemu wynik liczby rzeczywistej może być wypisany z przecinkiem albo kropką jako separatorem dziesiętnym.

11. Prosty przykład z polem prostokąta

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        double bokA = 4.5;
        double bokB = 3.2;

        double pole = bokA * bokB;

        Console.WriteLine($"Pole prostokąta wynosi {pole:F2}.");
    }
}

```

Taki komunikat jest czytelniejszy niż wypisanie samej liczby.

Typowe błędy

- Brak znaku \$ przed interpolowanym tekstem.
- Zapisanie nazwy zmiennej jako zwykłego tekstu, na przykład "imie" zamiast {imie}.
- Brak nawiasów klamrowych wokół zmiennej w interpolacji.
- Mylenie formatowania wyniku ze zmianą wartości zmiennej.
- Zbyt długi zapis z wieloma operatorami +, który trudno czytać.

Zapamiętaj

- Console.WriteLine() wypisuje dane w konsoli.
- Operator + może łączyć tekst z wartością zmiennej.
- Interpolacja napisów zaczyna się od znaku \$ przed cudzysłowem.
- Zmienne w interpolowanym tekście zapisujemy w nawiasach klamrowych {}.
- \$"Uczeń: {imie}" jest zwykle czytelniejsze niż długie łączenie tekstu za pomocą +.
- {wynik:F2} oznacza wypisanie liczby z dwoma miejscami po przecinku.
- {wynik:F1} oznacza wypisanie liczby z jednym miejscem po przecinku.
- {wynik:F0} oznacza wypisanie liczby bez miejsc po przecinku.
- Formatowanie zmienia sposób pokazania wartości, ale nie musi zmieniać samej wartości zmiennej.

Ćwiczenia

1. Wypisz imię i wiek użytkownika w jednym zdaniu.
2. Przepisz przykład z operatorem + na interpolację napisów.
3. Wypisz wynik działania w formacie: 5 + 3 = 8.
4. Oblicz pole prostokąta i wypisz wynik w czytelnym zdaniu.

5. Wczytaj cenę produktu i wypisz ją z dwoma miejscami po przecinku.
6. Wczytaj dwie liczby typu double i wypisz średnią z dokładnością do dwóch miejsc po przecinku.
7. Sprawdź różnicę między {wynik}, {wynik:F2} i {wynik:F0}.
8. Napisz trzy komunikaty dla użytkownika z użyciem interpolacji napisów.

16. Podsumowanie działu

02-wejscie-wyjscie/04-podsumowanie.md

Podsumowanie działu - wejście i wyjście w konsoli

Cel lekcji

Powtórzysz najważniejsze informacje z działu o wejściu i wyjściu w konsoli oraz zobaczysz jeden przykład łączący wczytywanie danych, konwersję, obliczenia i formatowanie wyniku.

Po tym podsumowaniu powinieneś umieć:

- wypisać komunikat dla użytkownika,
- wczytać tekst z konsoli,
- zamienić tekst na liczbę,
- wykonać proste obliczenie,
- wypisać wynik w czytelnej formie.

1. Console.WriteLine()

`Console.WriteLine()` wypisuje dane w konsoli i przechodzi do nowej linii.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Witaj w programie!");
    }
}
```

Używamy go do pokazywania komunikatów, pytań i wyników.

2. Console.ReadLine()

`Console.ReadLine()` wczytuje tekst wpisany przez użytkownika i czeka na naciśnięcie Enter.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj imię:");
        string imie = Console.ReadLine();

        Console.WriteLine("Witaj, " + imie + "!");
    }
}
```

`Console.ReadLine()` zawsze zwraca string.

3. Console.ReadLine() zwraca string

Nawet jeśli użytkownik wpisze 10, program najpierw traktuje tę wartość jako tekst "10".

Tekst "10" nie jest tym samym co liczba 10.

Jeżeli chcemy wykonać obliczenia, trzeba wykonać konwersję.

4. Konwersja na int

Do liczb całkowitych możemy użyć `int.Parse()` albo `Convert.ToInt32()`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę punktów:");
        int punkty = int.Parse(Console.ReadLine());

        Console.WriteLine("Punkty: " + punkty);
    }
}
```

Alternatywny zapis:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę punktów:");
        int punkty = Convert.ToInt32(Console.ReadLine());

        Console.WriteLine("Punkty: " + punkty);
    }
}
```

5. Konwersja na double

Do liczb rzeczywistych możemy użyć `double.Parse()` albo `Convert.ToDouble()`.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj cenę:");
        double cena = double.Parse(Console.ReadLine());

        Console.WriteLine("Cena: " + cena);
    }
}
```

Alternatywny zapis:

Kod: csharp

```
using System;

class Program
{
```

```

static void Main()
{
    Console.WriteLine("Podaj cenę:");
    double cena = Convert.ToDouble(Console.ReadLine());

    Console.WriteLine("Cena: " + cena);
}

```

Przy double trzeba pamiętać, że separator dziesiętny zależy od ustawień systemu. Na polskich ustawieniach często używany jest przecinek, na przykład 12,50.

6. TryParse

TryParse jest bezpieczniejszym sposobem konwersji, ponieważ pozwala sprawdzić, czy zamiana tekstu na liczbę się udała.

Na tym etapie wystarczy zapamiętać, że TryParse istnieje. Szczegółowo wrócimy do niego później, przy sprawdzaniu poprawności danych.

7. Interpolacja napisów

Interpolacja napisów pozwala czytelnie wstawiać wartości zmiennych do tekstu.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        string imie = "Anna";
        int punkty = 25;

        Console.WriteLine($"Uczeń: {imie}, punkty: {punkty}");
    }
}

```

Interpolowany tekst zaczyna się od \$, a zmienne zapisujemy w nawiasach klamrowych {}.

8. Formatowanie liczb

Format {wynik:F2} wypisuje liczbę z dwoma miejscami po przecinku.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        double wynik = 10.0 / 3.0;

        Console.WriteLine($"Wynik: {wynik:F2}");
    }
}

```

Formatowanie dotyczy sposobu wypisania wyniku. Nie musi zmieniać samej wartości zmiennej.

9. Przykład całościowy

Ten przykład łączy wypisywanie komunikatów, wczytywanie danych, konwersję typów, obliczenia i formatowanie wyniku.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj imię:");
        string imie = Console.ReadLine();

        Console.WriteLine("Podaj liczbę punktów:");
        int punkty = int.Parse(Console.ReadLine());

        Console.WriteLine("Podaj maksymalną liczbę punktów:");
        int maksimum = int.Parse(Console.ReadLine());

        double procent = (double)punkty / maksimum * 100;

        Console.WriteLine($"Uczeń: {imie}");
        Console.WriteLine($"Wynik: {punkty}/{maksimum}");
        Console.WriteLine($"Procent: {procent:F2}%");
    }
}
```

W obliczeniu procentu użyto (`double`), aby zamienić wartość punkty na liczbę rzeczywistą.

Bez tego można przypadkowo otrzymać dzielenie całkowite, czyli wynik bez części dziesiętnej.

Format `F2` dotyczy sposobu wypisania wyniku. Dzięki temu procent jest pokazany z dwoma miejscami po przecinku.

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B[Wypisz pytania]
    C[Wczytaj tekst]
    D[Zamień na liczby]
    E[Wykonaj obliczenia]
    F[Sformatuj wynik]
    G([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
```

Diagram pokazuje, że dane z konsoli najpierw są tekstem, a dopiero po konwersji można użyć ich w obliczeniach.

10. Typowe błędy

- Brak konwersji przed obliczeniami.
- Wpisanie tekstu tam, gdzie program oczekuje liczby.
- Pomylenie tekstu "10" z liczbą 10.

- Sklejenie tekstów "10" i "20" zamiast dodania liczb.
- Problem przecinka lub kropki przy liczbach typu double.
- Brak komunikatu przed `Console.ReadLine()`.
- Brak znaku \$ przy interpolacji napisów.

Najważniejsze do zapamiętania

- `Console.WriteLine()` wypisuje dane.
- `Console.ReadLine()` wczytuje tekst.
- `Console.ReadLine()` zwraca string.
- Przed obliczeniami tekst trzeba zamienić na liczbę.
- `int.Parse()` i `Convert.ToInt32()` służą do konwersji na int.
- `double.Parse()` i `Convert.ToDouble()` służą do konwersji na double.
- `TryParse` jest bezpieczniejszą konwersją, do której wrócimy później.
- Interpolacja ma zapis `$"Tekst {zmienna}"`.
- `{wynik:F2}` wypisuje liczbę z dwoma miejscami po przecinku.

Minićwiczenia powtórkowe

1. Wczytaj imię i wiek, a następnie wypisz je w jednym zdaniu.
2. Wczytaj dwie liczby całkowite i wypisz ich sumę.
3. Wczytaj dwie liczby typu double i wypisz średnią z dokładnością do dwóch miejsc po przecinku.
4. Wczytaj cenę netto i wypisz prosty komunikat z tą ceną.
5. Sprawdź, co się stanie, gdy do `int.Parse()` wpiszesz tekst.
6. Popraw program, który skleja teksty "10" i "20" zamiast dodać liczby.

17. Wprowadzenie

03-warunki/README.md

03 - Warunki

W tym dziale poznasz instrukcje warunkowe. Dzięki nim program może wykonać różne instrukcje zależnie od sytuacji.

Lekcje

- if (01-if.md)
- if else (02-if-else.md)
- else if (03-else-if.md)
- switch (04-switch.md)
- Podsumowanie działu (05-podsumowanie.md)

Po tym dziale

Uczeń powinien umieć sprawdzić warunek, wybrać jedną z kilku możliwości i zauważyć typowe błędy w warunkach.

18. if

03-warunki/01-if.md

Instrukcja warunkowa if

Cel lekcji

Nauczysz się używać instrukcji `if`, aby wykonać fragment programu tylko wtedy, gdy warunek ma wartość `true`.

Po tej lekcji powinieneś umieć:

- wyjaśnić, po co stosuje się instrukcje warunkowe,
- zapisać prostą instrukcję `if`,
- rozpoznać warunek dający wynik `true` albo `false`,
- wskazać, które instrukcje należą do bloku `if`,
- zauważyć typowe błędy, na przykład użycie `=` zamiast `==`.

1. Krótkie wprowadzenie

Program bez instrukcji warunkowych wykonuje instrukcje po kolei, od góry do dołu.

Instrukcja `if` pozwala wykonać fragment kodu tylko w określonej sytuacji. Decyzja zależy od warunku.

Warunek to wyrażenie, które daje wynik:

- `true` - warunek jest spełniony,
- `false` - warunek nie jest spełniony.

Jeżeli warunek w `if` ma wartość `true`, blok kodu zostanie wykonany. Jeżeli warunek ma wartość `false`, blok zostanie pominięty.

2. Składnia if

Schemat instrukcji `if`:

Kod: csharp

```
if (warunek)
{
    // instrukcje wykonywane, gdy warunek jest true
}
```

Najważniejsze elementy:

- warunek zapisujemy w nawiasach okrągłych `()`,
- instrukcje zależne od warunku zapisujemy w nawiasach klamrowych `{}`,
- jeśli warunek jest `false`, blok `if` zostaje pominięty.

3. Pierwszy przykład

Kod: csharp

```
using System;

class Program
{
```



```

static void Main()
{
    int wiek = 18;

    if (wiek >= 18)
    {
        Console.WriteLine("Osoba jest pełnoletnia.");
    }
}

```

Warunek `wiek >= 18` daje wynik `true`, ponieważ zmienna `wiek` ma wartość 18.

Dlatego komunikat zostanie wypisany.

4. Przykład z warunkiem niespełnionym

Kod: `csharp`

```

using System;

class Program
{
    static void Main()
    {
        int punkty = 30;

        if (punkty >= 50)
        {
            Console.WriteLine("Wynik jest wystarczający.");
        }

        Console.WriteLine("Koniec programu.");
    }
}

```

Warunek `punkty >= 50` daje wynik `false`, ponieważ 30 nie jest większe ani równe 50.

Blok `if` zostanie pominięty.

Komunikat `"Koniec programu."` wypisze się zawsze, bo znajduje się poza blokiem `if`.

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart TD
    A([Start]) --> B{"Czy warunek<br/>jest true?"}
    B -- "tak" --> C[Wykonaj blok if]
    B -- "nie" --> D[Pomiń blok if]
    C --> E[Dalszy kod]
    D --> E
    E --> F([Koniec])

```

Diagram pokazuje, że blok `if` wykonuje się tylko wtedy, gdy warunek ma wartość `true`.

5. Warunki z operatorami

Operatory porównania były już omawiane w lekcji o operatorach. W instrukcji `if` używamy ich do budowania warunków.

Przykłady warunków:

Kod: csharp

```
int x = 10;

x > 5
x < 5
x == 10
x != 10
x >= 10
x <= 7
x % 2 == 0
```

Warto pamiętać:

- `==` służy do porównania równości,
- `=` służy do przypisania wartości,
- `%` daje resztę z dzielenia,
- warunek `x % 2 == 0` sprawdza, czy liczba jest parzysta.

6. Zgadnij wynik warunku

Najpierw spójrz na przykłady z odpowiedziami.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int x = 10;

        Console.WriteLine(x > 5);        // true
        Console.WriteLine(x == 10);      // true
        Console.WriteLine(x != 10);      // false
        Console.WriteLine(x % 2 == 0);   // true
        Console.WriteLine(x < 0);        // false
    }
}
```

Teraz spróbuj samodzielnie przewidzieć wyniki.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int a = 7;
        int b = 12;

        Console.WriteLine(a > b);
        Console.WriteLine(a < b);
        Console.WriteLine(a + 5 == b);
        Console.WriteLine(b % 2 == 0);
        Console.WriteLine(a != 7);
    }
}
```

Odpowiedzi:

- `a > b` daje `false`,
- `a < b` daje `true`,
- `a + 5 == b` daje `true`,
- `b % 2 == 0` daje `true`,
- `a != 7` daje `false`.

7. Przykład z danymi z klawiatury - liczba dodatnia

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        if (liczba > 0)
        {
            Console.WriteLine("Liczba jest dodatnia.");
        }
    }
}
```

Komunikat zostanie wypisany tylko wtedy, gdy użytkownik poda liczbę większą od 0.

8. Przykład z parzystością

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        if (liczba % 2 == 0)
        {
            Console.WriteLine("Liczba jest parzysta.");
        }
    }
}
```

Operator `%` oblicza resztę z dzielenia.

Liczba parzysta przy dzieleniu przez 2 daje resztę 0.

Dlatego warunek `liczba % 2 == 0` sprawdza parzystość.

9. Przykład z progiem punktów

Kod: `csharp`

```
using System;

class Program
{
```

```

static void Main()
{
    Console.WriteLine("Podaj liczbę punktów:");
    int punkty = int.Parse(Console.ReadLine());

    if (punkty >= 50)
    {
        Console.WriteLine("Próg punktowy został osiągnięty.");
    }
}

```

Blok if wykona się tylko wtedy, gdy zmienna punkty ma wartość co najmniej 50.

10. Przykład z temperaturą

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj temperaturę:");
        double temperatura = double.Parse(Console.ReadLine());

        if (temperatura > 30)
        {
            Console.WriteLine("Uwaga: wysoka temperatura.");
        }
    }
}

```

Komunikat ostrzegawczy pojawi się tylko wtedy, gdy temperatura jest większa niż 30.

11. Typowe błędy

Błąd 1 - użycie = zamiast ==

Niepoprawnie:

Kod: csharp

```

if (liczba = 10)
{
    Console.WriteLine("Liczba wynosi 10.");
}

```

Poprawnie:

Kod: csharp

```

if (liczba == 10)
{
    Console.WriteLine("Liczba wynosi 10.");
}

```

= oznacza przypisanie wartości.

== oznacza porównanie równości.

Błąd 2 - instrukcja poza blokiem if

Kod: csharp

```

using System;

```

```

class Program
{
    static void Main()
    {
        int punkty = 30;

        if (punkty >= 50)
        {
            Console.WriteLine("Wynik jest wystarczający.");
        }

        Console.WriteLine("Ten komunikat wypisze się zawsze.");
    }
}

```

Tylko instrukcje w nawiasach klamrowych należą do bloku `if`.

Instrukcje poza blokiem wykonują się niezależnie od warunku.

Błąd 3 - brak nawiasów klamrowych przy kilku instrukcjach

W C# czasem można spotkać `if` bez nawiasów klamrowych, gdy zależna jest tylko jedna instrukcja.

W tym kursie dla czytelności zawsze stosujemy nawiasy klamrowe przy `if`, nawet jeśli w bloku jest jedna instrukcja.

Czytelny zapis:

Kod: csharp

```

if (punkty >= 50)
{
    Console.WriteLine("Wynik jest wystarczający.");
}

```

12. Zapowiedź kolejnej lekcji

W tej lekcji instrukcja `if` wykonuje kod tylko wtedy, gdy warunek jest spełniony.

W kolejnej lekcji pojawi się `if else`, czyli wybór jednej z dwóch możliwości.

Zapamiętaj

- `if` sprawdza warunek.
- Warunek daje wynik `true` albo `false`.
- Blok `if` wykonuje się tylko dla `true`.
- Do porównania równości używamy `==`.
- Do przypisania używamy `=`.
- `%` pozwala sprawdzić resztę z dzielenia.
- Kod poza blokiem `if` wykonuje się niezależnie od warunku.
- W tym kursie zapisujemy nawiasy klamrowe przy każdym `if`.

Ćwiczenia

1. Wczytaj wiek i wypisz komunikat, jeśli wiek jest większy lub równy 18.
2. Wczytaj liczbę i wypisz komunikat, jeśli liczba jest dodatnia.
3. Wczytaj liczbę i wypisz komunikat, jeśli liczba jest równa 0.
4. Wczytaj liczbę punktów i wypisz komunikat, jeśli wynik jest większy lub równy 50.

5. Wczytaj liczbę i sprawdź, czy jest parzysta.
6. Wczytaj temperaturę i wypisz ostrzeżenie, jeśli jest większa niż 30.
7. Dla zmiennej `int x = 15` określ, które warunki dają `true`:
 - `x > 10`
 - `x < 10`
 - `x == 15`
 - `x != 15`
 - `x % 2 == 0`
8. Popraw błędny zapis `z = na ==`.
9. Wskaż, które instrukcje znajdują się w bloku `if`, a które poza nim.
10. Dopisz własny przykład warunku, który sprawdza liczbę punktów.

19. if else

03-warunki/02-if-else.md

Instrukcja warunkowa if else

Cel lekcji

Nauczysz się używać instrukcji `if else`, aby wybrać jedną z dwóch ścieżek wykonania programu.

Po tej lekcji powinieneś umieć:

- wyjaśnić, kiedy używa się `if else`,
- zapisać blok `if` i blok `else`,
- wskazać, który blok wykona się dla warunku `true`,
- wskazać, który blok wykona się dla warunku `false`,
- rozpoznać typowe błędy przy `if else`.

1. Krótkie przypomnienie if

Instrukcja `if` wykonuje blok kodu tylko wtedy, gdy warunek daje wynik `true`.

Jeśli warunek daje wynik `false`, blok `if` jest pomijany.

Czasami chcemy wykonać inny kod, gdy warunek nie jest spełniony. Do tego służy `else`.

2. Składnia if else

Schemat:

Kod: `csharp`

```
if (warunek)
{
    // instrukcje wykonywane, gdy warunek jest true
}
else
{
    // instrukcje wykonywane, gdy warunek jest false
}
```

Najważniejsze informacje:

- `if` i `else` tworzą jeden wybór,
- wykona się dokładnie jeden z dwóch bloków,
- `else` nie ma własnego warunku,
- `else` oznacza „w przeciwnym razie”.

3. Pierwszy przykład - pełnoletniość

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj wiek:");
    }
}
```

```

        int wiek = int.Parse(Console.ReadLine());

        if (wiek >= 18)
        {
            Console.WriteLine("Osoba jest pełnoletnia.");
        }
        else
        {
            Console.WriteLine("Osoba nie jest pełnoletnia.");
        }
    }
}

```

Jeśli warunek `wiek >= 18` daje `true`, wykonana się pierwszy blok.

W przeciwnym razie wykonana się blok `else`.

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart TD
    A([Start]) --> B{"Czy warunek<br/>jest true?"}
    B -- "tak" --> C["Wykonaj blok if"]
    B -- "nie" --> D["Wykonaj blok else"]
    C --> E["Dalszy kod"]
    D --> E
    E --> F([Koniec])

```

Diagram pokazuje, że w instrukcji `if else` wykonuje się dokładnie jedna z dwóch ścieżek.

4. Liczba dodatnia albo niedodatnia

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        if (liczba > 0)
        {
            Console.WriteLine("Liczba jest dodatnia.");
        }
        else
        {
            Console.WriteLine("Liczba nie jest dodatnia.");
        }
    }
}

```

„Liczba nie jest dodatnia” oznacza, że jest równa 0 albo ujemna.

Dokładniejsze rozróżnienie kilku przypadków będzie możliwe przy `else if`.

5. Liczba parzysta albo nieparzysta

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        if (liczba % 2 == 0)
        {
            Console.WriteLine("Liczba jest parzysta.");
        }
        else
        {
            Console.WriteLine("Liczba jest nieparzysta.");
        }
    }
}
```

Warunek `liczba % 2 == 0` sprawdza, czy reszta z dzielenia przez 2 jest równa 0.

Jeśli warunek jest fałszywy, liczba jest nieparzysta.

6. Próg punktów

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę punktów:");
        int punkty = int.Parse(Console.ReadLine());

        if (punkty >= 50)
        {
            Console.WriteLine("Próg został osiągnięty.");
        }
        else
        {
            Console.WriteLine("Próg nie został osiągnięty.");
        }
    }
}
```

Program wybiera jeden z dwóch komunikatów zależnie od liczby punktów.

7. Proste hasło

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj hasło:");
        string haslo = Console.ReadLine();
    }
}
```

```

        if (haslo == "admin")
        {
            Console.WriteLine("Hasło poprawne.");
        }
        else
        {
            Console.WriteLine("Hasło błędne.");
        }
    }
}

```

To jest przykład dydaktyczny. W prawdziwych programach haseł nie zapisuje się bezpośrednio w kodzie programu.

8. Zgadnij, który blok się wykona

Przykład z odpowiedzią:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int x = 10;

        if (x > 5)
        {
            Console.WriteLine("A");
        }
        else
        {
            Console.WriteLine("B");
        }
    }
}

```

Odpowiedź: program wypisze A, ponieważ $x > 5$ daje true.

Drugi przykład z odpowiedzią:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int x = 3;

        if (x % 2 == 0)
        {
            Console.WriteLine("Parzysta");
        }
        else
        {
            Console.WriteLine("Nieparzysta");
        }
    }
}

```

Odpowiedź: program wypisze Nieparzysta, ponieważ $3 \% 2$ daje 1.

Spróbuj teraz przewidzieć wyniki samodzielnie:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int a = 7;
        int b = 12;

        if (a > b)
        {
            Console.WriteLine("Pierwszy");
        }
        else
        {
            Console.WriteLine("Drugi");
        }

        if (a + 5 == b)
        {
            Console.WriteLine("Równe");
        }
        else
        {
            Console.WriteLine("Różne");
        }

        if (b % 2 == 0)
        {
            Console.WriteLine("Parzyste");
        }
        else
        {
            Console.WriteLine("Nieparzyste");
        }
    }
}
```

Odpowiedzi:

- pierwszy przykład wypisze Drugi,
- drugi przykład wypisze Równe,
- trzeci przykład wypisze Parzyste.

9. If bez klamerek

W C# można zapisać if bez nawiasów klamrowych, jeśli zależna od warunku jest tylko jedna instrukcja.

Kod: csharp

```
if (wiek >= 18)
    Console.WriteLine("Pełnoletni");
```

W tym kursie stosujemy nawiasy klamrowe.

Zapis z klamrami jest czytelniejszy, łatwiej uniknąć błędów przy dopisywaniu kolejnych instrukcji, a na egzaminie i w nauce lepiej pisać jasno.

Wersja zalecana:

Kod: csharp

```
if (wiek >= 18)
{
    Console.WriteLine("Pełnoletni");
}
```

```
}
```

10. Operator warunkowy ?:

W C# istnieje skrócony zapis wyboru jednej z dwóch wartości.

Często bywa potocznie nazywany jednolinijkowym `if`, ale formalnie jest to operator warunkowy `?:`, a nie instrukcja `if`.

Nadaje się do prostych wyborów wartości. Nie powinien zastępować czytelnego `if else` w bardziej złożonym kodzie.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 20;

        string komunikat = wiek >= 18 ? "Pełnoletni" : "Niepełnoletni";

        Console.WriteLine(komunikat);
    }
}
```

Schemat:

Kod: csharp

```
warunek ? wartośćGdyTrue : wartośćGdyFalse
```

Jeśli warunek jest `true`, wybrana zostanie wartość po znaku `?`.

Jeśli warunek jest `false`, wybrana zostanie wartość po znaku `:`.

W tym kursie podstawowym zapisem pozostaje klasyczne `if else`.

11. Typowe błędy

Błąd 1 - `else` bez wcześniejszego `if`

Niepoprawnie:

Kod: csharp

```
else
{
    Console.WriteLine("Błąd");
}
```

`else` nie działa samodzielnie. Musi być powiązane z wcześniejszym `if`.

Błąd 2 - średnik po `if`

Niepoprawnie:

Kod: csharp

```
if (wiek >= 18);
{
    Console.WriteLine("Pełnoletni");
}
```

Średnik po `if` kończy instrukcję.

Blok poniżej nie działa wtedy tak, jak uczeń oczekuje.

W tym kursie nie stawiamy średnika po warunku `if`.

Błąd 3 - brak klamerek przy rozbudowanym bloku

W tym kursie konsekwentnie stosujemy nawiasy klamrowe przy `if` i `else`.

Kod: `csharp`

```
if (punkty >= 50)
{
    Console.WriteLine("Próg został osiągnięty.");
}
else
{
    Console.WriteLine("Próg nie został osiągnięty.");
}
```

Dzięki temu od razu widać, które instrukcje należą do którego bloku.

Błąd 4 - mylenie `=` i `==`

`=` przypisuje wartość.

`==` porównuje wartości.

Poprawnie:

Kod: `csharp`

```
if (haslo == "admin")
{
    Console.WriteLine("Hasło poprawne.");
}
else
{
    Console.WriteLine("Hasło błędne.");
}
```

12. Zapowiedź kolejnej lekcji

Instrukcja `if else` pozwala wybrać jedną z dwóch możliwości.

W kolejnej lekcji pojawi się `else if`, czyli wybór spośród wielu warunków.

Zapamiętaj

- `if else` pozwala wybrać jedną z dwóch ścieżek.
- Gdy warunek jest `true`, wykonuje się blok `if`.
- Gdy warunek jest `false`, wykonuje się blok `else`.
- `else` nie ma własnego warunku.
- `else` nie działa bez `if`.
- W kursie stosujemy nawiasy klamrowe.
- Operator `?:` służy do krótkiego wyboru wartości, ale nie zastępuje czytelnego `if else` w trudniejszych przykładach.

Ćwiczenia

1. Wczytaj wiek i wypisz, czy osoba jest pełnoletnia.
2. Wczytaj liczbę i wypisz, czy jest dodatnia, czy niedodatnia.

3. Wczytaj liczbę i wypisz, czy jest parzysta, czy nieparzysta.
4. Wczytaj liczbę punktów i wypisz, czy próg 50 punktów został osiągnięty.
5. Wczytaj hasło i wypisz, czy jest poprawne.
6. Dla zmiennej `int x = 8` określ, który blok się wykona przy warunku `x > 10`.
7. Popraw błędny zapis z `else` bez `if`.
8. Popraw błędny zapis ze średnikiem po `if`.
9. Przepisz prosty przykład `if else` na operator warunkowy `?:`.
10. Wskaż, kiedy lepiej użyć klasycznego `if else` zamiast operatora `?:`.

20. else if

03-warunki/03-else-if.md

Instrukcja else if - wiele warunków

Cel lekcji

Nauczysz się używać `else if`, aby sprawdzać kolejne warunki, gdy wcześniejsze warunki nie zostały spełnione.

Po tej lekcji powinieneś umieć:

- zapisać łańcuch `if / else if / else`,
- wyjaśnić, w jakiej kolejności sprawdzane są warunki,
- wskazać pierwszy pasujący blok,
- dobrać kolejność warunków przy progach liczbowych,
- odróżnić kilka osobnych `if` od jednego łańcucha wyboru.

1. Krótkie przypomnienie

`if` sprawdza jeden warunek.

`if else` wybiera jedną z dwóch możliwości.

`else if` pozwala sprawdzić kolejne warunki, gdy wcześniejsze warunki były fałszywe.

2. Składnia else if

Schemat:

Kod: `csharp`

```
if (warunek1)
{
    // instrukcje, gdy warunek1 jest true
}
else if (warunek2)
{
    // instrukcje, gdy warunek1 jest false, a warunek2 jest true
}
else
{
    // instrukcje, gdy żaden wcześniejszy warunek nie był true
}
```

Najważniejsze zasady:

- warunki są sprawdzane od góry do dołu,
- wykonuje się pierwszy blok, którego warunek jest `true`,
- pozostałe bloki są pomijane,
- `else` na końcu jest opcjonalne, ale często przydatne.

3. Liczba dodatnia, ujemna albo zero

Kod: `csharp`

```
using System;
```

```

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        if (liczba > 0)
        {
            Console.WriteLine("Liczba jest dodatnia.");
        }
        else if (liczba < 0)
        {
            Console.WriteLine("Liczba jest ujemna.");
        }
        else
        {
            Console.WriteLine("Liczba jest równa zero.");
        }
    }
}

```

Jeśli $liczba > 0$, wykonuje się pierwszy blok.

Jeśli nie, sprawdzany jest kolejny warunek.

Jeśli liczba nie jest ani dodatnia, ani ujemna, zostaje 0, więc wykonuje się blok else.

4. Próg punktów

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę punktów:");
        int punkty = int.Parse(Console.ReadLine());

        if (punkty >= 90)
        {
            Console.WriteLine("Wynik bardzo dobry.");
        }
        else if (punkty >= 75)
        {
            Console.WriteLine("Wynik dobry.");
        }
        else if (punkty >= 50)
        {
            Console.WriteLine("Wynik wystarczający.");
        }
        else
        {
            Console.WriteLine("Wynik niewystarczający.");
        }
    }
}

```

Kolejność warunków ma znaczenie.

Najpierw sprawdzamy najwyższy próg. Gdy punkty wynoszą 92, wykona się tylko pierwszy pasujący blok.

Program nie sprawdza dalej pozostałych else if po znalezieniu pasującego warunku.

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart TD
  A([Start])
  B{"punkty >= 90?"}
  C["Bardzo dobry"]
  D{"punkty >= 75?"}
  E["Dobry"]
  F{"punkty >= 50?"}
  G["Wystarczający"]
  H["Niewystarczający"]
  I([Koniec])

  A --> B
  B -- "tak" --> C
  B -- "nie" --> D
  D -- "tak" --> E
  D -- "nie" --> F
  F -- "tak" --> G
  F -- "nie" --> H
  C --> I
  E --> I
  G --> I
  H --> I
```

Diagram pokazuje, że `else if` sprawdza warunki po kolei i zatrzymuje się na pierwszym pasującym przypadku.

5. Temperatura

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj temperaturę:");
        double temperatura = double.Parse(Console.ReadLine());

        if (temperatura < 0)
        {
            Console.WriteLine("Mróz.");
        }
        else if (temperatura < 15)
        {
            Console.WriteLine("Chłodno.");
        }
        else if (temperatura < 25)
        {
            Console.WriteLine("Ciepło.");
        }
        else
        {
            Console.WriteLine("Gorąco.");
        }
    }
}
```

Warunki są sprawdzane po kolei.

Przy temperaturze 10 pierwszy warunek jest false, drugi warunek jest true, więc program wypisze Chłodno..

Po wykonaniu drugiego bloku reszta łańcucha jest pomijana.

6. Prosty przykład z zakresem i operatorem &&

Operator && oznacza „i”. Oba warunki muszą być true.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj ocenę:");
        int ocena = int.Parse(Console.ReadLine());

        if (ocena >= 1 && ocena <= 6)
        {
            Console.WriteLine("Ocena mieści się w szkolnym zakresie.");
        }
        else
        {
            Console.WriteLine("Ocena poza zakresem.");
        }
    }
}
```

Ten przykład tylko przypomina operator logiczny. Pełne użycie operatorów logicznych można rozwijać później.

7. Proste menu tekstowe

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Wybierz opcję:");
        Console.WriteLine("1 - Start");
        Console.WriteLine("2 - Pomoc");
        Console.WriteLine("3 - Wyjście");

        int opcja = int.Parse(Console.ReadLine());

        if (opcja == 1)
        {
            Console.WriteLine("Wybrano start.");
        }
        else if (opcja == 2)
        {
            Console.WriteLine("Wybrano pomoc.");
        }
        else if (opcja == 3)
        {
            Console.WriteLine("Wybrano wyjście.");
        }
        else
        {
            Console.WriteLine("Nieznana opcja.");
        }
    }
}
```

```
    }  
  }  
}
```

Do takich wyborów często dobrze pasuje instrukcja switch, którą poznamy w kolejnej lekcji.

8. Zgadnij, który blok się wykona

Przykład z odpowiedzią:

Kod: csharp

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int punkty = 80;  
  
        if (punkty >= 90)  
        {  
            Console.WriteLine("A");  
        }  
        else if (punkty >= 75)  
        {  
            Console.WriteLine("B");  
        }  
        else if (punkty >= 50)  
        {  
            Console.WriteLine("C");  
        }  
        else  
        {  
            Console.WriteLine("D");  
        }  
    }  
}
```

Odpowiedź: program wypisze B, ponieważ punkty ≥ 75 jest pierwszym spełnionym warunkiem.

Spróbuj przewidzieć wyniki kolejnych przykładów:

Kod: csharp

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int x = 0;  
  
        if (x > 0)  
        {  
            Console.WriteLine("Dodatnia");  
        }  
        else if (x < 0)  
        {  
            Console.WriteLine("Ujemna");  
        }  
        else  
        {  
            Console.WriteLine("Zero");  
        }  
  
        int temperatura = 20;  
  
        if (temperatura < 0)
```

```

    {
        Console.WriteLine("Mróz");
    }
    else if (temperatura < 15)
    {
        Console.WriteLine("Chłodno");
    }
    else if (temperatura < 25)
    {
        Console.WriteLine("Ciepło");
    }
    else
    {
        Console.WriteLine("Gorąco");
    }

    int opcja = 4;

    if (opcja == 1)
    {
        Console.WriteLine("Start");
    }
    else if (opcja == 2)
    {
        Console.WriteLine("Pomoc");
    }
    else if (opcja == 3)
    {
        Console.WriteLine("Wyjście");
    }
    else
    {
        Console.WriteLine("Nieznana opcja");
    }
}
}

```

Odpowiedzi:

- pierwszy przykład wypisze Zero,
- drugi przykład wypisze Ciepło,
- trzeci przykład wypisze Nieznana opcja.

9. Typowe błędy

Błąd 1 - zła kolejność progów

Niepoprawnie:

Kod: csharp

```

if (punkty >= 50)
{
    Console.WriteLine("Wynik wystarczający.");
}
else if (punkty >= 75)
{
    Console.WriteLine("Wynik dobry.");
}
else if (punkty >= 90)
{
    Console.WriteLine("Wynik bardzo dobry.");
}

```

Dla punktów 95 pierwszy warunek punkty >= 50 jest już true.

Kolejne warunki nie zostaną sprawdzone.

Dlatego przy progach od góry trzeba zaczynać od najwyższego progu.

Poprawnie:

Kod: csharp

```
if (punkty >= 90)
{
    Console.WriteLine("Wynik bardzo dobry.");
}
else if (punkty >= 75)
{
    Console.WriteLine("Wynik dobry.");
}
else if (punkty >= 50)
{
    Console.WriteLine("Wynik wystarczający.");
}
else
{
    Console.WriteLine("Wynik niewystarczający.");
}
```

Błąd 2 - kilka osobnych if zamiast else if

Kilka osobnych if oznacza kilka niezależnych sprawdzeń.

Kod: csharp

```
if (punkty >= 90)
{
    Console.WriteLine("Bardzo dobry.");
}

if (punkty >= 75)
{
    Console.WriteLine("Dobry.");
}

if (punkty >= 50)
{
    Console.WriteLine("Wystarczający.");
}
```

Przy punktach 95 mogą wypisać się wszystkie trzy komunikaty.

Jeśli chcemy wybrać jedną kategorię, stosujemy if / else if / else.

Błąd 3 - else w złym miejscu

else odnosi się do najbliższego pasującego if albo else if.

Dla czytelności zawsze używamy nawiasów klamrowych.

10. Zapowiedź kolejnej lekcji

Instrukcja else if dobrze nadaje się do wielu warunków.

Gdy sprawdzamy jedną zmienną na kilka konkretnych wartości, wygodna może być instrukcja switch, którą poznamy w kolejnej lekcji.

Zapamiętaj

- else if pozwala sprawdzić kolejny warunek.

- Warunki są sprawdzane od góry do dołu.
- Wykonuje się pierwszy pasujący blok.
- Po wykonaniu jednego bloku reszta łańcucha jest pomijana.
- Kolejność warunków ma znaczenie.
- `else` na końcu obsługuje przypadek, gdy żaden warunek nie był `true`.
- Kilka osobnych `if` to nie to samo co `if / else if / else`.

Ćwiczenia

1. Wczytaj liczbę i wypisz, czy jest dodatnia, ujemna, czy równa zero.
2. Wczytaj liczbę punktów i wypisz kategorię wyniku: bardzo dobry, dobry, wystarczający, niewystarczający.
3. Wczytaj temperaturę i wypisz komunikat: mróz, chłódno, ciepło, gorąco.
4. Wczytaj numer dnia tygodnia od 1 do 7 i wypisz nazwę dnia, używając `if / else if / else`.
5. Wczytaj numer opcji menu i wypisz odpowiedni komunikat.
6. Popraw błędną kolejność progów punktowych.
7. Zamień kilka osobnych `if` na jeden łańcuch `if / else if / else`.
8. Dla podanych wartości przewidź, który blok programu się wykona.
9. Dodaj końcowe `else` obsługujące niepoprawną wartość.
10. Napisz własny przykład wyboru jednej z co najmniej trzech możliwości.

21. switch

03-warunki/04-switch.md

Instrukcja switch

Cel lekcji

Nauczysz się używać instrukcji switch, aby wygodnie wybrać jedną z wielu ścieżek działania programu na podstawie wartości jednej zmiennej lub wyrażenia.

Po tej lekcji powinieneś umieć:

- wyjaśnić, kiedy switch jest czytelniejszy niż if / else if,
- zapisać klasyczną instrukcję switch,
- użyć case, break i default,
- wskazać, który przypadek zostanie wykonany,
- dobrać switch albo if / else if do prostego problemu.

1. Krótkie przypomnienie else if

else if pozwala sprawdzać wiele warunków.

Czasem jednak sprawdzamy jedną zmienną na kilka konkretnych wartości, na przykład:

- numer opcji menu,
- numer dnia tygodnia,
- ocenę od 1 do 6,
- znak działania: +, -, *, /.

W takiej sytuacji switch może być czytelniejszy niż długi łańcuch if / else if.

2. Składnia switch

Schemat:

Kod: csharp

```
switch (zmienna)
{
    case wartosc1:
        // instrukcje
        break;

    case wartosc2:
        // instrukcje
        break;

    default:
        // instrukcje, gdy żadna wartość nie pasuje
        break;
}
```

Najważniejsze elementy:

- switch sprawdza wartość podaną w nawiasie,
- case oznacza konkretny przypadek,
- break kończy wykonanie danego przypadku,

- default działa wtedy, gdy żaden case nie pasuje,
- default jest podobny do końcowego else.

3. Pierwszy przykład - menu

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Wybierz opcję:");
        Console.WriteLine("1 - Start");
        Console.WriteLine("2 - Pomoc");
        Console.WriteLine("3 - Wyjście");

        int opcja = int.Parse(Console.ReadLine());

        switch (opcja)
        {
            case 1:
                Console.WriteLine("Wybrano start.");
                break;

            case 2:
                Console.WriteLine("Wybrano pomoc.");
                break;

            case 3:
                Console.WriteLine("Wybrano wyjście.");
                break;

            default:
                Console.WriteLine("Nieznana opcja.");
                break;
        }
    }
}
```

Jeśli opcja ma wartość 1, wykona się case 1.

Jeśli opcja ma wartość 2, wykona się case 2.

Jeśli opcja ma wartość 3, wykona się case 3.

Dla innej wartości wykona się default.

4. Dzień tygodnia

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj numer dnia tygodnia od 1 do 7:");
        int dzien = int.Parse(Console.ReadLine());

        switch (dzien)
        {
            case 1:
                Console.WriteLine("Poniedziałek");
```



```

        break;

    case 2:
        Console.WriteLine("Wtorek");
        break;

    case 3:
        Console.WriteLine("Środa");
        break;

    case 4:
        Console.WriteLine("Czwartek");
        break;

    case 5:
        Console.WriteLine("Piątek");
        break;

    case 6:
        Console.WriteLine("Sobota");
        break;

    case 7:
        Console.WriteLine("Niedziela");
        break;

    default:
        Console.WriteLine("Niepoprawny numer dnia.");
        break;
    }
}

```

Ten przykład dobrze pasuje do switch, ponieważ sprawdzamy jedną zmienną `dzien` i konkretne wartości od 1 do 7.

5. Ocena szkolna

Kod: **csharp**

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj ocenę od 1 do 6:");
        int ocena = int.Parse(Console.ReadLine());

        switch (ocena)
        {
            case 1:
                Console.WriteLine("Niedostateczny");
                break;

            case 2:
                Console.WriteLine("Dopuszczający");
                break;

            case 3:
                Console.WriteLine("Dostateczny");
                break;

            case 4:
                Console.WriteLine("Dobry");
                break;
        }
    }
}

```

```

        case 5:
            Console.WriteLine("Bardzo dobry");
            break;

        case 6:
            Console.WriteLine("Celujący");
            break;

        default:
            Console.WriteLine("Ocena poza zakresem.");
            break;
    }
}
}

```

default obsługuje sytuację, gdy użytkownik poda wartość spoza zakresu 1-6.

6. Wybór działania kalkulatora

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj pierwszą liczbę:");
        double a = double.Parse(Console.ReadLine());

        Console.WriteLine("Podaj drugą liczbę:");
        double b = double.Parse(Console.ReadLine());

        Console.WriteLine("Wybierz działanie:");
        Console.WriteLine("+ dodawanie");
        Console.WriteLine("- odejmowanie");
        Console.WriteLine("* mnożenie");
        Console.WriteLine("/ dzielenie");

        string dzialanie = Console.ReadLine();

        switch (dzialanie)
        {
            case "+":
                Console.WriteLine($"Wynik: {a + b}");
                break;

            case "-":
                Console.WriteLine($"Wynik: {a - b}");
                break;

            case "*":
                Console.WriteLine($"Wynik: {a * b}");
                break;

            case "/":
                Console.WriteLine($"Wynik: {a / b}");
                break;

            default:
                Console.WriteLine("Nieznane działanie.");
                break;
        }
    }
}

```

To jest prosty przykład dydaktyczny. Nie sprawdza jeszcze dzielenia przez zero ani błędnych danych wejściowych. Walidacja danych będzie rozwijana później.

7. Switch a if / else if

Używaj switch, gdy:

- sprawdzasz jedną zmienną,
- porównujesz ją z konkretnymi wartościami,
- masz kilka lub kilkanaście prostych przypadków.

Używaj if / else if, gdy:

- warunki są zakresami, na przykład punkty ≥ 90 ,
- warunki są bardziej złożone,
- używasz porównań typu $>$, $<$, $\geq$, $\leq$,
- sprawdzasz kilka różnych zmiennych.

Progi punktów zwykle lepiej wyglądają jako if / else if:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int punkty = 82;

        if (punkty >= 90)
        {
            Console.WriteLine("Wynik bardzo dobry.");
        }
        else if (punkty >= 75)
        {
            Console.WriteLine("Wynik dobry.");
        }
        else if (punkty >= 50)
        {
            Console.WriteLine("Wynik wystarczający.");
        }
        else
        {
            Console.WriteLine("Wynik niewystarczający.");
        }
    }
}
```

Tutaj warunki są zakresami, więc klasyczne if / else if jest czytelniejsze.

8. Zgadnij, który case się wykona

Przykład z odpowiedzią:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int opcja = 2;
```

```

switch (opcja)
{
    case 1:
        Console.WriteLine("Start");
        break;

    case 2:
        Console.WriteLine("Pomoc");
        break;

    default:
        Console.WriteLine("Inna opcja");
        break;
}
}

```

Odpowiedź: program wypisze Pomoc, ponieważ opcja ma wartość 2.

Spróbuj przewidzieć wyniki:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int dzien = 5;

        switch (dzien)
        {
            case 1:
                Console.WriteLine("Poniedziałek");
                break;

            case 5:
                Console.WriteLine("Piątek");
                break;

            default:
                Console.WriteLine("Inny dzień");
                break;
        }

        string znak = "*";

        switch (znak)
        {
            case "+":
                Console.WriteLine("Dodawanie");
                break;

            case "*":
                Console.WriteLine("Mnożenie");
                break;

            default:
                Console.WriteLine("Nieznane działanie");
                break;
        }

        int ocena = 7;

        switch (ocena)
        {
            case 1:
                Console.WriteLine("Niedostateczny");

```

```

        break;

    case 6:
        Console.WriteLine("Celujący");
        break;

    default:
        Console.WriteLine("Ocena poza zakresem");
        break;
    }
}

```

Odpowiedzi:

- pierwszy przykład wypisze Piątek,
- drugi przykład wypisze Mnożenie,
- trzeci przykład wypisze Ocena poza zakresem.

9. Typowe błędy

Błąd 1 - brak break

break kończy dany przypadek.

W podstawowych przykładach w tym kursie po każdym case stosujemy break.

Brak break jest częstym błędem i może powodować problem z kompilacją albo nieoczekiwane działanie zależnie od kodu.

Błąd 2 - brak default

default obsługuje wartości, których nie przewidziano w case.

Warto dodawać default szczególnie przy danych od użytkownika.

Błąd 3 - zły typ wartości w case

Jeśli switch sprawdza int, case powinien zawierać wartości typu int.

Kod: csharp

```

switch (opcja)
{
    case 1:
        Console.WriteLine("Start");
        break;
}

```

Jeśli switch sprawdza string, case powinien zawierać tekst w cudzysłowie.

Kod: csharp

```

switch (dzialanie)
{
    case "+":
        Console.WriteLine("Dodawanie");
        break;
}

```

Błąd 4 - używanie switch do zakresów

Klasyczny switch dobrze sprawdza konkretne wartości.

Do zakresów typu punkty `>= 90` zwykle lepsze jest `if / else if`.

10. Krótka wzmianka o nowoczesnym switch

W nowszych wersjach C# istnieją także bardziej zaawansowane formy switch, na przykład switch expression.

W tym kursie na razie używamy klasycznej instrukcji switch, bo jest czytelna i wystarczająca do podstaw programowania konsolowego.

11. Zapowiedź kolejnej części

Po poznaniu if, if else, else if i switch można budować programy podejmujące decyzje.

W kolejnych działach pojawią się pętle, które pozwalają wielokrotnie wykonywać instrukcje.

Zapamiętaj

- switch wybiera przypadek na podstawie wartości.
- case oznacza konkretną wartość.
- break kończy obsługę danego przypadku.
- default obsługuje pozostałe wartości.
- switch jest dobry do wielu konkretnych wartości jednej zmiennej.
- if / else if jest lepszy do zakresów i bardziej złożonych warunków.

Ćwiczenia

1. Wczytaj numer dnia tygodnia i wypisz nazwę dnia za pomocą switch.
2. Wczytaj ocenę od 1 do 6 i wypisz jej opis.
3. Napisz menu z opcjami: 1 - Start, 2 - Ustawienia, 3 - Wyjście.
4. Wczytaj znak działania +, -, *, / i wypisz nazwę działania.
5. Rozbuduj przykład kalkulatora o działanie modulo dla liczb całkowitych.
6. Dodaj default do przykładu, który go nie ma.
7. Wskaż, który case wykona się dla podanej wartości.
8. Wyjaśnij, kiedy lepiej użyć switch, a kiedy if / else if.
9. Popraw przykład, w którym case ma tekst, a switch sprawdza liczbę.
10. Napisz własny przykład switch z co najmniej trzema przypadkami i default.

22. Podsumowanie działu

03-warunki/05-podsumowanie.md

Podsumowanie działu - instrukcje warunkowe

Cel lekcji

Powtórzysz najważniejsze informacje z działu o instrukcjach warunkowych i zobaczysz, kiedy używać `if`, `if else`, `else if` oraz `switch`.

Po tym podsumowaniu powinieneś umieć:

- rozpoznać warunek dający `true` albo `false`,
- dobrać odpowiednią instrukcję warunkową,
- odróżnić `=` od `==`,
- wskazać typowe błędy w instrukcjach warunkowych,
- wyjaśnić, kiedy lepszy jest `switch`, a kiedy `if / else if`.

1. Po co są instrukcje warunkowe

Instrukcje warunkowe pozwalają programowi podejmować decyzje.

Bez instrukcji warunkowych program wykonuje polecenia po kolei. Dzięki warunkom może wykonać różne instrukcje zależnie od danych.

Przykłady decyzji:

- czy liczba jest dodatnia,
- czy osoba jest pełnoletnia,
- czy wynik punktowy jest wystarczający,
- którą opcję menu wybrał użytkownik.

2. Warunek

Warunek to wyrażenie, które daje wynik `true` albo `false`.

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        int x = 12;

        Console.WriteLine(x > 10);
        Console.WriteLine(x == 5);
    }
}
```

Warunki są podstawą działania instrukcji `if`, `if else`, `else if` i często także wyboru, czy dana ścieżka programu ma się wykonać.

3. Najważniejsze konstrukcje

Konstrukcja	Kiedy używać	Przykład
---	---	---
`if`	gdy sprawdzamy jeden warunek	`liczba > 0`
`if else`	gdy są dwie możliwości	pełnoletni / niepełnoletni
`else if`	gdy jest kilka warunków	progi punktowe
`switch`	gdy sprawdzamy jedną wartość	menu, dzień tygodnia

4. if

if wykonuje kod tylko wtedy, gdy warunek jest spełniony.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 8;

        if (liczba > 0)
        {
            Console.WriteLine("Liczba jest dodatnia.");
        }
    }
}
```

Jeśli warunek jest false, blok if zostaje pominięty.

5. if else

if else wybiera jedną z dwóch możliwości.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 16;

        if (wiek >= 18)
        {
            Console.WriteLine("Osoba pełnoletnia.");
        }
        else
        {
            Console.WriteLine("Osoba niepełnoletnia.");
        }
    }
}
```

Gdy warunek jest true, wykonuje się blok if. Gdy warunek jest false, wykonuje się blok else.

6. else if

else if pozwala wybrać jedną możliwość spośród wielu warunków.

Kod: csharp

```
using System;

class Program
{

```



```

static void Main()
{
    int punkty = 80;

    if (punkty >= 90)
    {
        Console.WriteLine("Bardzo dobry");
    }
    else if (punkty >= 75)
    {
        Console.WriteLine("Dobry");
    }
    else if (punkty >= 50)
    {
        Console.WriteLine("Wystarczający");
    }
    else
    {
        Console.WriteLine("Niewystarczający");
    }
}
}

```

Warunki są sprawdzane od góry do dołu. Wykonuje się pierwszy pasujący blok, a reszta łańcucha jest pomijana.

7. switch

switch wybiera przypadek na podstawie jednej wartości.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int opcja = 2;

        switch (opcja)
        {
            case 1:
                Console.WriteLine("Start");
                break;

            case 2:
                Console.WriteLine("Pomoc");
                break;

            default:
                Console.WriteLine("Nieznana opcja");
                break;
        }
    }
}

```

case oznacza konkretną wartość, break kończy obsługę przypadku, a default obsługuje pozostałe wartości.

8. Operatory często używane w warunkach

Najczęściej spotykane operatory:

- == - porównanie równości,

- != - różne,
- > - większe,
- < - mniejsze,
- >= - większe lub równe,
- <= - mniejsze lub równe,
- % - reszta z dzielenia,
- && - logiczne „i”,
- || - logiczne „lub”.

Przykłady:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 12;

        Console.WriteLine(liczba == 12);
        Console.WriteLine(liczba != 0);
        Console.WriteLine(liczba % 2 == 0);
        Console.WriteLine(liczba >= 1 && liczba <= 20);
    }
}
```

9. Różnica między = i ==

= przypisuje wartość do zmiennej.

== porównuje wartości.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 10;

        if (liczba == 10)
        {
            Console.WriteLine("Liczba wynosi 10.");
        }
    }
}
```

W warunkach do sprawdzania równości używamy ==.

10. Kilka if a if / else if / else

Kilka osobnych if oznacza kilka niezależnych sprawdzeń.

if / else if / else oznacza jeden łańcuch wyboru.

Jeśli chcemy wybrać jedną kategorię wyniku, zwykle używamy if / else if / else.

11. Kiedy switch, a kiedy if / else if

switch jest dobry, gdy:

- sprawdzamy jedną zmienną,
- porównujemy ją z konkretnymi wartościami,
- mamy menu, wybór dnia tygodnia albo wybór działania.

if / else if jest lepszy, gdy:

- sprawdzamy zakresy,
- używamy warunków typu punkty ≥ 90 ,
- warunki są bardziej złożone,
- trzeba porównać kilka różnych wartości.

12. Przykład całościowy

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Podaj liczbę punktów:");
        int punkty = int.Parse(Console.ReadLine());

        if (punkty >= 50)
        {
            Console.WriteLine("Test zaliczony.");
        }
        else
        {
            Console.WriteLine("Test niezaliczony.");
        }

        Console.WriteLine("Wybierz opcję:");
        Console.WriteLine("1 - Pokaż wynik");
        Console.WriteLine("2 - Pokaż opis");
        Console.WriteLine("3 - Zakończ");

        int opcja = int.Parse(Console.ReadLine());

        switch (opcja)
        {
            case 1:
                Console.WriteLine($"Liczba punktów: {punkty}");
                break;

            case 2:
                Console.WriteLine("To jest prosty przykład użycia instrukcji warunkowych.");
                break;

            case 3:
                Console.WriteLine("Koniec programu.");
                break;

            default:
                Console.WriteLine("Nieznana opcja.");
                break;
        }
    }
}
```

if else sprawdza, czy test jest zaliczony.

switch obsługuje wybór użytkownika z menu.

Są to dwa różne zastosowania instrukcji warunkowych.

Najczęstsze błędy

1. Użycie = zamiast ==.
2. Brak nawiasów klamrowych.
3. Średnik po if.
4. Zła kolejność warunków w else if.
5. Kilka osobnych if zamiast if / else if / else.
6. Brak default w switch przy danych od użytkownika.
7. Używanie switch do zakresów, gdzie czytelniejszy jest if / else if.

Zgadnij, co wypisze program

Przykład 1:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int x = 12;

        if (x > 10)
        {
            Console.WriteLine("A");
        }
        else
        {
            Console.WriteLine("B");
        }
    }
}
```

Przykład 2:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int punkty = 80;

        if (punkty >= 90)
        {
            Console.WriteLine("Bardzo dobry");
        }
        else if (punkty >= 75)
        {
            Console.WriteLine("Dobry");
        }
        else if (punkty >= 50)
        {
            Console.WriteLine("Wystarczający");
        }
    }
}
```

```

        else
        {
            Console.WriteLine("Niewystarczający");
        }
    }
}

```

Przykład 3:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int opcja = 3;

        switch (opcja)
        {
            case 1:
                Console.WriteLine("Start");
                break;

            case 2:
                Console.WriteLine("Pomoc");
                break;

            case 3:
                Console.WriteLine("Wyjście");
                break;

            default:
                Console.WriteLine("Nieznana opcja");
                break;
        }
    }
}

```

Odpowiedzi:

- przykład 1 wypisze A,
- przykład 2 wypisze Dobry,
- przykład 3 wypisze Wyjście.

Najważniejsze do zapamiętania

- Instrukcje warunkowe pozwalają programowi podejmować decyzje.
- Warunek daje wynik true albo false.
- if wykonuje kod tylko przy spełnionym warunku.
- if else wybiera jedną z dwóch możliwości.
- else if pozwala sprawdzić wiele warunków.
- switch wybiera przypadek na podstawie jednej wartości.
- = przypisuje wartość, a == porównuje wartości.
- Kolejność warunków w else if ma znaczenie.
- default w switch obsługuje pozostałe wartości.

Minićwiczenia powtórkowe

1. Wczytaj liczbę i sprawdź, czy jest dodatnia, ujemna, czy równa zero.
2. Wczytaj wiek i wypisz, czy osoba jest pełnoletnia.
3. Wczytaj liczbę punktów i wypisz kategorię wyniku.
4. Wczytaj numer dnia tygodnia i użyj switch do wypisania nazwy dnia.
5. Wczytaj znak działania +, -, *, / i użyj switch do wypisania nazwy działania.
6. Popraw błąd `z =` zamiast `==`.
7. Popraw złą kolejność progów punktowych.
8. Wyjaśnij, czym różni się `if else` od `else if`.
9. Wyjaśnij, kiedy `switch` jest czytelniejszy niż `else if`.
10. Napisz własny przykład programu z menu tekstowym.

23. Wprowadzenie

04-petle/README.md

04 - Pętle

W tym dziale poznasz powtarzanie instrukcji. Pętle pozwalają wykonać ten sam fragment kodu wiele razy.

Lekcje

- while (01-while.md)
- do while (02-do-while.md)
- for (03-for.md)
- Tablice jednowymiarowe - podstawy (04-tablice-jednowymiarowe.md)
- Pętla for po tablicy (05-for-po-tablicy.md)
- Suma i średnia elementów tablicy (06-suma-srednia-tablicy.md)
- Minimum i maksimum elementów tablicy (07-minimum-maksimum-tablicy.md)
- Zliczanie elementów tablicy spełniających warunek (08-licznik-akumulator.md)
- Pętla foreach (09-foreach.md)
- break i continue (10-break-continue.md)

Po tym dziale

Uczeń powinien umieć powtarzać instrukcje, korzystać z podstaw tablic, zliczać powtórzenia, sumować wartości i przerwać pętlę w odpowiednim momencie.

24. while

04-petle/01-while.md

Pętla while

Cel lekcji

Nauczysz się używać pętli while, aby wielokrotnie wykonywać blok instrukcji, dopóki warunek jest spełniony.

Po tej lekcji powinieneś umieć:

- wyjaśnić, czym jest pętla,
- zapisać podstawową pętlę while,
- użyć licznika do kontrolowania liczby powtórzeń,
- przewidzieć, ile razy wykona się pętla,
- rozpoznać prostą pętlę nieskończoną.

1. Krótkie wprowadzenie

Program bez pętli wykonuje instrukcje po kolei.

Pętla pozwala powtarzać fragment kodu wiele razy.

Pętla while wykonuje blok kodu, dopóki warunek ma wartość true.

Jeśli warunek ma wartość false, pętla się kończy.

2. Składnia while

Schemat:

Kod: csharp

```
while (warunek)
{
    // instrukcje wykonywane wielokrotnie
}
```

Najważniejsze elementy:

- warunek zapisujemy w nawiasach okrągłych (),
- instrukcje pętli zapisujemy w nawiasach klamrowych {},
- po każdym wykonaniu bloku warunek jest sprawdzany ponownie.

3. Pierwszy przykład - liczby od 1 do 5

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int licznik = 1;

        while (licznik <= 5)
```



```

        {
            Console.WriteLine(licznik);
            licznik++;
        }
    }
}

```

W tym przykładzie:

- licznik zaczyna od 1,
- pętla działa, dopóki `licznik <= 5`,
- `licznik++` zwiększa wartość licznika o 1,
- bez zwiększania licznika pętla mogłaby działać bez końca.

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart TD
    A([Start]) --> B[Ustaw licznik]
    B --> C{licznik <= 5?}
    C -- "tak" --> D[Wypisz licznik]
    D --> E[Zwiększ licznik]
    E --> C
    C -- "nie" --> F([Koniec])

```

Diagram pokazuje, że w pętli `while` warunek jest sprawdzany przed każdym obiegiem.

4. Warunek od początku fałszywy

Pętla `while` sprawdza warunek przed pierwszym wykonaniem bloku.

Kod: `csharp`

```

using System;

class Program
{
    static void Main()
    {
        int licznik = 10;

        while (licznik <= 5)
        {
            Console.WriteLine(licznik);
            licznik++;
        }

        Console.WriteLine("Koniec programu.");
    }
}

```

Warunek `licznik <= 5` jest `false` już na początku.

Blok pętli nie wykona się ani razu.

Napis "Koniec programu." wypisze się normalnie, bo jest poza pętlą.

5. Liczby parzyste

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 2;

        while (liczba <= 10)
        {
            Console.WriteLine(liczba);
            liczba += 2;
        }
    }
}
```

`liczba += 2` zwiększa wartość zmiennej o 2.

Dlatego program wypisuje kolejne liczby parzyste.

6. Suma liczb od 1 do 5

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int licznik = 1;
        int suma = 0;

        while (licznik <= 5)
        {
            suma += licznik;
            licznik++;
        }

        Console.WriteLine($"Suma: {suma}");
    }
}
```

`suma` to zmienna przechowująca wynik narastająco.

`suma += licznik` dodaje aktualną wartość licznika do sumy.

Taki sposób pracy nazywamy akumulacją wartości. Dokładniej licznik i akumulator zostaną uporządkowane w kolejnych lekcjach działu.

7. Wczytywanie kilku liczb

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
```

```

    int licznik = 1;
    int suma = 0;

    while (licznik <= 3)
    {
        Console.WriteLine("Podaj liczbę:");
        int liczba = int.Parse(Console.ReadLine());

        suma += liczba;
        licznik++;
    }

    Console.WriteLine($"Suma podanych liczb: {suma}");
}

```

Pętla wykona się 3 razy.

Za każdym razem użytkownik poda jedną liczbę.

Zmienna suma przechowuje łączny wynik.

8. Zgadnij, ile razy wykona się pętla

Przykład z odpowiedzią:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int i = 1;

        while (i <= 3)
        {
            Console.WriteLine(i);
            i++;
        }
    }
}

```

Odpowiedź: pętla wykona się 3 razy i wypisze 1, 2, 3.

Spróbuj przewidzieć wyniki:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int a = 0;

        while (a < 4)
        {
            Console.WriteLine(a);
            a++;
        }

        int b = 5;

        while (b > 0)
        {

```

```

        Console.WriteLine(b);
        b--;
    }

    int c = 10;

    while (c < 5)
    {
        Console.WriteLine(c);
        c++;
    }
}

```

Odpowiedzi:

- pierwszy przykład wykona się 4 razy i wypisze 0, 1, 2, 3,
- drugi przykład wykona się 5 razy i wypisze 5, 4, 3, 2, 1,
- trzeci przykład nie wykona się ani razu.

9. Typowe błędy

Błąd 1 - brak zmiany licznika

Niepoprawnie:

Kod: **csharp**

```

int licznik = 1;

while (licznik <= 5)
{
    Console.WriteLine(licznik);
}

```

licznik się nie zmienia, więc warunek cały czas pozostaje true.

Może powstać pętla nieskończona.

Poprawnie:

Kod: **csharp**

```

int licznik = 1;

while (licznik <= 5)
{
    Console.WriteLine(licznik);
    licznik++;
}

```

Błąd 2 - zły warunek

Warunek musi prowadzić do zakończenia pętli.

Trzeba sprawdzić, czy zmienna sterująca zmienia się w dobrą stronę.

Przykład: jeśli licznik rośnie, warunek powinien mieć taką granicę, którą licznik może przekroczyć.

Błąd 3 - instrukcja poza blokiem pętli

Tylko instrukcje w nawiasach klamrowych należą do pętli.

Kod poza pętlą wykona się dopiero po jej zakończeniu.

Błąd 4 - średnik po while

Niepoprawnie:

Kod: csharp

```
while (licznik <= 5);  
{  
    Console.WriteLine(licznik);  
    licznik++;  
}
```

Średnik po while kończy pustą instrukcję pętli.

Blok poniżej nie działa wtedy tak, jak uczeń oczekuje.

W tym kursie nie stawiamy średnika po warunku while.

10. Zapowiedź kolejnej lekcji

Pętla while sprawdza warunek przed pierwszym wykonaniem bloku.

W kolejnej lekcji pojawi się pętla do while, która wykonuje blok co najmniej raz.

Zapamiętaj

- while powtarza blok instrukcji, dopóki warunek jest true.
- Warunek jest sprawdzany przed każdym wykonaniem pętli.
- Pętla while może nie wykonać się ani razu.
- Licznik często służy do kontrolowania liczby powtórzeń.
- Trzeba zmieniać zmienną sterującą pętlą.
- Brak zmiany warunku może prowadzić do pętli nieskończonej.

Ćwiczenia

1. Wypisz liczby od 1 do 10 za pomocą while.
2. Wypisz liczby od 10 do 1 za pomocą while.
3. Wypisz liczby parzyste od 2 do 20.
4. Wypisz liczby nieparzyste od 1 do 19.
5. Oblicz sumę liczb od 1 do 10.
6. Wczytaj 5 liczb i oblicz ich sumę.
7. Wczytaj 3 liczby i oblicz ich średnią.
8. Sprawdź, ile razy wykona się podana pętla.
9. Popraw pętlę nieskończoną.
10. Napisz własny przykład pętli while z licznikiem.

25. do while

04-petle/02-do-while.md

Pętla do while

Cel lekcji

Nauczysz się używać pętli `do while` i rozpoznawać sytuacje, w których kod powinien wykonać się co najmniej raz.

Przypomnienie pętli while

Pętla `while` sprawdza warunek przed pierwszym wykonaniem instrukcji.

Jeżeli warunek od razu jest fałszywy, instrukcje w pętli nie wykonają się ani razu.

Czasami chcemy, aby program najpierw coś zrobił, a dopiero potem zapytał, czy ma powtórzyć działanie. Do takich sytuacji pasuje pętla `do while`.

Składnia pętli do while

Kod: `csharp`

```
do
{
    // instrukcje wykonywane co najmniej raz
}
while (warunek);
```

W pętli `do while` najpierw wykonuje się blok instrukcji zapisany po `do`. Dopiero potem program sprawdza warunek w nawiasie przy `while`.

Jeżeli warunek ma wartość `true`, pętla wykona się ponownie. Jeżeli warunek ma wartość `false`, pętla zakończy działanie.

Ważne: po warunku w pętli `do while` stawiamy średnik:

Kod: `csharp`

```
while (warunek);
```

To różni się od zwykłej pętli `while`, w której po warunku nie stawiamy średnika.

Przykład: liczby od 1 do 5

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        int licznik = 1;

        do
        {
            Console.WriteLine(licznik);
            licznik++;
        }
        while (licznik <= 5);
    }
}
```

```
    }  
}
```

W tym przykładzie:

- zmienna `licznik` zaczyna od wartości 1,
- program wypisuje wartość zmiennej `licznik`,
- instrukcja `licznik++` zwiększa `licznik` o 1,
- pętla powtarza się tak długo, jak `licznik <= 5`.

Program wypisze:

Kod: csharp

```
1  
2  
3  
4  
5
```

Warunek fałszywy od początku

Pętla `do while` wykona się przynajmniej raz, nawet wtedy, gdy warunek jest fałszywy już na początku.

Kod: csharp

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int licznik = 10;  
  
        do  
        {  
            Console.WriteLine(licznik);  
            licznik++;  
        }  
        while (licznik <= 5);  
  
        Console.WriteLine("Koniec programu.");  
    }  
}
```

Warunek `licznik <= 5` jest fałszywy, bo `licznik` ma wartość 10. Mimo to program najpierw wykona blok po `do`, czyli wypisze 10. Dopiero potem sprawdzi warunek i zakończy pętlę.

while a do while

Najważniejsza różnica:

- `while` sprawdza warunek przed wykonaniem bloku,
- `while` może nie wykonać się ani razu,
- `do while` wykonuje blok przed sprawdzeniem warunku,
- `do while` wykonuje się co najmniej raz.

Porównaj dwa fragmenty:

Kod: csharp

```
using System;  
  
class Program  
{
```

```

static void Main()
{
    int x = 10;

    while (x < 5)
    {
        Console.WriteLine("while");
    }

    do
    {
        Console.WriteLine("do while");
    }
    while (x < 5);
}

```

Napis "while" nie zostanie wypisany, ponieważ warunek $x < 5$ jest fałszywy już przed wejściem do pętli.

Napis "do while" zostanie wypisany jeden raz, ponieważ pętla do while najpierw wykonuje blok, a dopiero potem sprawdza warunek.

Wyjaśnienie

Wyobraź sobie nauczyciela, który mówi:

Najpierw rozwiąż jedno zadanie. Potem zapytam, czy chcesz rozwiązać kolejne.

Tak działa do while: najpierw wykonuje działanie, potem pyta o dalsze powtarzanie.

Zwykle while działa inaczej: najpierw pyta, czy można zacząć, a dopiero potem wykonuje działanie.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        string czyPowtorzyc;

        do
        {
            Console.WriteLine("Wykonuję zadanie.");
            Console.WriteLine("Czy powtórzyć? tak/nie");
            czyPowtorzyc = Console.ReadLine();
        }
        while (czyPowtorzyc == "tak");
    }
}

```

Ten program wykona zadanie co najmniej raz. Jeżeli użytkownik wpisze tak, program powtórzy działanie.

Proste menu konsolowe

Pętla do while często pasuje do menu, ponieważ menu powinno pokazać się użytkownikowi przynajmniej raz.

Kod: csharp

```

using System;

class Program

```



```

{
    static void Main()
    {
        int opcja;

        do
        {
            Console.WriteLine("Menu:");
            Console.WriteLine("1 - Start");
            Console.WriteLine("2 - Pomoc");
            Console.WriteLine("0 - Wyjście");
            Console.WriteLine("Wybierz opcję:");

            opcja = int.Parse(Console.ReadLine());

            if (opcja == 1)
            {
                Console.WriteLine("Wybrano start.");
            }
            else if (opcja == 2)
            {
                Console.WriteLine("Wybrano pomoc.");
            }
            else if (opcja == 0)
            {
                Console.WriteLine("Koniec programu.");
            }
            else
            {
                Console.WriteLine("Nieznana opcja.");
            }
        }
        while (opcja != 0);
    }
}

```

W tym programie menu pokazuje się co najmniej raz. Pętla powtarza się tak długo, jak użytkownik nie wybierze opcji 0.

Powtarzanie prostego obliczenia

Pętla `while` może służyć do powtarzania prostych działań.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        string decyzja;

        do
        {
            Console.WriteLine("Podaj pierwszą liczbę:");
            int pierwszaLiczba = int.Parse(Console.ReadLine());

            Console.WriteLine("Podaj drugą liczbę:");
            int drugaLiczba = int.Parse(Console.ReadLine());

            int suma = pierwszaLiczba + drugaLiczba;

            Console.WriteLine("Suma: " + suma);
            Console.WriteLine("Czy chcesz wykonać obliczenie ponownie? tak/nie");
            decyzja = Console.ReadLine();
        }
    }
}

```

```

        while (decyzja == "tak");
    }
}

```

Porównanie `decyzja == "tak"` jest dokładne. Na tym etapie kursu przyjmujemy, że użytkownik wpisuje dokładnie tak. Tekst "Tak" nie jest tym samym co "tak".

Ile razy wykona się pętla?

Przykład z odpowiedzią:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int i = 1;

        do
        {
            Console.WriteLine(i);
            i++;
        }
        while (i <= 3);
    }
}

```

Ta pętla wykona się 3 razy i wypisze:

Kod: csharp

```

1
2
3

```

Spróbuj najpierw samodzielnie przewidzieć wyniki kolejnych przykładów.

Kod: csharp

```

int a = 10;

do
{
    Console.WriteLine(a);
    a++;
}
while (a < 5);

```

Kod: csharp

```

int b = 3;

do
{
    Console.WriteLine(b);
    b--;
}
while (b > 0);

```

Kod: csharp

```

int c = 0;

do
{
    Console.WriteLine(c);
    c++;
}

```

```
}  
while (c < 4);
```

Odpowiedzi:

- pierwszy przykład wykona się 1 raz i wypisze 10,
- drugi przykład wykona się 3 razy i wypisze 3, 2, 1,
- trzeci przykład wykona się 4 razy i wypisze 0, 1, 2, 3.

Typowe błędy

Brak zmiany zmiennej sterującej

Gorszy przykład:

Kod: csharp

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int licznik = 1;  
  
        do  
        {  
            Console.WriteLine(licznik);  
        }  
        while (licznik <= 5);  
    }  
}
```

Wartość zmiennej licznik się nie zmienia, więc warunek cały czas pozostaje prawdziwy. Taka pętla może działać bez końca.

Lepszy przykład:

Kod: csharp

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int licznik = 1;  
  
        do  
        {  
            Console.WriteLine(licznik);  
            licznik++;  
        }  
        while (licznik <= 5);  
    }  
}
```

Brak średnika po while

Gorszy przykład:

Kod: csharp

```
do  
{  
    Console.WriteLine("Test");  
}
```

```
while (true)
```

Poprawny przykład:

Kod: csharp

```
do
{
    Console.WriteLine("Test");
}
while (true);
```

W pętli `do while` średnik po warunku jest wymagany.

Mylenie while i do while

Pętla `while` może nie wykonać się ani razu. Pętla `do while` wykona się co najmniej raz.

Dlatego `do while` jest wygodna na przykład przy menu lub pytaniu użytkownika, czy chce powtórzyć działanie.

Zmienna zadeklarowana w złym miejscu

Jeżeli zmienna jest używana w warunku po bloku `do`, trzeba zadeklarować ją przed pętlą.

Poprawny przykład:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int opcja;

        do
        {
            Console.WriteLine("Wybierz opcję:");
            opcja = int.Parse(Console.ReadLine());
        }
        while (opcja != 0);
    }
}
```

Zmienna `opcja` jest dostępna zarówno wewnątrz bloku `do`, jak i w warunku po `while`.

Kiedy używać do while?

Pętla `do while` jest przydatna wtedy, gdy blok kodu musi wykonać się co najmniej raz.

Przykłady:

- pokazanie menu,
- zapytanie użytkownika o decyzję,
- powtórzenie prostego obliczenia,
- wykonanie pierwszej próby przed sprawdzeniem warunku.

W następnej lekcji poznasz pętlę `for`, która dobrze pasuje do sytuacji, gdy z góry znamy liczbę powtórzeń.

Zapamiętaj

- `do while` wykonuje blok instrukcji co najmniej raz.
- Warunek jest sprawdzany po wykonaniu bloku.
- Po warunku w `do while` trzeba postawić średnik.
- Trzeba zmieniać zmienną sterującą, aby uniknąć pętli nieskończonej.
- `do while` dobrze pasuje do menu i powtarzania działania na życzenie użytkownika.
- `while` i `do while` różnią się momentem sprawdzania warunku.

Ćwiczenia

1. Wypisz liczby od 1 do 5 za pomocą pętli `do while`.
2. Wypisz liczby od 5 do 1 za pomocą pętli `do while`.
3. Sprawdź, co się stanie, gdy warunek w `do while` będzie fałszywy od początku.
4. Napisz proste menu z opcjami 1, 2 i 0.
5. Napisz program, który pyta użytkownika, czy powtórzyć działanie.
6. Wczytaj liczby i sumuj je, dopóki użytkownik nie wpisze 0.
7. Popraw program, w którym brakuje średnika po `while`.
8. Popraw program, w którym pętla działa bez końca.
9. Porównaj działanie `while` i `do while` dla tego samego fałszywego warunku.
10. Wymyśl własny przykład, w którym kod powinien wykonać się co najmniej raz.

26. for

04-petle/03-for.md

Pętla for

Cel lekcji

Nauczysz się używać pętli for, gdy z góry znasz liczbę powtórzeń albo chcesz sterować pętlą za pomocą licznika.

Krótkie przypomnienie pętli

Pętla while działa dopóki warunek ma wartość true.

Pętla do while wykonuje blok instrukcji co najmniej raz, a dopiero potem sprawdza warunek.

Pętla for jest szczególnie wygodna wtedy, gdy znamy liczbę powtórzeń. Często używa licznika, który zwiększa się lub zmniejsza po każdym obiegu pętli.

Składnia pętli for

Ogólny schemat pętli for wygląda tak:

Kod: csharp

```
for (inicjalizacja; warunek; zmiana)
{
    // instrukcje wykonywane w pętli
}
```

Nagłówek pętli for ma trzy części:

- inicjalizacja wykonuje się raz na początku,
- warunek jest sprawdzany przed każdym obiegiem pętli,
- zmiana licznika wykonuje się po wykonaniu bloku pętli.

Jeżeli warunek ma wartość true, blok pętli się wykonuje. Po wykonaniu bloku program zmienia licznik i ponownie sprawdza warunek.

Typowy zapis:

Kod: csharp

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

W tym zapisie:

- `int i = 1` oznacza start licznika,
- `i <= 5` oznacza warunek działania pętli,
- `i++` oznacza zwiększenie licznika o 1 po każdym obiegu.

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
```

```

flowchart LR
    A([Start]) --> B["Inicjalizacja<br/>int i = 1"]
    B --> C{"Warunek<br/>i <= 5?"}
    C -- "Ciało pętli" --> D["Zmiana<br/>i++"]
    D --> E["Koniec"]
    C -- "tak" --> D
    C -- "nie" --> E

```

Diagram pokazuje kolejność części pętli for: start licznika, sprawdzenie warunku, wykonanie bloku i zmianę licznika.

Pierwszy przykład: liczby od 1 do 5

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine(i);
        }
    }
}

```

Pętla zaczyna od $i = 1$. Działa dopóki $i \leq 5$. Po każdym wykonaniu bloku zmienna i zwiększa się o 1.

Program wypisze:

Kod: csharp

```

1
2
3
4
5

```

Przykład: liczby od 0 do 4

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i);
        }
    }
}

```

Ten zapis jest bardzo częsty w programowaniu.

Licznik zaczyna od 0. Warunek $i < 5$ daje wartości 0, 1, 2, 3, 4.

Taki sposób przyda się później przy tablicach, gdzie indeksy zaczynają się od 0. Na razie wystarczy zapamiętać sam schemat.

Przykład: odliczanie w dół

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 5; i >= 1; i--)
        {
            Console.WriteLine(i);
        }
    }
}
```

W tym przykładzie:

- licznik zaczyna od 5,
- pętla działa dopóki $i \geq 1$,
- $i--$ zmniejsza licznik o 1,
- program wypisze 5, 4, 3, 2, 1.

Przykład: liczby parzyste

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 2; i <= 10; i += 2)
        {
            Console.WriteLine(i);
        }
    }
}
```

Instrukcja $i += 2$ zwiększa licznik o 2. Dlatego program wypisze liczby 2, 4, 6, 8, 10.

Wyjaśnienie

Wyobraź sobie, że nauczyciel mówi:

Rozwiąż zadania od 1 do 10. Po każdym zadaniu przejdź do następnego numeru. Zatrzymaj się po zadaniu 10.

To jest właśnie pętla for:

- ustawiamy licznik,
- sprawdzamy, czy licznik nadal mieści się w zakresie,
- wykonujemy instrukcje,
- zwiększamy licznik.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int numerZadania = 1; numerZadania <= 10; numerZadania++)
        {
            Console.WriteLine($"Rozwiązuję zadanie numer {numerZadania}");
        }
    }
}
```

W tym przykładzie:

- `int numerZadania = 1` oznacza, od czego zaczynamy,
- `numerZadania <= 10` oznacza, że pętla działa dopóki warunek jest prawdziwy,
- `numerZadania++` oznacza przejście do następnego numeru po każdym obiegu.

Proste porównanie:

- `while`: najpierw sprawdzam, czy mam coś robić,
- `do while`: robię raz, potem pytam, czy powtórzyć,
- `for`: mam licznik i wykonuję zadanie określoną liczbę razy.

Przykład: suma liczb od 1 do 10

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int suma = 0;

        for (int i = 1; i <= 10; i++)
        {
            suma += i;
        }

        Console.WriteLine($"Suma liczb od 1 do 10 wynosi: {suma}");
    }
}
```

W tym programie:

- `suma` zaczyna od 0,
- w każdym obiegu dodajemy aktualną wartość `i`,
- `suma += i` oznacza: dodaj `i` do zmiennej `suma`,
- `suma` jest przykładem akumulatora, czyli zmiennej zbierającej wynik.

Przykład: wczytanie 5 liczb i obliczenie sumy

Kod: csharp

```
using System;

class Program
{
    static void Main()
```

```

{
    int suma = 0;

    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine($"Podaj liczbę {i}:");
        int liczba = int.Parse(Console.ReadLine());

        suma += liczba;
    }

    Console.WriteLine($"Suma podanych liczb: {suma}");
}

```

Pętla wykona się dokładnie 5 razy. Za każdym razem użytkownik poda jedną liczbę.

Zmienna `i` służy jako numer kolejnego powtórzenia. Zmienna `suma` przechowuje narastający wynik.

Przykład: średnia z kilku liczb

Kod: `csharp`

```

using System;

class Program
{
    static void Main()
    {
        int liczbaOcen = 3;
        double suma = 0;

        for (int i = 1; i <= liczbaOcen; i++)
        {
            Console.WriteLine($"Podaj ocenę {i}:");
            double ocena = double.Parse(Console.ReadLine());

            suma += ocena;
        }

        double srednia = suma / liczbaOcen;

        Console.WriteLine($"Średnia: {srednia:F2}");
    }
}

```

W tym przykładzie:

- `liczbaOcen` określa, ile razy pętla ma się wykonać,
- `suma` przechowuje sumę ocen,
- `srednia` to suma podzielona przez liczbę ocen,
- `{srednia:F2}` wypisuje wynik z dwoma miejscami po przecinku.

Zgadnij, ile razy wykona się pętla

Przykład z odpowiedzią:

Kod: `csharp`

```

for (int i = 1; i <= 3; i++)
{
    Console.WriteLine(i);
}

```

Odpowiedź: pętla wykona się 3 razy i wypisze 1, 2, 3.

Spróbuj najpierw samodzielnie przewidzieć wyniki kolejnych przykładów.

Kod: csharp

```
for (int i = 0; i < 4; i++)
{
    Console.WriteLine(i);
}
```

Kod: csharp

```
for (int i = 5; i >= 1; i--)
{
    Console.WriteLine(i);
}
```

Kod: csharp

```
for (int i = 2; i <= 10; i += 2)
{
    Console.WriteLine(i);
}
```

Kod: csharp

```
for (int i = 10; i < 5; i++)
{
    Console.WriteLine(i);
}
```

Odpowiedzi:

- pierwszy przykład wykona się 4 razy i wypisze 0, 1, 2, 3,
- drugi przykład wykona się 5 razy i wypisze 5, 4, 3, 2, 1,
- trzeci przykład wykona się 5 razy i wypisze 2, 4, 6, 8, 10,
- czwarty przykład nie wykona się ani razu.

Porównanie for i while

Pętlę for wybieramy zwykle wtedy, gdy:

- znamy liczbę powtórzeń,
- potrzebujemy licznika,
- chcemy przejść przez kolejne numery,
- później będziemy przechodzić po elementach tablicy.

Pętlę while wybieramy zwykle wtedy, gdy:

- nie wiemy z góry, ile razy pętla ma się wykonać,
- czekamy na spełnienie warunku,
- powtarzamy działanie do momentu zmiany sytuacji.

Przykłady:

- wypisz liczby od 1 do 10 - for,
- pytaj użytkownika, dopóki nie wpisze 0 - while albo do while.

Typowe błędy

Błąd 1: zły warunek końcowy

Przykład:

Kod: csharp

```
for (int i = 1; i < 5; i++)
{
    Console.WriteLine(i);
}
```

Ta pętla wypisze 1, 2, 3, 4. Nie wypisze 5.

Jeśli chcemy wypisać liczby od 1 do 5, warunek powinien być `i <= 5`.

Poprawnie:

Kod: csharp

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

Błąd 2: brak zmiany licznika w dobrą stronę

Niepoprawnie:

Kod: csharp

```
for (int i = 1; i <= 5; i--)
{
    Console.WriteLine(i);
}
```

Zmienna `i` zmniejsza się zamiast zwiększać. Warunek `i <= 5` może pozostawać prawdziwy, więc może powstać pętla nieskończona.

Błąd 3: średnik po for

Niepoprawnie:

Kod: csharp

```
for (int i = 1; i <= 5; i++);
{
    Console.WriteLine("To nie jest poprawny zapis pętli.");
}
```

Średnik po nagłówku `for` kończy pustą instrukcję pętli. Blok poniżej nie działa wtedy tak, jak uczeń zwykle oczekuje.

W tym kursie nie stawiamy średnika po nagłówku `for`.

Błąd 4: mylenie `<` i `<=`

Warunek `i < 5` oznacza wartości mniejsze niż 5.

Warunek `i <= 5` oznacza wartości mniejsze lub równe 5.

Ta różnica często decyduje o liczbie powtórzeń pętli.

Błąd 5: za długa lub nieczytelna pętla

Pętla powinna mieć czytelny licznik. Ciało pętli powinno być poprawnie wcięte.

Jeśli w pętli dzieje się zbyt dużo rzeczy, kod staje się trudniejszy do czytania.

Zapowiedź kolejnej lekcji

Pętla `for` porządkuje powtarzanie instrukcji z licznikiem. W kolejnych lekcjach uporządkujemy pojęcia licznika i akumulatora, a później omówimy `break` i `continue`.

Przy tablicach pojawi się też pętla `foreach`, ale na razie nie jest potrzebna.

Zapamiętaj

- `for` jest wygodny, gdy znamy liczbę powtórzeń.
- Nagłówek `for` ma trzy części: inicjalizację, warunek i zmianę.
- Licznik często nazywa się `i`.
- `i++` zwiększa licznik o 1.
- `i--` zmniejsza licznik o 1.
- `i += 2` zwiększa licznik o 2.
- Warunek decyduje, czy pętla wykona kolejny obieg.
- Trzeba uważać na różnicę między `<` i `<=`.
- Nie stawiamy średnika po nagłówku `for`.

Ćwiczenia

1. Wypisz liczby od 1 do 10 za pomocą `for`.
2. Wypisz liczby od 10 do 1 za pomocą `for`.
3. Wypisz liczby parzyste od 2 do 20.
4. Wypisz liczby nieparzyste od 1 do 19.
5. Oblicz sumę liczb od 1 do 100.
6. Wczytaj 5 liczb i oblicz ich sumę.
7. Wczytaj 4 oceny i oblicz średnią.
8. Sprawdź, ile razy wykona się podana pętla.
9. Popraw pętlę z błędnym warunkiem `<` zamiast `<=`.
10. Popraw pętlę, w której licznik zmienia się w złą stronę.
11. Napisz pętlę, która wypisze: 3, 6, 9, 12, 15.
12. Napisz własny przykład pętli `for` z licznikiem.

27. Tablice jednowymiarowe - podstawy

04-petle/04-tablice-jednowymiarowe.md

Tablice jednowymiarowe - podstawy

Cel lekcji

Nauczysz się, czym jest tablica jednowymiarowa, jak ją utworzyć oraz jak odwoływać się do jej elementów za pomocą indeksów.

Krótkie wprowadzenie

Pojedyncza zmienna przechowuje jedną wartość.

Jeśli mamy kilka ocen, punktów albo temperatur, tworzenie wielu osobnych zmiennych jest niewygodne.

Tablica pozwala przechowywać wiele wartości tego samego typu pod jedną nazwą.

Przykład problemu bez tablicy:

Kod: csharp

```
int ocena1 = 5;
int ocena2 = 4;
int ocena3 = 3;
int ocena4 = 5;
```

Przy większej liczbie danych taki zapis szybko staje się niewygodny. Trudno potem wygodnie przetwarzać takie dane w pętli.

Czym jest tablica

Tablica to struktura, która przechowuje wiele elementów tego samego typu. Każdy element ma swój numer, czyli indeks.

Tablica przypomina rząd ponumerowanych szuflad. Każda szuflada przechowuje jedną wartość. Nazwa tablicy wskazuje cały rząd szuflad, a indeks wskazuje konkretną szufladę.

Deklaracja i utworzenie tablicy

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] oceny = new int[4];

        oceny[0] = 5;
        oceny[1] = 4;
        oceny[2] = 3;
        oceny[3] = 5;

        Console.WriteLine(oceny[0]);
        Console.WriteLine(oceny[1]);
        Console.WriteLine(oceny[2]);
        Console.WriteLine(oceny[3]);
    }
}
```

```
}
```

W tym przykładzie:

- `int[]` oznacza tablicę liczb całkowitych,
- `oceny` to nazwa tablicy,
- `new int[4]` tworzy tablicę na 4 elementy,
- elementy mają indeksy 0, 1, 2, 3,
- nie ma elementu o indeksie 4,
- `oceny[0]` oznacza pierwszy element tablicy.

Indeksy zaczynają się od 0

Pierwszy element tablicy ma indeks 0.

Drugi element ma indeks 1.

Trzeci element ma indeks 2.

Ostatni element tablicy ma indeks `Length - 1`.

Tablica o długości 4:

Indeks	Element
---	---
`0`	pierwszy
`1`	drugi
`2`	trzeci
`3`	czwarty

To bardzo ważna zasada. Wiele błędów w tablicach wynika z pomylenia długości tablicy z ostatnim indeksem.

Inicjalizacja tablicy gotowymi wartościami

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        int[] oceny = { 5, 4, 3, 5 };

        Console.WriteLine(oceny[0]);
        Console.WriteLine(oceny[1]);
        Console.WriteLine(oceny[2]);
        Console.WriteLine(oceny[3]);
    }
}
```

Jeśli znamy wartości od razu, możemy zapisać je w nawiasach klamrowych.

C# sam rozpozna, że tablica ma 4 elementy. Indeksy nadal zaczynają się od 0.

Tablice różnych typów

Kod: **csharp**

```
int[] punkty = { 10, 15, 20 };
double[] temperatury = { 21.5, 22.0, 19.8 };
string[] imiona = { "Anna", "Jan", "Ewa" };
```

```
bool[] wyniki = { true, false, true };
```

Tablica przechowuje elementy jednego typu:

- `int[]` to tablica liczb całkowitych,
- `double[]` to tablica liczb rzeczywistych,
- `string[]` to tablica tekstów,
- `bool[]` to tablica wartości `true` albo `false`.

Zmiana wartości elementu tablicy

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        int[] punkty = { 10, 15, 20 };

        Console.WriteLine(punkty[1]);

        punkty[1] = 18;

        Console.WriteLine(punkty[1]);
    }
}
```

`punkty[1]` oznacza drugi element tablicy.

Najpierw ma wartość 15. Potem zmieniamy go na 18. Po zmianie wypisze się 18.

Właściwość Length

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        int[] punkty = { 10, 15, 20 };

        Console.WriteLine($"Liczba elementów: {punkty.Length}");
    }
}
```

`Length` zwraca liczbę elementów tablicy.

Dla tablicy `{ 10, 15, 20 }` wartość `Length` wynosi 3.

Ostatni indeks to `punkty.Length - 1`. W kolejnej lekcji `Length` będzie bardzo ważne przy pętli `for`.

Wyjaśnienie

Wyobraź sobie komodę z pięcioma szufladami na oceny.

Nie chcesz mieć pięciu osobnych zmiennych:

- `ocena1`,
- `ocena2`,

- ocena3,
- ocena4,
- ocena5.

Wolisz mieć jedną komodę o nazwie oceny.

Każda szuflada ma swój numer, czyli indeks:

- oceny[0] - pierwsza szuflada,
- oceny[1] - druga szuflada,
- oceny[2] - trzecia szuflada.

Tablica to uporządkowany zestaw miejsc na dane.

Nazwa tablicy mówi, o którą komodę chodzi. Indeks mówi, do której szuflady zaglądasz.

Ważne: w C# numerowanie zaczyna się od 0, więc pierwsza szuflada ma indeks 0, a nie 1.

Typowe błędy

Błąd 1: odwołanie do nieistniejącego indeksu

Kod: `csharp`

```
int[] liczby = { 10, 20, 30 };

Console.WriteLine(liczby[3]);
```

Tablica ma 3 elementy. Poprawne indeksy to 0, 1, 2.

Indeks 3 nie istnieje. Taki błąd powoduje błąd wykonania programu.

Błąd 2: mylenie długości z ostatnim indeksem

Jeśli `Length` wynosi 3, ostatni indeks to 2.

Ostatni indeks to zawsze `Length - 1`.

Błąd 3: próba przechowywania różnych typów w jednej tablicy

`int[]` przechowuje liczby całkowite. Nie można do niej wpisać tekstu.

Tablica ma jeden określony typ elementów.

Błąd 4: zbyt wiele osobnych zmiennych zamiast tablicy

Jeśli dane są podobnego rodzaju, na przykład oceny, punkty albo temperatury, tablica jest zwykle wygodniejsza niż wiele osobnych zmiennych.

Zapowiedź kolejnej lekcji

W tej lekcji odwoływaliśmy się do elementów tablicy pojedynczo.

W kolejnej lekcji połączymy tablice z pętlą `for`, aby wygodnie przechodzić po wszystkich elementach tablicy.

Zapamiętaj

- Tablica przechowuje wiele wartości tego samego typu.
- `int[]` oznacza tablicę liczb całkowitych.

- Indeks wskazuje konkretny element tablicy.
- Indeksy zaczynają się od 0.
- Pierwszy element to `tablica[0]`.
- Ostatni element to `tablica[tablica.Length - 1]`.
- `Length` oznacza liczbę elementów tablicy.
- Odwołanie do nieistniejącego indeksu powoduje błąd.

Ćwiczenia

1. Utwórz tablicę `int[]` z trzema ocenami i wypisz pierwszy element.
2. Utwórz tablicę `string[]` z trzema imionami i wypisz drugie imię.
3. Utwórz tablicę `double[]` z trzema temperaturami i wypisz ostatni element.
4. Zmień wartość drugiego elementu tablicy punktów.
5. Wypisz długość tablicy za pomocą `Length`.
6. Dla tablicy o długości 5 wskaż poprawne indeksy.
7. Wyjaśnij, dlaczego `tablica[5]` jest błędna dla tablicy pięcioelementowej.
8. Utwórz tablicę `bool[]` z trzema wartościami.
9. Przepisz przykład z `ocena1`, `ocena2`, `ocena3` na tablicę.
10. Wskaż, który element jest pierwszy, a który ostatni w podanej tablicy.

28. Pętla for po tablicy

04-petle/05-for-po-tablicy.md

Pętla for po tablicy

Cel lekcji

Nauczysz się przechodzić po wszystkich elementach tablicy za pomocą pętli for.

Krótkie przypomnienie

Tablica przechowuje wiele wartości tego samego typu.

Pierwszy element tablicy ma indeks 0.

Ostatni element ma indeks `Length - 1`.

`Length` oznacza liczbę elementów tablicy.

Pętla for dobrze pasuje do tablic, bo licznik pętli może być indeksem elementu.

Problem bez pętli

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        int[] oceny = { 5, 4, 3, 5 };

        Console.WriteLine(oceny[0]);
        Console.WriteLine(oceny[1]);
        Console.WriteLine(oceny[2]);
        Console.WriteLine(oceny[3]);
    }
}
```

Dla czterech elementów jeszcze da się to zapisać ręcznie.

Dla 20 albo 100 elementów taki sposób jest niewygodny. Lepiej użyć pętli.

Pętla for po tablicy

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        int[] oceny = { 5, 4, 3, 5 };

        for (int i = 0; i < oceny.Length; i++)
        {
            Console.WriteLine(oceny[i]);
        }
    }
}
```

W tym przykładzie:

- `i` zaczyna od 0, bo pierwszy indeks tablicy to 0,
- warunek `i < oceny.Length` oznacza, że pętla działa dla poprawnych indeksów,
- dla tablicy długości 4 poprawne indeksy to 0, 1, 2, 3,
- gdy `i` osiągnie 4, warunek `i < oceny.Length` będzie fałszywy,
- `oceny[i]` oznacza element tablicy o aktualnym indeksie.

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart LR
  A([Start]) --> B["i = 0"]
  B --> C{"i < Length?"}
  C -- "tak" --> D["Odczytaj<br/>tablica[i]"]
  D --> E["i++"]
  E --> C
  C -- "nie" --> F([Koniec])
```

Diagram pokazuje, że pętla przechodzi tylko po poprawnych indeksach tablicy.

Dlaczego używamy `i < tablica.Length`

Kod: csharp

```
int[] liczby = { 10, 20, 30 };

for (int i = 0; i < liczby.Length; i++)
{
    Console.WriteLine(liczby[i]);
}
```

Dla tej tablicy `Length` wynosi 3.

Poprawne indeksy to 0, 1, 2.

Warunek `i < liczby.Length` daje kolejno `i = 0`, `i = 1`, `i = 2`.

Warunek `i <= liczby.Length` próbowałby użyć indeksu 3, a indeks 3 nie istnieje.

Niepoprawnie:

Kod: csharp

```
for (int i = 0; i <= liczby.Length; i++)
{
    Console.WriteLine(liczby[i]);
}
```

To typowy błąd. Pętla dojdzie do `i == 3`, ale `liczby[3]` nie istnieje.

Wypisywanie indeksu i wartości

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] punkty = { 10, 15, 20 };

        for (int i = 0; i < punkty.Length; i++)
        {
            Console.WriteLine($"Indeks {i}: {punkty[i]}");
        }
    }
}
```

`i` jest indeksem.

`punkty[i]` jest wartością zapisaną pod tym indeksem.

Taki zapis pomaga zrozumieć różnicę między numerem elementu a jego wartością.

Zmiana elementów tablicy w pętli

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] punkty = { 10, 15, 20 };

        for (int i = 0; i < punkty.Length; i++)
        {
            punkty[i] = punkty[i] + 1;
        }

        for (int i = 0; i < punkty.Length; i++)
        {
            Console.WriteLine(punkty[i]);
        }
    }
}
```

Pierwsza pętla zwiększa każdy element tablicy o 1.

Druga pętla wypisuje zmienione wartości.

`punkty[i]` może być użyte zarówno do odczytu, jak i do zapisu.

Wczytywanie danych do tablicy

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = new int[5];

        for (int i = 0; i < liczby.Length; i++)
        {
            Console.WriteLine($"Podaj liczbę o indeksie {i}:");
            liczby[i] = int.Parse(Console.ReadLine());
        }
    }
}
```

```

    }

    Console.WriteLine("Zawartość tablicy:");

    for (int i = 0; i < liczby.Length; i++)
    {
        Console.WriteLine(liczby[i]);
    }
}

```

`new int[5]` tworzy tablicę na 5 liczb.

Pierwsza pętla wczytuje dane do tablicy. Druga pętla wypisuje dane z tablicy.

Używamy `Length`, aby kod działał poprawnie także po zmianie rozmiaru tablicy.

Indeks w tablicy a numer dla użytkownika

Indeksy w tablicy zaczynają się od 0, ale użytkownikowi często pokazujemy numerację od 1.

Wtedy w komunikacie można użyć `i + 1`.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[] oceny = new int[3];

        for (int i = 0; i < oceny.Length; i++)
        {
            Console.WriteLine($"Podaj ocenę numer {i + 1}:");
            oceny[i] = int.Parse(Console.ReadLine());
        }

        for (int i = 0; i < oceny.Length; i++)
        {
            Console.WriteLine($"Ocena numer {i + 1}: {oceny[i]}");
        }
    }
}

```

`i` jest indeksem technicznym.

`i + 1` jest numerem czytelnym dla użytkownika.

Nie zmienia to indeksowania tablicy. Pierwszy element nadal ma indeks 0.

Wyjaśnienie

Wyobraź sobie rząd szuflad z ocenami.

Każda szuflada ma numer techniczny: 0, 1, 2, 3.

Pętla `for` to osoba, która idzie od pierwszej szuflady do ostatniej:

- zaczyna od szuflady 0,
- sprawdza, czy numer szuflady jest jeszcze mniejszy niż liczba szuflad,
- zagląda do szuflady,
- przechodzi do następnej.

Kod:

Kod: csharp

```
for (int i = 0; i < oceny.Length; i++)
{
    Console.WriteLine(oceny[i]);
}
```

oznacza: dla każdego poprawnego indeksu tablicy wypisz wartość z tej szuflady.

Typowe błędy

Błąd 1: użycie `i <= tablica.Length`

Niepoprawnie:

Kod: csharp

```
for (int i = 0; i <= liczby.Length; i++)
{
    Console.WriteLine(liczby[i]);
}
```

Poprawnie:

Kod: csharp

```
for (int i = 0; i < liczby.Length; i++)
{
    Console.WriteLine(liczby[i]);
}
```

Length to liczba elementów. Ostatni indeks to `Length - 1`, dlatego używamy `i < Length`.

Błąd 2: zaczynanie od 1

Niepoprawnie:

Kod: csharp

```
for (int i = 1; i < liczby.Length; i++)
{
    Console.WriteLine(liczby[i]);
}
```

Ten kod pomija element o indeksie 0. Pierwszy element tablicy nie zostanie wypisany.

Błąd 3: mylenie indeksu z wartością

`i` to indeks.

`tablica[i]` to wartość.

`Console.WriteLine(i)` wypisuje indeks.

`Console.WriteLine(tablica[i])` wypisuje element tablicy.

Błąd 4: wpisanie danych do zmiennej zamiast do tablicy

Jeśli dane mają zostać zapamiętane, trzeba przypisać je do tablicy:

Kod: csharp

```
liczby[i] = int.Parse(Console.ReadLine());
```

Sama zmienna `liczba` przechowuje tylko jedną aktualną wartość.

Zapowiedź kolejnej lekcji

W tej lekcji nauczyliśmy się przechodzić po elementach tablicy pętlą `for`.

W kolejnej lekcji użyjemy tej techniki do sumowania elementów tablicy i obliczania średniej.

Zapamiętaj

- Pętla `for` dobrze pasuje do tablic.
- Indeks pętli zwykle zaczyna się od 0.
- Warunek powinien mieć postać `i < tablica.Length`.
- Ostatni indeks to `tablica.Length - 1`.
- `tablica[i]` oznacza element o aktualnym indeksie.
- `i` oznacza indeks, a `tablica[i]` oznacza wartość.
- Przy komunikatach dla użytkownika można używać `i + 1`.

Ćwiczenia

1. Utwórz tablicę z pięcioma liczbami i wypisz wszystkie elementy za pomocą `for`.
2. Utwórz tablicę `string[]` z imionami i wypisz każde imię.
3. Wypisz indeks i wartość każdego elementu tablicy.
4. Wczytaj 4 liczby do tablicy i wypisz je.
5. Wczytaj 3 oceny do tablicy i wypisz je z numerami od 1.
6. Zwiększ każdy element tablicy punktów o 1.
7. Zmień każdy element tablicy liczb całkowitych na jego podwojoną wartość.
8. Popraw pętlę z błędem `i <= tablica.Length`.
9. Popraw pętlę, która pomija element o indeksie 0.
10. Wyjaśnij różnicę między `i` oraz `tablica[i]`.

29. Suma i średnia elementów tablicy

04-petle/06-suma-srednia-tablicy.md

Suma i średnia elementów tablicy

Cel lekcji

Nauczysz się obliczać sumę elementów tablicy oraz średnią na podstawie sumy i liczby elementów.

Krótkie przypomnienie

Tablica przechowuje wiele wartości.

Pętla for pozwala przejść po wszystkich elementach tablicy.

Akumulator pozwala gromadzić wynik.

Do sumowania elementów tablicy używamy zwykle zmiennej suma.

Suma elementów tablicy

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 4, 7, 2, 9 };

        int suma = 0;

        for (int i = 0; i < liczby.Length; i++)
        {
            suma += liczby[i];
        }

        Console.WriteLine($"Suma: {suma}");
    }
}
```

W tym przykładzie:

- suma zaczyna od 0,
- pętla przechodzi po indeksach 0, 1, 2, 3,
- `liczby[i]` oznacza aktualny element tablicy,
- `suma += liczby[i]` dodaje aktualny element do sumy,
- po zakończeniu pętli suma zawiera sumę wszystkich elementów.

Przebieg działania krok po kroku

Dla tablicy:

Kod: csharp

```
int[] liczby = { 4, 7, 2, 9 };
```

akumulator zmienia się tak:

i	liczby[i]	suma po dodaniu
---	---	---
`0`	`4`	`4`
`1`	`7`	`11`
`2`	`2`	`13`
`3`	`9`	`22`

Tabela pokazuje, jak akumulator zmienia się w kolejnych obiegach pętli.

Ostateczna suma wynosi 22.

Średnia elementów tablicy

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] oceny = { 5, 4, 3, 5 };

        int suma = 0;

        for (int i = 0; i < oceny.Length; i++)
        {
            suma += oceny[i];
        }

        double srednia = (double)suma / oceny.Length;

        Console.WriteLine($"Suma ocen: {suma}");
        Console.WriteLine($"Średnia ocen: {srednia:F2}");
    }
}
```

Średnia to suma podzielona przez liczbę elementów.

Liczbę elementów pobieramy z `oceny.Length`.

Użycie `(double)suma` wymusza dzielenie rzeczywiste. Bez tego moglibyśmy otrzymać dzielenie całkowite.

Zapis `{srednia:F2}` wypisuje wynik z dwoma miejscami po przecinku.

Dlaczego używamy Length

Nie wpisujemy na sztywno liczby elementów, jeśli można użyć `tablica.Length`.

Dzięki temu kod łatwiej poprawić, gdy zmieni się rozmiar tablicy.

Jeśli tablica ma 4 elementy, `Length` wynosi 4. Jeśli później dodamy elementy, pętla nadal może działać poprawnie.

Lepszy zapis:

Kod: csharp

```
for (int i = 0; i < oceny.Length; i++)
{
    suma += oceny[i];
}
```

Słabszy zapis:

Kod: csharp

```
for (int i = 0; i < 4; i++)
{
    suma += oceny[i];
}
```

Liczba 4 działa tylko dla konkretnej tablicy. Length jest bezpieczniejsze i bardziej uniwersalne.

Wczytywanie liczb do tablicy i obliczanie średniej

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = new int[5];

        for (int i = 0; i < liczby.Length; i++)
        {
            Console.WriteLine($"Podaj liczbę numer {i + 1}:");
            liczby[i] = int.Parse(Console.ReadLine());
        }

        int suma = 0;

        for (int i = 0; i < liczby.Length; i++)
        {
            suma += liczby[i];
        }

        double srednia = (double)suma / liczby.Length;

        Console.WriteLine($"Suma: {suma}");
        Console.WriteLine($"Średnia: {srednia:F2}");
    }
}
```

Pierwsza pętla wczytuje dane do tablicy.

Druga pętla oblicza sumę.

Średnia powstaje po zakończeniu sumowania.

$i + 1$ służy tylko do czytelnej numeracji dla użytkownika.

Suma i średnia liczb rzeczywistych

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        double[] temperatury = { 21.5, 22.0, 19.8, 20.7 };

        double suma = 0;

        for (int i = 0; i < temperatury.Length; i++)
        {
            suma += temperatury[i];
        }

        double srednia = suma / temperatury.Length;
    }
}
```

```

        Console.WriteLine($"Średnia temperatura: {srednia:F2}");
    }
}

```

Dla tablicy `double[]` zmienna `suma` też powinna być typu `double`.

Nie trzeba tu rzutować sumy na `double`, bo `suma` już jest typu `double`.

Wyjaśnienie

Wyobraź sobie, że masz kilka kartek z punktami uczniów: 4, 7, 2, 9.

Chcesz policzyć sumę.

Bierzesz pustą kartkę z napisem `suma = 0`.

Potem przechodzisz po kolei po każdej kartce:

- dodajesz 4,
- potem dodajesz 7,
- potem dodajesz 2,
- potem dodajesz 9.

Na końcu na kartce masz wynik 22.

W programie ta kartka to zmienna `suma`.

Przechodzenie po kartkach to pętla `for`.

Aktualna kartka to `liczby[i]`.

Średnia to następny krok: bierzesz sumę i dzielisz przez liczbę kartek.

Typowe błędy

Błąd 1: brak wyzerowania sumy przed pętlą

`suma` powinna zwykle zaczynać od 0.

Jeśli `suma` ma wcześniejszą wartość, wynik będzie błędny.

Poprawnie:

Kod: `csharp`

```
int suma = 0;
```

Błąd 2: dzielenie całkowite przy średniej

Niepoprawnie:

Kod: `csharp`

```
int suma = 17;
int liczbaElementow = 4;

double srednia = suma / liczbaElementow;
```

`suma` i `liczbaElementow` są typu `int`. Wynik dzielenia może zostać policzony jako liczba całkowita, a część po przecinku zostanie utracona.

Poprawnie:

Kod: `csharp`

```
double srednia = (double)suma / liczbaElementow;
```

Błąd 3: użycie złej liczby elementów

Zamiast wpisywać ręcznie liczbę elementów, lepiej użyć `tablica.Length`.

Wtedy kod jest mniej podatny na błędy.

Błąd 4: obliczanie średniej w złym miejscu

Sumę liczymy w pętli.

Średnią najczęściej obliczamy po zakończeniu pętli, gdy mamy już pełną sumę.

Błąd 5: pomylenie indeksu z wartością

`i` to indeks.

`liczby[i]` to wartość elementu.

Do sumy dodajemy `liczby[i]`, a nie samo `i`, jeśli chcemy sumować elementy tablicy.

Zgadnij, jaki będzie wynik

Przykład z odpowiedzią:

Kod: csharp

```
int[] liczby = { 2, 4, 6 };
int suma = 0;

for (int i = 0; i < liczby.Length; i++)
{
    suma += liczby[i];
}

Console.WriteLine(suma);
```

Odpowiedź: program wypisze 12.

Spróbuj najpierw samodzielnie przewidzieć wyniki kolejnych przykładów.

Kod: csharp

```
int[] liczby = { 1, 3, 5, 7 };
int suma = 0;

for (int i = 0; i < liczby.Length; i++)
{
    suma += liczby[i];
}

Console.WriteLine(suma);
```

Kod: csharp

```
int[] punkty = { 10, 20, 30 };
int suma = 0;

for (int i = 0; i < punkty.Length; i++)
{
    suma += punkty[i];
}

double srednia = (double)suma / punkty.Length;

Console.WriteLine(srednia);
```

Kod: csharp

```
double[] temperatury = { 20.5, 21.5 };
```

```
double suma = 0;

for (int i = 0; i < temperatury.Length; i++)
{
    suma += temperatury[i];
}

Console.WriteLine(suma);
```

Odpowiedzi:

- pierwszy przykład wypisze 16,
- drugi przykład wypisze 20,
- trzeci przykład wypisze 42.

Zapowiedź kolejnej lekcji

Suma i średnia to jedne z najczęstszych operacji na tablicach.

W kolejnej lekcji użyjemy pętli do znalezienia najmniejszego i największego elementu tablicy.

Zapamiętaj

- Suma elementów tablicy wymaga akumulatora.
- Akumulator zwykle zaczyna od 0.
- W pętli dodajemy do sumy `tablica[i]`.
- `Length` oznacza liczbę elementów.
- Średnia to suma podzielona przez liczbę elementów.
- Przy średniej z liczb całkowitych warto wymusić dzielenie rzeczywiste przez `(double)`.
- `{wynik:F2}` pozwala wypisać wynik z dwoma miejscami po przecinku.

Ćwiczenia

1. Utwórz tablicę pięciu liczb i oblicz ich sumę.
2. Utwórz tablicę pięciu liczb i oblicz ich średnią.
3. Wczytaj 5 liczb do tablicy i oblicz ich sumę.
4. Wczytaj 5 liczb do tablicy i oblicz średnią.
5. Utwórz tablicę ocen typu `int[]` i oblicz średnią ocen.
6. Utwórz tablicę temperatur typu `double[]` i oblicz średnią temperaturę.
7. Popraw program, który dzieli dwie liczby `int` i traci część po przecinku.
8. Popraw program, który dodaje do sumy `i` zamiast `tablica[i]`.
9. Sprawdź, co się stanie, gdy zmienisz liczbę elementów tablicy, ale używasz `Length`.
10. Napisz własny przykład sumowania elementów tablicy.

30. Minimum i maksimum elementów tablicy

04-petle/07-minimum-maksimum-tablicy.md

Minimum i maksimum elementów tablicy

Cel lekcji

Nauczysz się znajdować najmniejszy i największy element tablicy za pomocą pętli oraz zmiennych pomocniczych `minimum` i `maksimum`.

Krótkie wprowadzenie

Mając wiele liczb w tablicy, często chcemy znaleźć najmniejszą albo największą wartość.

Nie robimy tego ręcznie. Używamy pętli i porównujemy kolejne elementy tablicy.

Dlaczego zaczynamy od pierwszego elementu

Nie ustawiamy `minimum` na 0, bo tablica może zawierać tylko liczby dodatnie albo tylko liczby ujemne.

Nie ustawiamy `maksimum` na 0, bo tablica może zawierać same liczby ujemne.

Bezpieczny sposób to przyjąć pierwszy element tablicy jako początkowe `minimum` albo `maksimum`.

Kod: csharp

```
int minimum = liczby[0];
int maksimum = liczby[0];
```

Na początku zakładamy, że pierwszy element jest jednocześnie najmniejszy i największy.

Potem sprawdzamy kolejne elementy:

- jeśli znajdziemy mniejszy, aktualizujemy `minimum`,
- jeśli znajdziemy większy, aktualizujemy `maksimum`.

Znalezienie minimum

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 7, 3, 9, 2, 5 };

        int minimum = liczby[0];

        for (int i = 1; i < liczby.Length; i++)
        {
            if (liczby[i] < minimum)
            {
                minimum = liczby[i];
            }
        }

        Console.WriteLine($"Minimum: {minimum}");
    }
}
```

W tym przykładzie:

- minimum zaczyna od `liczby[0]`,
- pętla zaczyna od `i = 1`, bo pierwszy element jest już przyjęty jako początkowe minimum,
- jeśli `liczby[i]` jest mniejsze od minimum, aktualizujemy minimum,
- po zakończeniu pętli minimum zawiera najmniejszą wartość.

Przebieg szukania minimum krok po kroku

Dla tablicy:

Kod: **csharp**

```
int[] liczby = { 7, 3, 9, 2, 5 };
```

porównania wyglądają tak:

i	liczby[i]	minimum przed porównaniem	decyzja	minimum po porównaniu
1	3	7	3 < 7, zmiana minimum	3
2	9	3	9 < 3 fałsz	3
3	2	3	2 < 3, zmiana minimum	2
4	5	2	5 < 2 fałsz	2

Ostateczne minimum to 2.

Znalezienie maksimum

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 7, 3, 9, 2, 5 };

        int maksimum = liczby[0];

        for (int i = 1; i < liczby.Length; i++)
        {
            if (liczby[i] > maksimum)
            {
                maksimum = liczby[i];
            }
        }

        Console.WriteLine($"Maksimum: {maksimum}");
    }
}
```

maksimum zaczyna od `liczby[0]`.

Jeśli znajdziemy większą wartość, aktualizujemy maksimum.

Po zakończeniu pętli maksimum zawiera największą wartość.

Minimum i maksimum w jednej pętli

Kod: **csharp**

```
using System;

class Program
```



```

{
    static void Main()
    {
        int[] liczby = { 7, 3, 9, 2, 5 };

        int minimum = liczby[0];
        int maksimum = liczby[0];

        for (int i = 1; i < liczby.Length; i++)
        {
            if (liczby[i] < minimum)
            {
                minimum = liczby[i];
            }

            if (liczby[i] > maksimum)
            {
                maksimum = liczby[i];
            }
        }

        Console.WriteLine($"Minimum: {minimum}");
        Console.WriteLine($"Maksimum: {maksimum}");
    }
}

```

W jednej pętli można sprawdzać oba warunki.

Każdy element porównujemy z aktualnym minimum i aktualnym maksimum. Na końcu mamy obie wartości.

Wczytanie liczb do tablicy i znalezienie minimum oraz maksimum

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[] liczby = new int[5];

        for (int i = 0; i < liczby.Length; i++)
        {
            Console.WriteLine($"Podaj liczbę numer {i + 1}:");
            liczby[i] = int.Parse(Console.ReadLine());
        }

        int minimum = liczby[0];
        int maksimum = liczby[0];

        for (int i = 1; i < liczby.Length; i++)
        {
            if (liczby[i] < minimum)
            {
                minimum = liczby[i];
            }

            if (liczby[i] > maksimum)
            {
                maksimum = liczby[i];
            }
        }

        Console.WriteLine($"Minimum: {minimum}");
        Console.WriteLine($"Maksimum: {maksimum}");
    }
}

```

```
    }  
}
```

Pierwsza pętla wczytuje dane.

Druga pętla szuka minimum i maksimum.

Zaczynamy porównywanie od indeksu 1, bo indeks 0 jest wartością startową.

Wyjaśnienie

Wyobraź sobie, że masz stos kartek z wynikami uczniów.

Bierzesz pierwszą kartkę i mówisz: na razie to jest najmniejszy wynik i największy wynik.

Potem bierzesz kolejne kartki:

- jeśli wynik jest mniejszy niż zapisane minimum, podmieniasz minimum,
- jeśli wynik jest większy niż zapisane maksimum, podmieniasz maksimum.

Na końcu nie musisz pamiętać wszystkich porównań. Wystarczy, że masz aktualne minimum i aktualne maksimum.

W programie:

- `liczby[0]` to pierwsza kartka,
- `minimum` to najmniejszy znaleziony wynik,
- `maksimum` to największy znaleziony wynik,
- pętla `for` przechodzi po kolejnych kartkach.

Typowe błędy

Błąd 1: ustawienie minimum na 0

Niepoprawnie:

Kod: `csharp`

```
int minimum = 0;
```

Jeśli wszystkie liczby w tablicy są dodatnie, wynik może przypadkowo wyjść 0, mimo że 0 nie ma w tablicy.

Minimum powinno pochodzić z tablicy.

Poprawnie:

Kod: `csharp`

```
int minimum = liczby[0];
```

Błąd 2: ustawienie maksimum na 0

Niepoprawnie:

Kod: `csharp`

```
int maksimum = 0;
```

Jeśli wszystkie liczby są ujemne, program błędnie uzna 0 za maksimum.

Maksimum powinno pochodzić z tablicy.

Poprawnie:

Kod: csharp

```
int maksimum = liczby[0];
```

Błąd 3: rozpoczynanie pętli od złego indeksu

Jeśli `minimum = liczby[0]`, pętlę można zacząć od `i = 1`.

Można też zacząć od `i = 0`, ale wtedy pierwszy element zostanie porównany sam ze sobą.

W tej lekcji stosujemy `i = 1`, bo jest to czytelne.

Błąd 4: pomylenie znaków porównania

Do minimum używamy `<`.

Do maksimum używamy `>`.

Pomylenie znaków daje odwrotny wynik.

Błąd 5: pusta tablica

W tej lekcji zakładamy, że tablica ma co najmniej jeden element.

Dla pustej tablicy zapis `liczby[0]` byłby błędny. Obsługę takich przypadków można omówić później.

Zgadnij, jaki będzie wynik

Przykład z odpowiedzią:

Kod: csharp

```
int[] liczby = { 8, 4, 6 };
int minimum = liczby[0];

for (int i = 1; i < liczby.Length; i++)
{
    if (liczby[i] < minimum)
    {
        minimum = liczby[i];
    }
}

Console.WriteLine(minimum);
```

Odpowiedź: program wypisze 4.

Spróbuj najpierw samodzielnie przewidzieć wyniki kolejnych przykładów.

Kod: csharp

```
int[] liczby = { 3, 9, 1, 5 };
int maksimum = liczby[0];

for (int i = 1; i < liczby.Length; i++)
{
    if (liczby[i] > maksimum)
    {
        maksimum = liczby[i];
    }
}

Console.WriteLine(maksimum);
```

Kod: csharp

```
int[] liczby = { -5, -2, -9 };
int maksimum = liczby[0];
```

```

for (int i = 1; i < liczby.Length; i++)
{
    if (liczby[i] > maksimum)
    {
        maksimum = liczby[i];
    }
}

Console.WriteLine(maksimum);

```

Kod: csharp

```

int[] liczby = { 10, 10, 10 };
int minimum = liczby[0];

for (int i = 1; i < liczby.Length; i++)
{
    if (liczby[i] < minimum)
    {
        minimum = liczby[i];
    }
}

Console.WriteLine(minimum);

```

Odpowiedzi:

- pierwszy przykład wypisze 9,
- drugi przykład wypisze -2,
- trzeci przykład wypisze 10.

Zapowiedź kolejnej lekcji

Minimum i maksimum to podstawowe operacje na tablicach.

W kolejnej lekcji można wykorzystać warunki do zliczania elementów tablicy spełniających określone kryteria, na przykład liczb dodatnich, parzystych albo większych od podanej wartości.

Zapamiętaj

- Minimum to najmniejsza wartość w tablicy.
- Maksimum to największa wartość w tablicy.
- Bezpiecznie zaczynać od tablica[0].
- Do minimum używamy porównania <.
- Do maksimum używamy porównania >.
- Pętla może zaczynać się od indeksu 1, jeśli tablica[0] jest wartością startową.
- Nie ustawiamy minimum i maksimum na 0 bez uzasadnienia.
- Zakładamy, że tablica ma co najmniej jeden element.

Ćwiczenia

1. Utwórz tablicę liczb i znajdź jej minimum.
2. Utwórz tablicę liczb i znajdź jej maksimum.
3. Wczytaj 5 liczb do tablicy i znajdź minimum.
4. Wczytaj 5 liczb do tablicy i znajdź maksimum.
5. Wczytaj 5 liczb do tablicy i znajdź minimum oraz maksimum w jednej pętli.

6. Sprawdź działanie programu dla samych liczb ujemnych.
7. Popraw program, w którym minimum ustawiono na 0.
8. Popraw program, w którym maksimum ustawiono na 0.
9. Wyjaśnij, dlaczego ostatni indeks tablicy to `Length - 1`.
10. Napisz własny przykład szukania minimum albo maksimum.

31. Zliczanie elementów tablicy spełniających warunek

04-petle/08-licznik-akumulator.md

Zliczanie elementów tablicy spełniających warunek

Cel lekcji

Nauczysz się rozróżniać licznik i akumulator oraz używać ich przy zliczaniu elementów tablicy spełniających warunek.

Krótkie wprowadzenie

W pętlach często potrzebujemy czegoś więcej niż samo powtarzanie instrukcji.

Czasem chcemy policzyć, ile razy coś się zdarzyło. Czasem chcemy zgromadzić wynik, na przykład sumę liczb.

Do takich zadań służą licznik i akumulator.

Licznik

Licznik to zmienna, która zwykle zwiększa się o 1, aby policzyć liczbę wystąpień, powtórzeń albo elementów tablicy spełniających warunek.

Przykład prostego licznika:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int licznik = 0;

        for (int i = 1; i <= 5; i++)
        {
            licznik++;
        }

        Console.WriteLine($"Liczba powtórzeń: {licznik}");
    }
}
```

W tym przykładzie:

- licznik zaczyna od 0,
- licznik++ zwiększa jego wartość o 1,
- po pięciu powtórzeniach licznik ma wartość 5.

Akumulator

Akumulator to zmienna, która gromadzi wynik, najczęściej sumę. Do akumulatora dodajemy kolejne wartości.

Przykład akumulatora:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int suma = 0;

        for (int i = 1; i <= 5; i++)
        {
            suma += i;
        }

        Console.WriteLine($"Suma: {suma}");
    }
}
```

W tym przykładzie:

- suma zaczyna od 0,
- suma += i oznacza dodanie aktualnej wartości i do sumy,
- po zakończeniu pętli zmienna suma przechowuje wynik.

Licznik a akumulator: różnica

Pojęcie	Do czego służy	Typowa operacja	Przykład
Licznik	liczy wystąpienia	<code>`licznik++`</code>	liczba liczb parzystych
Akumulator	gromadzi wynik	<code>`suma += liczba`</code>	suma podanych liczb

Licznik najczęściej zwiększamy o 1. Akumulator zwiększamy o wartość, którą chcemy dodać do wyniku.

Nazwa zmiennej powinna pokazywać jej rolę, na przykład `liczbaParzystych` albo `sumaPunktow`.

Przykład: zliczanie liczb parzystych

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczbaParzystych = 0;

        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine($"Podaj liczbę {i}:");
            int liczba = int.Parse(Console.ReadLine());

            if (liczba % 2 == 0)
            {
                liczbaParzystych++;
            }
        }

        Console.WriteLine($"Liczba parzystych: {liczbaParzystych}");
    }
}
```

W tym programie:

- liczbaParzystych jest licznikiem,
- zwiększamy ją tylko wtedy, gdy liczba jest parzysta,
- warunek `liczba % 2 == 0` sprawdza parzystość.

Przykład: suma podanych liczb

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int suma = 0;

        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine($"Podaj liczbę {i}:");
            int liczba = int.Parse(Console.ReadLine());

            suma += liczba;
        }

        Console.WriteLine($"Suma: {suma}");
    }
}
```

W tym programie:

- suma jest akumulatorem,
- każda podana liczba jest dodawana do sumy,
- po zakończeniu pętli mamy łączny wynik.

Przykład: suma i średnia

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczbaOcen = 4;
        double suma = 0;

        for (int i = 1; i <= liczbaOcen; i++)
        {
            Console.WriteLine($"Podaj ocenę {i}:");
            double ocena = double.Parse(Console.ReadLine());

            suma += ocena;
        }

        double srednia = suma / liczbaOcen;

        Console.WriteLine($"Średnia: {srednia:F2}");
    }
}
```

W tym przykładzie:

- suma jest akumulatorem,

- liczba0cen mówi, przez ile dzielimy sumę,
- średnia to suma podzielona przez liczbę wartości.

Przykład: licznik i akumulator razem

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczbaDodatnich = 0;
        int sumaDodatnich = 0;

        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine($"Podaj liczbę {i}:");
            int liczba = int.Parse(Console.ReadLine());

            if (liczba > 0)
            {
                liczbaDodatnich++;
                sumaDodatnich += liczba;
            }
        }

        Console.WriteLine($"Liczba dodatnich: {liczbaDodatnich}");
        Console.WriteLine($"Suma dodatnich: {sumaDodatnich}");
    }
}
```

W tym programie:

- `liczbaDodatnich` jest licznikiem,
- `sumaDodatnich` jest akumulatorem,
- obie zmienne są aktualizowane tylko dla liczb dodatnich.

Wyjaśnienie

Wyobraź sobie, że sprawdzasz kartkówki.

Masz dwa pudełka:

- pierwsze pudełko służy do liczenia, ile prac już sprawdziłeś,
- drugie pudełko służy do zbierania punktów ze wszystkich prac.

Pierwsze pudełko to licznik. Po każdej pracy zwiększasz go o 1.

Drugie pudełko to akumulator. Do niego dodajesz liczbę punktów z kolejnej pracy.

W programie:

- `licznik++` oznacza: policz jeszcze jeden element,
- `suma += punkty` oznacza: dodaj punkty do łącznej sumy.

Typowe wartości początkowe

Licznik zwykle zaczyna od 0.

Suma zwykle zaczyna od 0.

Dla średniej najpierw liczymy sumę, a potem dzielimy ją przez liczbę elementów.

Jeśli zliczamy elementy tablicy spełniające warunek, zwiększamy licznik wewnątrz instrukcji `if`.

Kod: `csharp`

```
int liczbaParzystych = 0;
int suma = 0;
```

Typowe błędy

Błąd 1: brak wartości początkowej

Licznik i akumulator trzeba poprawnie zainicjalizować. Najczęściej zaczynamy od 0.

Błąd 2: zwiększanie licznika w złym miejscu

Jeśli licznik liczb parzystych zwiększamy poza `if`, policzymy wszystkie liczby, a nie tylko parzyste.

Niepoprawnie:

Kod: `csharp`

```
if (liczba % 2 == 0)
{
    Console.WriteLine("Parzysta");
}

liczbaParzystych++;
```

Poprawnie:

Kod: `csharp`

```
if (liczba % 2 == 0)
{
    liczbaParzystych++;
}
```

Błąd 3: mylenie licznika z akumulatorem

Licznik zwykle zwiększamy o 1.

Akumulator zwiększamy o wartość, na przykład:

Kod: `csharp`

```
suma += liczba;
```

Błąd 4: dzielenie przez zły licznik przy średniej

Jeśli średnia dotyczy wszystkich liczb, dzielimy przez liczbę wszystkich liczb.

Jeśli średnia dotyczy tylko liczb dodatnich, dzielimy przez liczbę dodatnich.

Trzeba uważać, żeby licznik nie był równy 0. Pełne zabezpieczanie takiej sytuacji omówimy później.

Zgadnij, co będzie wynikiem

Przykład z odpowiedzią:

Kod: `csharp`

```
int licznik = 0;

for (int i = 1; i <= 3; i++)
{
    licznik++;
}
```

```
Console.WriteLine(licznik);
```

Odpowiedź: program wypisze 3.

Spróbuj najpierw samodzielnie przewidzieć wyniki kolejnych przykładów.

Kod: csharp

```
int suma = 0;

for (int i = 1; i <= 4; i++)
{
    suma += i;
}

Console.WriteLine(suma);
```

Kod: csharp

```
int liczbaParzystych = 0;

for (int i = 1; i <= 6; i++)
{
    if (i % 2 == 0)
    {
        liczbaParzystych++;
    }
}

Console.WriteLine(liczbaParzystych);
```

Kod: csharp

```
int suma = 0;
int licznik = 0;

for (int i = 1; i <= 5; i++)
{
    if (i > 2)
    {
        suma += i;
        licznik++;
    }
}

Console.WriteLine(suma);
Console.WriteLine(licznik);
```

Odpowiedzi:

- pierwszy przykład wypisze 10,
- drugi przykład wypisze 3,
- trzeci przykład wypisze 12 i 3.

Zapowiedź kolejnej lekcji

Licznik i akumulator bardzo często pojawiają się w zadaniach egzaminacyjnych i prostych algorytmach. W kolejnej lekcji omówimy instrukcje `break` i `continue`, które pozwalają sterować przebiegiem pętli.

Zapamiętaj

- Licznik służy do zliczania.
- Akumulator służy do gromadzenia wyniku.
- Licznik zwykle zwiększamy przez `licznik++`.
- Akumulator zwykle zwiększamy przez `suma += wartość`.

- Licznik i akumulator zwykle zaczynają od 0.
- Miejsce aktualizacji licznika ma znaczenie.
- Średnia wymaga sumy i liczby elementów.

Ćwiczenia

1. Wczytaj 5 liczb i policz, ile z nich jest parzystych.
2. Wczytaj 5 liczb i policz, ile z nich jest dodatnich.
3. Wczytaj 5 liczb i oblicz ich sumę.
4. Wczytaj 4 oceny i oblicz średnią.
5. Wczytaj 6 liczb i oblicz sumę tylko liczb dodatnich.
6. Wczytaj 6 liczb i policz liczby ujemne.
7. Wczytaj 5 liczb i policz liczby podzielne przez 3.
8. Popraw program, w którym licznik jest zwiększany w złym miejscu.
9. Wskaż, która zmienna jest licznikiem, a która akumulatorem.
10. Napisz własny przykład z licznikiem i akumulatorem.

32. Pętla foreach

04-petle/09-foreach.md

Pętla foreach

Cel lekcji

Nauczysz się używać pętli foreach, gdy chcesz wygodnie przejść po wartościach tablicy i nie potrzebujesz indeksu elementu.

Krótkie przypomnienie for po tablicy

Pętla for używa indeksu.

Indeks pozwala odwołać się do konkretnego elementu tablicy.

Czasem indeks jest potrzebny, ale czasem chcemy tylko odczytać wszystkie wartości.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 4, 7, 2, 9 };

        for (int i = 0; i < liczby.Length; i++)
        {
            Console.WriteLine(liczby[i]);
        }
    }
}
```

W takim kodzie `i` jest indeksem, a `liczby[i]` jest aktualną wartością z tablicy.

Składnia foreach

Schemat pętli foreach wygląda tak:

Kod: csharp

```
foreach (typ element in tablica)
{
    // instrukcje wykonywane dla każdego elementu
}
```

foreach przechodzi po kolejnych elementach tablicy.

W każdej iteracji zmienna `element` przechowuje aktualną wartość.

Nie używamy indeksu `i` i nie musimy pisać warunku `i < tablica.Length`.

Pierwszy przykład foreach

Kod: csharp

```
using System;

class Program
{
```

```

static void Main()
{
    int[] liczby = { 4, 7, 2, 9 };

    foreach (int liczba in liczby)
    {
        Console.WriteLine(liczba);
    }
}

```

W tym przykładzie:

- `liczby` to tablica,
- `liczba` to aktualny element tablicy,
- pętla wykona się raz dla każdego elementu,
- kod wypisze 4, 7, 2, 9.

Nazwa zmiennej w foreach

Zmienna w `foreach` powinna mieć nazwę pojedynczą, a tablica zwykle ma nazwę mnogą.

Dobre przykłady:

Kod: csharp

```

foreach (int liczba in liczby)
foreach (string imie in imiona)
foreach (double temperatura in temperatury)
foreach (int ocena in oceny)

```

`liczby` oznacza zbiór wielu liczb.

`liczba` oznacza jedną aktualną liczbę.

Taka konwencja poprawia czytelność kodu.

foreach z tablicą tekstów

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        string[] imiona = { "Anna", "Jan", "Ewa" };

        foreach (string imie in imiona)
        {
            Console.WriteLine($"Uczeń: {imie}");
        }
    }
}

```

Pętla przechodzi po kolejnych tekstach z tablicy `imiona`.

W każdym obiegu zmienna `imie` przechowuje jedno imię.

Sumowanie za pomocą foreach

Kod: csharp

```

using System;

```

```

class Program
{
    static void Main()
    {
        int[] liczby = { 4, 7, 2, 9 };

        int suma = 0;

        foreach (int liczba in liczby)
        {
            suma += liczba;
        }

        Console.WriteLine($"Suma: {suma}");
    }
}

```

W tym przykładzie:

- suma jest akumulatorem,
- liczba to aktualny element tablicy,
- suma += liczba dodaje aktualny element do sumy.

Średnia za pomocą foreach

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[] oceny = { 5, 4, 3, 5 };

        int suma = 0;

        foreach (int ocena in oceny)
        {
            suma += ocena;
        }

        double srednia = (double)suma / oceny.Length;

        Console.WriteLine($"Średnia: {srednia:F2}");
    }
}

```

Do sumowania używamy foreach.

Do liczby elementów nadajemy używamy oceny.Length.

Przy średniej z liczb typu int używamy (double), aby otrzymać dzielenie rzeczywiste.

Zliczanie elementów tablicy spełniających warunek

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 4, -2, 0, 7, -5, 3 };
    }
}

```

```

        int liczbaDodatnich = 0;

        foreach (int liczba in liczby)
        {
            if (liczba > 0)
            {
                liczbaDodatnich++;
            }
        }

        Console.WriteLine($"Liczba dodatnich: {liczbaDodatnich}");
    }
}

```

foreach dobrze pasuje, gdy sprawdzamy wartości tablicy.

Nie potrzebujemy indeksu.

Warunek dotyczy aktualnej wartości liczba.

Kiedy for, a kiedy foreach

Sytuacja	Lepszy wybór	
---	---	
Potrzebujesz indeksu	`for`	
Chcesz zmienić element tablicy po indeksie	`for`	
Chcesz wypisać wszystkie wartości	`foreach`	
Chcesz policzyć sumę wartości	`foreach` albo `for`	
Chcesz pokazać numer elementu użytkownikowi	`for`	
Chcesz używać `i + 1`	`for`	

foreach jest czytelny, gdy interesują nas same wartości.

for jest potrzebny, gdy ważny jest indeks.

W tym kursie uczymy obu zapisów, bo oba są przydatne.

Przykład, w którym lepszy jest for

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[] punkty = { 10, 15, 20 };

        for (int i = 0; i < punkty.Length; i++)
        {
            Console.WriteLine($"Indeks {i}: {punkty[i]}");
        }
    }
}

```

Tutaj potrzebujemy indeksu, dlatego for jest lepszy niż foreach.

Przykład zmiany elementów tablicy

Kod: csharp

```

using System;

class Program
{

```



```

static void Main()
{
    int[] punkty = { 10, 15, 20 };

    for (int i = 0; i < punkty.Length; i++)
    {
        punkty[i] += 1;
    }

    foreach (int punkt in punkty)
    {
        Console.WriteLine(punkt);
    }
}

```

Do zmiany elementów tablicy używamy indeksu, dlatego modyfikacja jest wykonana pętlą for.

Potem wartości można wypisać pętlą foreach.

Wyjaśnienie

Wyobraź sobie, że masz pudełko z kartkami.

Na każdej kartce jest jedna liczba.

Pętla for mówi: weź kartkę numer 0, potem numer 1, potem numer 2.

Pętla foreach mówi: weź po kolei każdą kartkę z pudełka.

Jeśli nie obchodzi cię numer kartki, tylko jej zawartość, foreach jest prostszy.

Jeśli potrzebujesz numeru kartki, użyj for.

Typowe błędy

Błąd 1: oczekiwanie indeksu w foreach

foreach daje aktualną wartość.

Nie daje automatycznie indeksu.

Jeśli potrzebujesz indeksu, użyj for.

Błąd 2: mylenie nazwy tablicy i elementu

Nieczytelnie:

Kod: csharp

```
foreach (int liczby in liczby)
```

Taka nazwa jest myląca.

Lepiej używać liczba dla pojedynczego elementu i liczby dla tablicy.

Poprawnie:

Kod: csharp

```
foreach (int liczba in liczby)
```

Błąd 3: próba użycia Length nie tam, gdzie trzeba

W foreach nie piszemy warunku `i < tablica.Length`.

foreach sam przechodzi po wszystkich elementach.

Błąd 4: używanie foreach, gdy trzeba zmieniać elementy po indeksie

Jeśli chcesz zmienić tablica[i], zwykle użyj for.

foreach jest najlepszy do odczytu wartości.

Zgadnij, co wypisze program

Przykład z odpowiedzią:

Kod: csharp

```
int[] liczby = { 2, 4, 6 };

foreach (int liczba in liczby)
{
    Console.WriteLine(liczba);
}
```

Odpowiedź: program wypisze 2, 4, 6.

Spróbuj najpierw samodzielnie przewidzieć wyniki kolejnych przykładów.

Kod: csharp

```
string[] imiona = { "Ala", "Ola" };

foreach (string imie in imiona)
{
    Console.WriteLine($"Cześć, {imie}");
}
```

Kod: csharp

```
int[] liczby = { 1, 2, 3, 4 };
int suma = 0;

foreach (int liczba in liczby)
{
    suma += liczba;
}

Console.WriteLine(suma);
```

Kod: csharp

```
int[] liczby = { -1, 5, 0, 7 };
int licznik = 0;

foreach (int liczba in liczby)
{
    if (liczba > 0)
    {
        licznik++;
    }
}

Console.WriteLine(licznik);
```

Odpowiedzi:

- pierwszy przykład wypisze Cześć, Ala oraz Cześć, Ola,
- drugi przykład wypisze 10,
- trzeci przykład wypisze 2.

Zapowiedź kolejnej lekcji

Pętla foreach upraszcza przechodzenie po wartościach tablicy.

W kolejnej lekcji można omówić break i continue, czyli instrukcje sterujące przebiegiem pętli.

Zapamiętaj

- foreach przechodzi po kolejnych wartościach tablicy.
- foreach nie wymaga indeksu.
- Zmienna w foreach przechowuje aktualny element.
- for jest lepszy, gdy potrzebujesz indeksu.
- foreach jest czytelny, gdy chcesz tylko odczytać wartości.
- Do zmiany elementów tablicy zwykle używamy for.
- Dobra nazwa to na przykład foreach (int liczba in liczby).

Ćwiczenia

1. Wypisz wszystkie elementy tablicy `int[]` za pomocą foreach.
2. Wypisz wszystkie imiona z tablicy `string[]`.
3. Oblicz sumę elementów tablicy za pomocą foreach.
4. Oblicz średnią ocen z tablicy, używając foreach.
5. Policz liczby dodatnie w tablicy za pomocą foreach.
6. Policz liczby parzyste w tablicy za pomocą foreach.
7. Wskaż, kiedy lepszy będzie for, a kiedy foreach.
8. Popraw nieczytelny zapis foreach (int liczby in liczby).
9. Napisz przykład, w którym potrzebujesz indeksu i dlatego używasz for.
10. Napisz własny przykład użycia foreach.

33. break i continue

04-petle/10-break-continue.md

Instrukcje break i continue

Cel lekcji

Nauczysz się używać instrukcji break i continue do sterowania przebiegiem pętli.

Krótkie wprowadzenie

Zwykle pętla wykonuje się tak długo, jak pozwala jej warunek.

Czasami chcemy zakończyć pętlę wcześniej.

Czasami chcemy pominąć tylko jeden obieg pętli.

Do tego służą instrukcje break i continue.

Instrukcja break

break natychmiast kończy najbliższą pętlę.

Program przechodzi do instrukcji po pętli.

break często stosuje się, gdy znaleziono szukaną wartość albo dalsze sprawdzanie nie ma sensu.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 10; i++)
        {
            if (i == 5)
            {
                break;
            }

            Console.WriteLine(i);
        }

        Console.WriteLine("Koniec pętli.");
    }
}
```

Program wypisze 1, 2, 3, 4.

Gdy i ma wartość 5, wykona się break i pętla zostanie przerwana.

Po pętli wykona się instrukcja:

Kod: csharp

```
Console.WriteLine("Koniec pętli.");
```

Instrukcja continue

continue pomija resztę bieżącego obiegu pętli.

Pętla nie kończy się całkowicie. Program przechodzi do następnego obiegu.

`continue` stosuje się, gdy chcemy pominąć wybrane wartości.

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            if (i == 3)
            {
                continue;
            }

            Console.WriteLine(i);
        }
    }
}
```

Program wypisze 1, 2, 4, 5.

Dla `i == 3` wykonuje się `continue`, więc `Console.WriteLine(i)` zostaje wtedy pominięte.

Pętla działa dalej.

Różnica między `break` i `continue`

Instrukcja	Co robi	Czy pętla działa dalej?
<code>break</code>	kończy najbliższą pętlę	nie
<code>continue</code>	pomija bieżący obieg	tak

`break` oznacza: zakończ pętlę.

`continue` oznacza: pomini ten przypadek i przejdź dalej.

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart TD
    A([Start]) --> B[Obieg pętli]
    B --> C[Instrukcja sterująca]
    C -- "break" --> D[wyjście z pętli]
    C -- "continue" --> E[następny obieg]
    D --> F[Kod po pętli]
    E --> B
    F --> G([Koniec])
```

Diagram pokazuje różnicę między zakończeniem całej pętli a przejściem do następnego obiegu.

Wyjaśnienie

Wyobraź sobie, że sprawdzasz stos kartek z liczbami.

break: szukasz pierwszej kartki z liczbą 0. Gdy ją znajdziesz, przestajesz przeglądać kolejne kartki. To jest break - kończysz całą pętlę.

continue: przeglądasz wszystkie kartki. Jeśli kartka ma liczbę ujemną, odkładasz ją i przechodzisz do następnej. To jest continue - pomijasz jeden przypadek, ale pracujesz dalej.

break przy szukaniu elementu w tablicy

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 4, 7, 2, 9, 5 };

        for (int i = 0; i < liczby.Length; i++)
        {
            if (liczby[i] == 9)
            {
                Console.WriteLine($"Znaleziono liczbę 9 pod indeksem {i}.");
                break;
            }

            Console.WriteLine($"Sprawdzam indeks {i}.");
        }

        Console.WriteLine("Po pętli.");
    }
}
```

Pętla sprawdza kolejne elementy tablicy.

Gdy znajdzie wartość 9, wypisuje komunikat i kończy pętlę.

Elementy po znalezionej wartości nie są już sprawdzane. To ma sens, gdy interesuje nas pierwsze wystąpienie.

break i zmienna bool: czy znaleziono

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 4, 7, 2, 9, 5 };
        int szukana = 9;
        bool znaleziono = false;

        for (int i = 0; i < liczby.Length; i++)
        {
            if (liczby[i] == szukana)
            {
                znaleziono = true;
                break;
            }
        }
    }
}
```

```

    }

    if (znaleziono)
    {
        Console.WriteLine("Znaleziono szukaną liczbę.");
    }
    else
    {
        Console.WriteLine("Nie znaleziono szukanej liczby.");
    }
}
}

```

Zmienna `znaleziono` zapamiętuje wynik wyszukiwania.

Zaczyna od `false`.

Jeśli znajdziemy szukaną wartość, ustawiamy `true` i przerywamy pętlę.

Po pętli można wypisać odpowiedni komunikat.

continue przy pomijaniu elementów tablicy

Kod: `csharp`

```

using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 4, -2, 0, 7, -5, 3 };

        foreach (int liczba in liczby)
        {
            if (liczba < 0)
            {
                continue;
            }

            Console.WriteLine(liczba);
        }
    }
}

```

Liczby ujemne są pomijane.

`continue` sprawia, że `Console.WriteLine(liczba)` nie wykona się dla liczb ujemnych.

Pętla przechodzi dalej do kolejnych elementów.

Wypisane zostaną tylko 4, 0, 7, 3.

continue przy sumowaniu tylko wybranych wartości

Kod: `csharp`

```

using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 4, -2, 0, 7, -5, 3 };

        int suma = 0;

        foreach (int liczba in liczby)
        {
            if (liczba < 0)
            {
                continue;
            }

            suma += liczba;
        }

        Console.WriteLine(suma);
    }
}

```

```

        {
            if (liczba < 0)
            {
                continue;
            }

            suma += liczba;
        }

        Console.WriteLine($"Suma liczb nieujemnych: {suma}");
    }
}

```

Liczby ujemne są pomijane.

`suma += liczba` wykonuje się tylko dla liczb nieujemnych.

Wynik obejmuje liczby dodatnie i zero.

break w pętli while

Kod: **csharp**

```

using System;

class Program
{
    static void Main()
    {
        while (true)
        {
            Console.WriteLine("Podaj liczbę. Wpisz 0, aby zakończyć:");
            int liczba = int.Parse(Console.ReadLine());

            if (liczba == 0)
            {
                break;
            }

            Console.WriteLine($"Podano: {liczba}");
        }

        Console.WriteLine("Program zakończony.");
    }
}

```

`while (true)` tworzy pętlę działającą bez ograniczenia.

`break` kończy ją po wpisaniu 0.

To przykład dydaktyczny. Taka pętla musi mieć jasny sposób zakończenia.

continue w pętli while: ostrożnie

Przykład błędu:

Kod: **csharp**

```

int i = 1;

while (i <= 5)
{
    if (i == 3)
    {
        continue;
    }

    Console.WriteLine(i);
}

```



```
    i++;  
}
```

Gdy `i` ma wartość 3, wykonuje się `continue`.

Instrukcja `i++` zostaje pominięta, więc `i` nadal ma wartość 3.

Powstaje pętla nieskończona.

Poprawny wariant:

Kod: csharp

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int i = 1;  
  
        while (i <= 5)  
        {  
            if (i == 3)  
            {  
                i++;  
                continue;  
            }  
  
            Console.WriteLine(i);  
            i++;  
        }  
    }  
}
```

Przed `continue` zwiększamy licznik. Dzięki temu pętla może dojść do końca.

Zgadnij, co wypisze program

Przykład z odpowiedzią:

Kod: csharp

```
for (int i = 1; i <= 5; i++)  
{  
    if (i == 3)  
    {  
        break;  
    }  
  
    Console.WriteLine(i);  
}
```

Odpowiedź: program wypisze 1, 2.

Spróbuj najpierw samodzielnie przewidzieć wyniki kolejnych przykładów.

Kod: csharp

```
for (int i = 1; i <= 5; i++)  
{  
    if (i == 3)  
    {  
        continue;  
    }  
  
    Console.WriteLine(i);  
}
```

Kod: csharp

```
int[] liczby = { 2, 4, 6, 8 };

foreach (int liczba in liczby)
{
    if (liczba > 5)
    {
        break;
    }

    Console.WriteLine(liczba);
}
```

Kod: csharp

```
int[] liczby = { 1, -2, 3, -4 };

foreach (int liczba in liczby)
{
    if (liczba < 0)
    {
        continue;
    }

    Console.WriteLine(liczba);
}
```

Odpowiedzi:

- pierwszy przykład wypisze 1, 2, 4, 5,
- drugi przykład wypisze 2, 4,
- trzeci przykład wypisze 1, 3.

Typowe błędy

Błąd 1: mylenie break i continue

break kończy całą najbliższą pętlę.

continue kończy tylko bieżący obieg pętli.

Błąd 2: oczekiwanie, że break zakończy cały program

break kończy pętlę albo case w switch.

Nie kończy całego programu. Po pętli wykonują się dalsze instrukcje.

Błąd 3: continue przed zmianą licznika w while

W pętli while trzeba pilnować, aby continue nie pominęło zmiany zmiennej sterującej.

Inaczej może powstać pętla nieskończona.

Błąd 4: nadużywanie break i continue

break i continue są przydatne, ale zbyt częste używanie może utrudnić czytanie programu.

Stosujemy je wtedy, gdy naprawdę upraszczają kod.

Błąd 5: break przy zliczaniu wszystkich elementów

Jeśli chcemy policzyć wszystkie pasujące elementy tablicy, zwykle nie używamy break.

break kończy pętlę po pierwszym znalezieniu.

Do zliczania wszystkich elementów pętla powinna przejść przez całą tablicę.

Kiedy używać break, a kiedy continue

Użyj break, gdy:

- szukasz pierwszego wystąpienia,
- dalsze sprawdzanie nie ma sensu,
- użytkownik wybrał zakończenie,
- chcesz wyjść z pętli.

Użyj continue, gdy:

- chcesz pominąć jeden przypadek,
- nie chcesz wykonywać dalszych instrukcji dla aktualnego elementu,
- pętla ma działać dalej dla kolejnych elementów.

Zapowiedź podsumowania działu

Po poznaniu pętli, tablic, foreach, licznika, akumulatora oraz instrukcji break i continue można rozwiązywać wiele typowych zadań programistycznych.

Następny krok to podsumowanie działu o pętlach i tablicach.

Zapamiętaj

- break kończy najbliższą pętlę.
- continue pomija resztę bieżącego obiegu.
- Po break program przechodzi za pętlę.
- Po continue pętla przechodzi do kolejnego obiegu.
- break dobrze pasuje do szukania pierwszego pasującego elementu.
- continue dobrze pasuje do pomijania wybranych elementów.
- W pętli while trzeba uważać, aby continue nie pominęło zmiany licznika.
- Do zliczania wszystkich pasujących elementów zwykle nie używamy break.

Ćwiczenia

1. Wypisz liczby od 1 do 10, ale przerwij pętlę po liczbie 5.
2. Wypisz liczby od 1 do 10 z pominięciem liczby 4.
3. Wypisz elementy tablicy do momentu napotkania liczby ujemnej.
4. Sprawdź, czy w tablicy znajduje się liczba 0. Po znalezieniu przerwij pętlę.
5. Wypisz tylko liczby dodatnie z tablicy, używając continue.
6. Oblicz sumę liczb nieujemnych z tablicy, używając continue.
7. Popraw pętlę while, w której continue powoduje pętlę nieskończoną.
8. Wyjaśnij różnicę między break i continue.
9. Wskaż, kiedy break jest błędem przy zliczaniu elementów.
10. Napisz własny przykład użycia break albo continue.

34. Wprowadzenie

05-metody/README.md

05 - Metody

W tym dziale uczeń poznaje metody w C#, czyli sposób porządkowania programu i wydzielania powtarzalnych fragmentów kodu.

Planowane lekcje:

- Po co są metody (01-po-co-sa-metody.md)
- Pierwsza metoda void (02-pierwsza-metoda-void.md)
- Parametry metody (03-parametry-metody.md)
- Metoda zwracająca wartość (04-metoda-zwracajaca-wartosc.md)
- void a return (05-void-a-return.md)
- void a typ zwracany
- Metody i tablice (06-metody-i-tablice.md)
- Ćwiczenia (07-cwiczenia.md)

35. Po co są metody

05-metody/01-po-co-sa-metody.md

Po co są metody

Problem: cały kod w Main

Na początku kursu większość programów pisaliśmy w metodzie Main.

Dla krótkich przykładów jest to wygodne. W większych programach szybko robi się jednak bałagan:

- kod jest długi,
- trudno znaleźć ważne fragmenty,
- te same instrukcje mogą powtarzać się wiele razy,
- poprawienie programu wymaga zmiany w kilku miejscach.

Przykład bez metody:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("=== PROGRAM ===");
        Console.WriteLine("Witaj w programie!");

        Console.WriteLine("-----");
        Console.WriteLine("Opcja 1 - Start");
        Console.WriteLine("Opcja 2 - Pomoc");
        Console.WriteLine("Opcja 0 - Koniec");
        Console.WriteLine("-----");

        Console.WriteLine("Dziękujemy za użycie programu.");
        Console.WriteLine("=== PROGRAM ===");
    }
}
```

Ten program jest jeszcze krótki, ale już widać powtarzające się fragmenty. W większym programie taki zapis byłby trudniejszy do czytania.

Metoda jako nazwany fragment kodu

Metoda to fragment kodu, który ma nazwę i można go wywołać.

Dzięki metodzie możemy nadać nazwę pewnej czynności, na przykład PokazNaglowek.

Kod: csharp

```
using System;

class Program
{
    static void PokazNaglowek()
    {
        Console.WriteLine("=== PROGRAM ===");
    }

    static void Main()
    {

```

```

        PokazNaglowek();

        Console.WriteLine("Witaj w programie!");
        Console.WriteLine("Opcja 1 - Start");
        Console.WriteLine("Opcja 2 - Pomoc");
        Console.WriteLine("Opcja 0 - Koniec");
    }
}

```

W tym przykładzie:

- PokazNaglowek to nazwa metody,
- kod metody znajduje się w nawiasach klamrowych,
- PokazNaglowek(); w metodzie Main oznacza wywołanie metody,
- po wykonaniu metody program wraca do miejsca, z którego metoda została wywołana.

Jak działa wywołanie metody

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B([Main])
    C["Wywołanie metody"]
    D["Wykonanie metody"]
    E["Powrót do Main"]
    F([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F

```

Program zaczyna działanie w metodzie Main. Gdy napotka wywołanie metody, przechodzi do tej metody, wykonuje jej instrukcje, a potem wraca do Main.

Metoda może być wywołana wiele razy

Metodę można wywołać więcej niż jeden raz.

To przydatne, gdy ten sam fragment kodu ma pojawić się w kilku miejscach programu.

Kod: csharp

```

using System;

class Program
{
    static void PokazSeparator()
    {
        Console.WriteLine("-----");
    }

    static void Main()
    {
        PokazSeparator();
        Console.WriteLine("Menu programu");
        PokazSeparator();
    }
}

```

```

        Console.WriteLine("1 - Start");
        Console.WriteLine("2 - Pomoc");
        Console.WriteLine("0 - Koniec");

        PokazSeparator();
    }
}

```

W tym przykładzie metoda `PokazSeparator` jest zapisana tylko raz, ale wywołujemy ją kilka razy.

Jeśli chcemy zmienić wygląd separatora, poprawiamy tylko jedną metodę.

Dlaczego to jest przydatne

Metody pomagają pisać czytelniejsze programy.

Najważniejsze korzyści:

- mniej powtarzania kodu,
- lepszy porządek,
- łatwiejsze poprawianie programu,
- łatwiejsze czytanie programu,
- przygotowanie do większych zadań.

Ważne pojęcia

Metoda to nazwany fragment kodu, który można wykonać przez wywołanie.

Wywołanie metody to użycie nazwy metody z nawiasami, na przykład:

Kod: csharp

```
PokazSeparator();
```

Ciało metody to instrukcje zapisane w nawiasach klamrowych metody.

`void` oznacza, że metoda wykonuje instrukcje, ale nie zwraca wyniku.

Na tym etapie nie omawiamy jeszcze parametrów ani `return`. Pojawią się w kolejnych lekcjach.

Ćwiczenia

1. Napisz metodę `PokazPowitanie`.
2. Napisz metodę `PokazMenu`.
3. Napisz metodę `PokazSeparator`.
4. Wywołaj jedną metodę kilka razy.
5. Popraw program, w którym ten sam fragment tekstu pojawia się kilka razy, przenosząc go do metody.

Podsumowanie

Metody pozwalają dzielić program na mniejsze, nazwane fragmenty.

Dzięki nim kod jest bardziej uporządkowany, mniej się powtarza i łatwiej go poprawiać.

W tej lekcji używaliśmy prostych metod `void`, które nie mają parametrów i nie zwracają wyniku.

36. Pierwsza metoda void

05-metody/02-pierwsza-metoda-void.md

Pierwsza metoda void

Co oznacza void

void oznacza, że metoda wykonuje instrukcje, ale nie oddaje wyniku do miejsca wywołania.

Taka metoda może na przykład wypisać tekst w konsoli, pokazać menu albo wyświetlić separator.

Metody zwracające wartość poznamy później. Na razie skupiamy się na prostych metodach void.

Składnia metody void

Ogólny schemat metody void wygląda tak:

Kod: csharp

```
static void NazwaMetody()
{
    // instrukcje
}
```

Najważniejsze elementy:

- `static` - na tym etapie używamy `static`, ponieważ wywołujemy metodę z `Main`, które też jest `static`,
- `void` - metoda wykonuje instrukcje, ale nie zwraca wyniku,
- `NazwaMetody` - nazwa metody,
- `()` - nawiasy okrągłe po nazwie metody,
- `{ }` - ciało metody, czyli instrukcje wykonywane przez metodę.

Nie rozwijamy jeszcze tematu obiektowości. W tym dziale chodzi o praktyczne użycie prostych metod.

Pierwszy przykład

Kod: csharp

```
using System;

class Program
{
    static void PokazPowitanie()
    {
        Console.WriteLine("Witaj w programie!");
    }

    static void Main()
    {
        PokazPowitanie();
    }
}
```

Sama definicja metody nie uruchamia jej.

Metoda działa dopiero wtedy, gdy zostanie wywołana:

Kod: csharp

```
PokazPowitanie();
```


Program zaczyna działanie w Main. Gdy napotka PokazPowitanie();, przechodzi do metody PokazPowitanie, wykonuje jej instrukcje, a potem wraca do Main.

Jak program przechodzi przez metodę

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart LR
  A([Start])
  B["Main"]
  C["Wywołanie<br/>PokazPowitanie"]
  D["Console.WriteLine"]
  E["Powrót<br/>do Main"]
  F([Koniec])

  A --> B
  B --> C
  C --> D
  D --> E
  E --> F
```

Diagram pokazuje, że metoda jest osobnym fragmentem kodu, ale program po jej wykonaniu wraca do miejsca wywołania.

Metoda z kilkoma instrukcjami

Jedna metoda może zawierać kilka instrukcji.

Kod: csharp

```
using System;

class Program
{
    static void PokazMenu()
    {
        Console.WriteLine("1. Dodaj");
        Console.WriteLine("2. Usuń");
        Console.WriteLine("3. Zakończ");
    }

    static void Main()
    {
        Console.WriteLine("Menu programu:");
        PokazMenu();
    }
}
```

Metoda PokazMenu wypisuje trzy linie tekstu. W Main wystarczy jedno wywołanie:

Kod: csharp

```
PokazMenu();
```

Kilka wywołań tej samej metody

Tę samą metodę można wywołać wiele razy.

Kod: csharp

```
using System;
```

```

class Program
{
    static void PokazSeparator()
    {
        Console.WriteLine("-----");
    }

    static void Main()
    {
        PokazSeparator();
        Console.WriteLine("Początek programu");
        PokazSeparator();
        Console.WriteLine("Główna część programu");
        PokazSeparator();
        Console.WriteLine("Koniec programu");
        PokazSeparator();
    }
}

```

Metoda PokazSeparator jest zapisana tylko raz, ale można jej używać w wielu miejscach programu.

Nazwy metod

Nazwa metody powinna mówić, co metoda robi.

W C# dla nazw metod zwykle stosuje się PascalCase. Oznacza to, że każde słowo zaczynamy wielką literą.

Dobre nazwy:

- PokazMenu
- ObliczSume
- WczytajLiczbe
- PokazSeparator

Słabe nazwy:

- X
- Test
- Metoda1

Na razie nie omawiamy metod z parametrami. Pojawią się w kolejnej lekcji.

Najczęstsze błędy

- Zdefiniowanie metody wewnątrz Main.
- Brak nawiasów przy wywołaniu metody.
- Literówka w nazwie metody.
- Zapomnienie średnika po wywołaniu metody.
- Oczekiwanie, że metoda wykona się sama bez wywołania.

Przykład wywołania metody musi zawierać nawiasy i średnik:

Kod: csharp

```
PokazPowitanie();
```

Ćwiczenia

1. Napisz metodę PokazPowitanie, która wypisuje krótkie powitanie.

2. Napisz metodę PokazMenu z trzema opcjami.
3. Napisz metodę PokazSeparator i wywołaj ją kilka razy.
4. Napisz metodę PokazAutora, która wypisuje imię autora programu.
5. Popraw program, w którym kilka razy powtarza się ten sam tekst, wydzielając ten tekst do metody void.

Podsumowanie

Metoda void wykonuje instrukcje, ale nie zwraca wyniku.

Metoda musi zostać wywołana, aby jej kod się wykonał.

Metody pomagają porządkować program i ograniczać powtarzanie kodu.

W kolejnych lekcjach pojawią się parametry i return.

37. Parametry metody

05-metody/03-parametry-metody.md

Parametry metody

Po co są parametry

Metoda bez parametrów zawsze działa tak samo.

Przykład:

Kod: csharp

```
static void PokazPowitanie()
{
    Console.WriteLine("Cześć!");
}
```

Taka metoda za każdym razem wypisze ten sam tekst.

Metoda z parametrem może wykonać podobne działanie dla różnych danych.

Kod: csharp

```
static void PokazPowitanie(string imie)
{
    Console.WriteLine($"Cześć, {imie}!");
}
```

Teraz metoda może przywitać różne osoby, zależnie od tego, jaką wartość prześlemy przy wywołaniu.

Parametr i argument

Parametr jest w definicji metody.

Argument jest wartością przekazaną podczas wywołania metody.

Przykład definicji metody:

Kod: csharp

```
static void PokazPowitanie(string imie)
```

Tutaj `imie` jest parametrem.

Przykład wywołania metody:

Kod: csharp

```
PokazPowitanie("Ała");
```

Tutaj `"Ała"` jest argumentem.

Można powiedzieć prosto: argument trafia do parametru.

Jak dane trafiają do metody

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
```

```

B["Main"]
C["Argument<br/>\\"Ala\\""]
D["Parametr<br/>imie"]
E["Console.WriteLine"]
F([Koniec])

A --> B
B --> C
C --> D
D --> E
E --> F

```

Wartość "Ala" jest przekazywana z miejsca wywołania metody do parametru imie.

Potem metoda może użyć parametru imie w swoich instrukcjach.

Metoda z jednym parametrem

Kod: csharp

```

using System;

class Program
{
    static void PokazPowitanie(string imie)
    {
        Console.WriteLine($"Cześć, {imie}!");
    }

    static void Main()
    {
        PokazPowitanie("Ala");
        PokazPowitanie("Bartek");
        PokazPowitanie("Celina");
    }
}

```

Ta sama metoda działa dla różnych wartości.

Za pierwszym razem argumentem jest "Ala", za drugim "Bartek", a za trzecim "Celina".

Typ parametru

Parametr ma typ, tak samo jak zmienna.

Kod: csharp

```

static void PokazWiek(int wiek)
{
    Console.WriteLine($"Masz {wiek} lat.");
}

```

Parametr wiek ma typ int, więc do metody trzeba przekazać liczbę całkowitą.

Przykład użycia:

Kod: csharp

```

PokazWiek(16);

```

Metoda z kilkoma parametrami

Metoda może mieć więcej niż jeden parametr.

Kod: csharp

```

using System;

```

```

class Program
{
    static void PokazDane(string imie, int wiek)
    {
        Console.WriteLine($"Imię: {imie}");
        Console.WriteLine($"Wiek: {wiek}");
    }

    static void Main()
    {
        PokazDane("Ola", 16);
    }
}

```

Kolejność argumentów musi odpowiadać kolejności parametrów.

W wywołaniu:

Kod: csharp

```
PokazDane("Ola", 16);
```

argument "Ola" trafia do parametru imie, a argument 16 trafia do parametru wiek.

Kilka wywołań z różnymi argumentami

Jedną metodę można wywołać wiele razy z różnymi argumentami.

Kod: csharp

```

using System;

class Program
{
    static void PokazOcene(string przedmiot, int ocena)
    {
        Console.WriteLine($"{przedmiot}: {ocena}");
    }

    static void Main()
    {
        PokazOcene("Matematyka", 5);
        PokazOcene("Informatyka", 6);
        PokazOcene("Język polski", 4);
    }
}

```

Metoda PokazOcene ma dwa parametry. Dzięki temu może wypisywać podobny komunikat dla różnych przedmiotów i ocen.

Najczęstsze błędy

- Brak argumentu przy wywołaniu metody.
- Zły typ argumentu.
- Pomyłona kolejność argumentów.
- Literówka w nazwie parametru.
- Mylenie parametru z argumentem.

Przykład błędu:

Kod: csharp

```
PokazWiek("szesnaście");
```

Jeśli parametr wiek ma typ int, to argument powinien być liczbą całkowitą, na przykład:

Kod: csharp

```
PokazWiek(16);
```

Ćwiczenia

1. Napisz metodę `PokazImie(string imie)`, która wypisuje imię.
2. Napisz metodę `PokazWiek(int wiek)`, która wypisuje wiek.
3. Napisz metodę `PokazUcznia(string imie, int wiek)`.
4. Wywołaj jedną metodę trzy razy z różnymi argumentami.
5. Napisz metodę `PokazOcene(string przedmiot, int ocena)`.
6. Popraw program, w którym kilka razy wypisywany jest podobny komunikat, zamieniając go na metodę z parametrem.

Podsumowanie

Parametr znajduje się w definicji metody.

Argument przekazujemy podczas wywołania metody.

Parametr ma typ.

Metoda z parametrami jest bardziej elastyczna, bo może działać dla różnych wartości.

W następnej lekcji pojawią się metody zwracające wartość.

38. Metoda zwracająca wartość

05-metody/04-metoda-zwracajaca-wartosc.md

Metoda zwracająca wartość

Od void do wartości zwracanej

Metoda void wykonuje instrukcje, ale nie zwraca wyniku.

Metoda z typem zwracanym, na przykład int, double albo string, zwraca wartość do miejsca wywołania.

Typ przed nazwą metody informuje, jakiego typu wartość metoda zwróci.

Przykład metody void:

Kod: csharp

```
static void PokazKomunikat()
{
    Console.WriteLine("Witaj!");
}
```

Przykład metody zwracającej wartość:

Kod: csharp

```
static int PodajLiczbe()
{
    return 5;
}
```

W drugim przykładzie metoda oddaje liczbę 5 do miejsca, w którym została wywołana.

Instrukcja return

return kończy działanie metody.

return przekazuje wartość do miejsca wywołania.

Wartość po return musi pasować do typu zwracanego metody.

Ogólny schemat:

Kod: csharp

```
static typ NazwaMetody()
{
    return wartość;
}
```

Jeśli metoda ma typ int, powinna zwrócić liczbę całkowitą. Jeśli metoda ma typ string, powinna zwrócić tekst.

Pierwszy przykład: metoda zwracająca int

Kod: csharp

```
using System;

class Program
{
    static int PodajLiczbe()
    {
```



```

        return 5;
    }

    static void Main()
    {
        int wynik = PodajLiczbe();
        Console.WriteLine(wynik);
    }
}

```

Krok po kroku:

- Main wywołuje metodę PodajLiczbe,
- metoda zwraca 5,
- wartość 5 trafia do zmiennej wynik,
- Console.WriteLine wypisuje wynik.

Jak działa return

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start]) --> B["Main"]
    B --> C["Wywołanie<br/>PodajLiczbe"]
    C --> D["return 5"]
    D --> E["wynik = 5"]
    E --> F["Console.WriteLine"]
    F --> G([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G

```

Diagram pokazuje, że wartość zwrócona przez metodę może zostać zapisana w zmiennej i użyta dalej w programie.

Metoda z parametrami i wartością zwracaną

Metoda może przyjmować parametry i zwracać wynik.

Kod: csharp

```

using System;

class Program
{
    static int Dodaj(int a, int b)
    {
        return a + b;
    }

    static void Main()
    {
        int suma = Dodaj(3, 4);
        Console.WriteLine(suma);
    }
}

```

```
}
```

W tym przykładzie:

- argumenty 3 i 4 trafiają do parametrów a i b,
- metoda oblicza $a + b$,
- return zwraca wynik,
- wynik można zapisać w zmiennej.

Użycie wyniku bez zmiennej

Wynik metody można przekazać bezpośrednio do `Console.WriteLine`.

Kod: csharp

```
using System;

class Program
{
    static int Dodaj(int a, int b)
    {
        return a + b;
    }

    static void Main()
    {
        Console.WriteLine(Dodaj(10, 20));
    }
}
```

Najpierw metoda `Dodaj(10, 20)` zwróci wynik, a potem `Console.WriteLine` wypisze go na ekranie.

Metoda zwracająca double

Kod: csharp

```
using System;

class Program
{
    static double ObliczSrednia(int a, int b)
    {
        return (a + b) / 2.0;
    }

    static void Main()
    {
        double srednia = ObliczSrednia(4, 5);
        Console.WriteLine($"Średnia: {srednia:F2}");
    }
}
```

Używamy `2.0`, aby otrzymać wynik zmiennoprzecinkowy.

Gdybyśmy użyli `2`, moglibyśmy przypadkowo wykonać dzielenie całkowite.

Metoda zwracająca string

Kod: csharp

```
using System;

class Program
{
    static string PrzygotujPowitanie(string imie)
```

```

    {
        return $"Cześć, {imie}!";
    }

    static void Main()
    {
        string tekst = PrzygotujPowitanie("Ala");
        Console.WriteLine(tekst);
    }
}

```

Metoda PrzygotujPowitanie zwraca tekst typu string.

Wynik metody zapisujemy w zmiennej tekst, a potem wypisujemy go na ekranie.

Najczęstsze błędy

- Brak return w metodzie, która ma zwracać wartość.
- Zwracanie wartości złego typu.
- Mylenie Console.WriteLine z return.
- Oczekiwanie, że Console.WriteLine zwraca wynik.
- Kod po return, który nie zostanie wykonany.

Console.WriteLine wypisuje tekst na ekranie.

return oddaje wartość z metody do miejsca wywołania.

Przykład błędu:

Kod: csharp

```

static int PodajLiczbe()
{
    Console.WriteLine(5);
}

```

Ta metoda ma zwracać int, ale nie ma instrukcji return.

Poprawnie:

Kod: csharp

```

static int PodajLiczbe()
{
    return 5;
}

```

Ćwiczenia

1. Napisz metodę PodajRok, która zwraca aktualny rok jako int.
2. Napisz metodę Dodaj, która przyjmuje dwie liczby int i zwraca ich sumę.
3. Napisz metodę Pomnoz, która przyjmuje dwie liczby int i zwraca ich iloczyn.
4. Napisz metodę ObliczSrednia, która przyjmuje dwie liczby int i zwraca średnią jako double.
5. Napisz metodę PrzygotujPowitanie, która przyjmuje imię i zwraca tekst powitania.
6. Zapisz wynik działania metody w zmiennej i wypisz go na ekranie.
7. Wywołaj metodę zwracającą wartość bezpośrednio wewnątrz Console.WriteLine.

Podsumowanie

Metoda void nie zwraca wartości.

Metoda z typem zwracanym musi zwrócić wartość przez `return`.

Typ zwracanej wartości musi zgadzać się z typem metody.

Wynik metody można zapisać w zmiennej albo użyć od razu.

`return` i `Console.WriteLine` to nie to samo.

39. void a return

05-metody/05-void-a-return.md

void a return

Dwa rodzaje metod

Na tym etapie znamy dwa podstawowe rodzaje metod:

- metody void, które wykonują instrukcje,
- metody zwracające wartość, na przykład int, double, string.

Przykład metody void:

Kod: csharp

```
static void PokazKomunikat()
{
    Console.WriteLine("Witaj!");
}
```

Przykład metody zwracającej wartość:

Kod: csharp

```
static int Dodaj(int a, int b)
{
    return a + b;
}
```

Pierwsza metoda coś robi, czyli wypisuje tekst.

Druga metoda coś oblicza i oddaje wynik do miejsca wywołania.

Metoda void

void oznacza brak zwracanej wartości.

Metoda void może wypisywać tekst, pokazywać menu, wyświetlać separator albo wykonywać serię instrukcji.

Nie można jednak zapisać jej wyniku do zmiennej, bo taka metoda nie oddaje wartości.

Kod: csharp

```
using System;

class Program
{
    static void PokazMenu()
    {
        Console.WriteLine("1. Start");
        Console.WriteLine("2. Pomoc");
        Console.WriteLine("3. Koniec");
    }

    static void Main()
    {
        PokazMenu();
    }
}
```

Metodę void wywołujemy jako osobną instrukcję:

Kod: csharp

```
PokazMenu();
```

Metoda zwracająca wartość

Metoda z typem zwracanym musi zwrócić wartość.

Typ zwracanej wartości jest zapisany przed nazwą metody.

return oddaje wynik do miejsca wywołania.

Kod: csharp

```
using System;

class Program
{
    static int Dodaj(int a, int b)
    {
        return a + b;
    }

    static void Main()
    {
        int wynik = Dodaj(3, 4);
        Console.WriteLine(wynik);
    }
}
```

Metoda Dodaj zwraca liczbę typu int.

Wynik można zapisać w zmiennej:

Kod: csharp

```
int wynik = Dodaj(3, 4);
```

Najważniejsza różnica

Cecha	void	metoda zwracająca wartość
---	---	---
Czy zwraca wartość?	Nie	Tak
Czy używa return z wartością?	Nie	Tak
Czy wynik można zapisać do zmiennej?	Nie	Tak
Przykład użycia	<code>PokazMenu();</code>	<code>int x = Dodaj(2, 3);</code>

Jak wybrać: void czy return

Użyj void, gdy metoda ma coś wykonać, na przykład:

- wypisać menu,
- pokazać separator,
- wyświetlić komunikat,
- wykonać serię instrukcji bez oddawania wyniku.

Użyj metody zwracającej wartość, gdy metoda ma coś obliczyć lub przygotować, na przykład:

- policzyć sumę,
- obliczyć średnią,
- przygotować tekst,
- sprawdzić i zwrócić wynik.

Diagram decyzyjny

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart TD
  A([Start])
  B{"Czy metoda<br/>ma oddać wynik?"}
  C["Użyj typu<br/>i return"]
  D["Użyj void"]
  E([Koniec])

  A --> B
  B -- "tak" --> C
  B -- "nie" --> D
  C --> E
  D --> E
```

Jeśli wynik metody ma być użyty później, wybieramy typ zwracany i return.

Jeśli metoda ma tylko wykonać instrukcje, wystarczy void.

Console.WriteLine to nie return

Console.WriteLine wypisuje tekst na ekranie.

return oddaje wartość z metody do miejsca wywołania.

To są dwie różne rzeczy.

Błędny przykład:

Kod: csharp

```
static int Dodaj(int a, int b)
{
    Console.WriteLine(a + b);
}
```

Ta metoda ma typ int, ale nie zwraca wartości.

Poprawna wersja:

Kod: csharp

```
static int Dodaj(int a, int b)
{
    return a + b;
}
```

Wypisanie wyniku wykonujemy w Main:

Kod: csharp

```
int wynik = Dodaj(3, 4);
Console.WriteLine(wynik);
```

Częsty błąd: próba zapisania wyniku metody void

Błędny przykład:

Kod: csharp

```
int wynik = PokazMenu();
```

To nie działa, bo PokazMenu jest metodą void i nie oddaje wartości.

Poprawne użycie:

Kod: csharp

```
PokazMenu();
```

Metodę void wywołujemy jako instrukcję, ale nie zapisujemy jej wyniku do zmiennej.

Ćwiczenia

1. Napisz metodę void PokazSeparator, która wypisuje linię separatora.
2. Napisz metodę int Dodaj, która zwraca sumę dwóch liczb.
3. Napisz metodę double ObliczSrednia, która zwraca średnią dwóch liczb.
4. Napisz metodę string PrzygotujPowitanie, która zwraca tekst powitania.
5. Wskaż, czy dana metoda powinna być void, czy powinna zwracać wartość: PokazMenu.
6. Wskaż, czy dana metoda powinna być void, czy powinna zwracać wartość: ObliczPoleProstokata.
7. Popraw metodę, która używa Console.WriteLine zamiast return.

Podsumowanie

void oznacza, że metoda nie zwraca wartości.

return oddaje wartość do miejsca wywołania.

Console.WriteLine tylko wypisuje tekst na ekranie.

Jeśli chcemy użyć wyniku w dalszych obliczeniach, metoda powinna zwracać wartość.

Wybór między void i return zależy od celu metody.

40. Metody i tablice

05-metody/06-metody-i-tablice.md

Metody i tablice

Po co przekazywać tablicę do metody

W poprzednich działach operacje na tablicach wykonywaliśmy bezpośrednio w metodzie Main.

W małych programach jest to wystarczające. W większych programach kod w Main może jednak stać się zbyt długi i trudny do czytania.

Do osobnych metod można przenieść na przykład:

- wypisanie elementów tablicy,
- policzenie sumy,
- obliczenie średniej,
- znalezienie minimum,
- znalezienie maksimum.

Dzięki temu metoda Main jest krótsza i lepiej pokazuje główny przebieg programu.

Tablica jako parametr metody

Tablicę można przekazać do metody jako parametr. Dla tablicy liczb całkowitych używamy zapisu `int[]`.

Kod: csharp

```
static void WypiszTablice(int[] liczby)
{
    // instrukcje
}
```

Najważniejsze elementy:

- `int[]` oznacza tablicę liczb całkowitych,
- `liczby` jest parametrem metody,
- do metody można przekazać tablicę utworzoną w Main.

Pierwszy przykład: wypisanie tablicy

Kod: csharp

```
using System;

class Program
{
    static void WypiszTablice(int[] liczby)
    {
        foreach (int liczba in liczby)
        {
            Console.WriteLine(liczba);
        }
    }

    static void Main()
    {
        int[] dane = { 5, 2, 7, 1 };
    }
}
```

```

        WypiszTablice(dane);
    }
}

```

Tablica dane została utworzona w metodzie Main. Następnie została przekazana do metody WypiszTablice.

W metodzie WypiszTablice ta sama tablica jest dostępna przez parametr liczby.

Jak tablica trafia do metody

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Main"]
    C["Tablica dane"]
    D["Parametr int[] liczby"]
    E["Pętla foreach"]
    F["Wypisanie elementów"]
    G([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G

```

Diagram pokazuje, że tablica utworzona w Main może zostać przekazana do metody. W metodzie używamy jej przez nazwę parametru.

Metoda zwracająca sumę elementów tablicy

Metoda może nie tylko wypisywać elementy tablicy, ale także obliczać wynik i zwracać go przez return.

Kod: csharp

```

using System;

class Program
{
    static int ObliczSume(int[] liczby)
    {
        int suma = 0;

        foreach (int liczba in liczby)
        {
            suma += liczba;
        }

        return suma;
    }

    static void Main()
    {
        int[] dane = { 5, 2, 7, 1 };

        int suma = ObliczSume(dane);
        Console.WriteLine(suma);
    }
}

```

```
}
```

W tym przykładzie metoda:

- przyjmuje tablicę jako parametr,
- przechodzi po jej elementach,
- oblicza sumę,
- zwraca wynik przez return.

Metoda zwracająca średnią

Kod: csharp

```
using System;

class Program
{
    static double ObliczSrednia(int[] liczby)
    {
        int suma = 0;

        foreach (int liczba in liczby)
        {
            suma += liczba;
        }

        return (double)suma / liczby.Length;
    }

    static void Main()
    {
        int[] dane = { 5, 2, 7, 1 };

        double srednia = ObliczSrednia(dane);
        Console.WriteLine(srednia);
    }
}
```

`liczby.Length` oznacza liczbę elementów tablicy.

Zapis `(double)suma` sprawia, że dzielenie daje wynik zmiennoprzecinkowy. Dzięki temu średnia może mieć część po przecinku.

Metoda zwraca wynik typu `double`.

Metoda zwracająca minimum

Kod: csharp

```
using System;

class Program
{
    static int ZnajdzMinimum(int[] liczby)
    {
        int minimum = liczby[0];

        foreach (int liczba in liczby)
        {
            if (liczba < minimum)
            {
                minimum = liczba;
            }
        }
    }
}
```

```

        return minimum;
    }

    static void Main()
    {
        int[] dane = { 5, 2, 7, 1 };

        int minimum = ZnajdzMinimum(dane);
        Console.WriteLine(minimum);
    }
}

```

Jako początkowe minimum przyjmujemy pierwszy element tablicy, czyli `liczby[0]`.

Dzięki temu metoda działa także wtedy, gdy wszystkie liczby w tablicy są większe od zera albo mniejsze od zera.

Porządkowanie programu

W większym programie można przygotować kilka metod, a w `Main` zostawić główny przebieg programu.

Kod: csharp

```

using System;

class Program
{
    static void WypiszTablice(int[] liczby)
    {
        foreach (int liczba in liczby)
        {
            Console.WriteLine(liczba);
        }
    }

    static int ObliczSume(int[] liczby)
    {
        int suma = 0;

        foreach (int liczba in liczby)
        {
            suma += liczba;
        }

        return suma;
    }

    static double ObliczSrednia(int[] liczby)
    {
        int suma = ObliczSume(liczby);
        return (double)suma / liczby.Length;
    }

    static int ZnajdzMinimum(int[] liczby)
    {
        int minimum = liczby[0];

        foreach (int liczba in liczby)
        {
            if (liczba < minimum)
            {
                minimum = liczba;
            }
        }

        return minimum;
    }
}

```

```

static void Main()
{
    int[] dane = { 5, 2, 7, 1 };

    WypiszTablice(dane);

    int suma = ObliczSume(dane);
    double srednia = ObliczSrednia(dane);
    int minimum = ZnajdzMinimum(dane);

    Console.WriteLine($"Suma: {suma}");
    Console.WriteLine($"Średnia: {srednia}");
    Console.WriteLine($"Minimum: {minimum}");
}

```

Metoda Main pokazuje, co dzieje się w programie. Szczegóły obliczeń znajdują się w osobnych metodach.

Najczęstsze błędy

- Pomylenie `int` z `int[]`.
- Brak `[]` przy parametrze tablicy.
- Próba użycia zmiennej z Main bez przekazania jej jako argumentu.
- Użycie `liczby.Length` na zwykłej liczbie typu `int`.
- Rozpoczęcie szukania minimum od 0 zamiast od pierwszego elementu tablicy.
- Brak `return` w metodzie, która ma zwracać wynik.

Ćwiczenia

1. Napisz metodę `WypiszTablice(int[] liczby)`, która wypisuje wszystkie elementy tablicy.
2. Napisz metodę `ObliczSume(int[] liczby)`, która zwraca sumę elementów tablicy.
3. Napisz metodę `ObliczSrednia(int[] liczby)`, która zwraca średnią jako `double`.
4. Napisz metodę `ZnajdzMinimum(int[] liczby)`, która zwraca najmniejszy element tablicy.
5. Napisz metodę `ZnajdzMaksimum(int[] liczby)`, która zwraca największy element tablicy.
6. Napisz metodę `PoliczParzyste(int[] liczby)`, która zwraca liczbę elementów parzystych.
7. W Main utwórz tablicę i wywołaj kilka metod analizujących tę samą tablicę.
8. Przepisz program z poprzedniego działu tak, aby obliczenia na tablicy były wykonane w metodach.

Podsumowanie

Tablica może być argumentem metody. Parametr tablicowy zapisujemy na przykład jako `int[] liczby`.

Metoda może wypisać elementy tablicy albo zwrócić wynik obliczeń, na przykład sumę, średnią lub minimum.

Metody porządkują algorytmy na tablicach. Dzięki nim Main może pokazywać główny przebieg programu, a szczegóły można przenieść do osobnych, nazwanych fragmentów kodu.

41. Ćwiczenia

05-metody/07-cwiczenia.md

Ćwiczenia — metody

Jak pracować z ćwiczeniami

Przy każdym ćwiczeniu pracuj spokojnie i krok po kroku:

- najpierw napisz nagłówek metody,
- potem ciało metody,
- potem wywołaj metodę w Main,
- sprawdź wynik dla kilku danych,
- nie pisz całego programu tylko w Main.

Celem tych ćwiczeń jest utrwalenie metod void, parametrów, return oraz metod pracujących na tablicach.

Ćwiczenia podstawowe — metody void

1. Napisz metodę `PokazPowitanie()`, która wypisuje krótkie powitanie.
2. Napisz metodę `PokazSeparator()`, która wypisuje linię złożoną z myślników.
3. Napisz metodę `PokazMenu()`, która wypisuje trzy opcje menu.
4. Napisz metodę `PokazAutora()`, która wypisuje imię autora programu.
5. Napisz program, w którym metody `PokazSeparator()` i `PokazMenu()` są wywołane kilka razy.

Ćwiczenia z parametrami

1. Napisz metodę `PokazImie(string imie)`, która wypisuje imię.
2. Napisz metodę `PokazWiek(int wiek)`, która wypisuje wiek.
3. Napisz metodę `PokazUcznia(string imie, int wiek)`, która wypisuje dane ucznia.
4. Napisz metodę `PokazOcene(string przedmiot, int ocena)`, która wypisuje nazwę przedmiotu i ocenę.
5. Napisz metodę `PokazZakres(int od, int do_)`, która wypisuje informację o zakresie liczb. Jeżeli nie chcesz używać nazwy `do_`, możesz użyć nazw `początek` i `koniec`.
6. Wywołaj jedną metodę z parametrami kilka razy z różnymi argumentami.

Ćwiczenia z return

`bool` to typ logiczny, który przyjmuje wartość `true` albo `false`.

1. Napisz metodę `int Dodaj(int a, int b)`, która zwraca sumę.
2. Napisz metodę `int Odejmij(int a, int b)`, która zwraca różnicę.
3. Napisz metodę `int Pomnoz(int a, int b)`, która zwraca iloczyn.
4. Napisz metodę `double Podziel(int a, int b)`, która zwraca wynik dzielenia jako `double`. Na tym etapie załóż, że `b` nie jest równe 0.
5. Napisz metodę `int Kwadrat(int x)`, która zwraca kwadrat liczby.
6. Napisz metodę `double ObliczSrednia(int a, int b)`, która zwraca średnią dwóch liczb.

7. Napisz metodę `string PrzygotujPowitanie(string imie)`, która zwraca tekst powitania.
8. Napisz metodę `bool CzyPełnoletni(int wiek)`, która zwraca `true`, jeśli wiek jest większy lub równy 18, w przeciwnym razie `false`.

Ćwiczenia z tablicami

1. Napisz metodę `void WypiszTablice(int[] liczby)`, która wypisuje wszystkie elementy tablicy.
2. Napisz metodę `int ObliczSume(int[] liczby)`, która zwraca sumę elementów.
3. Napisz metodę `double ObliczSrednia(int[] liczby)`, która zwraca średnią elementów.
4. Napisz metodę `int ZnajdzMinimum(int[] liczby)`, która zwraca najmniejszy element.
5. Napisz metodę `int ZnajdzMaksimum(int[] liczby)`, która zwraca największy element.
6. Napisz metodę `int PoliczParzyste(int[] liczby)`, która zwraca liczbę elementów parzystych.
7. Napisz metodę `int PoliczWiecezOd(int[] liczby, int prog)`, która zwraca liczbę elementów większych od podanego progu.
8. Napisz metodę `bool CzyZawiera(int[] liczby, int szukana)`, która zwraca `true`, jeśli tablica zawiera szukaną liczbę.

Zadania przekrojowe

1. Program „Analiza liczb”
 - utwórz tablicę liczb,
 - wypisz tablicę metodą,
 - oblicz sumę metodą,
 - oblicz średnią metodą,
 - znajdź minimum i maksimum metodami.
2. Program „Prosty kalkulator”
 - utwórz metody `Dodaj`, `Odejmij`, `Pomnoz`, `Podziel`,
 - w `Main` wywołaj każdą metodę dla przykładowych danych,
 - wypisz wyniki.
3. Program „Oceny ucznia”
 - utwórz tablicę ocen,
 - napisz metodę `ObliczSrednia`,
 - napisz metodę `ZnajdzNajlepszaOcene`,
 - napisz metodę `PoliczOcenyPozytywne`.
4. Program „Komunikaty”
 - napisz metodę `PrzygotujPowitanie`,
 - napisz metodę `PrzygotujPozegnanie`,
 - napisz metodę `PokazSeparator`,
 - użyj ich w `Main`.
5. Program „Liczby parzyste”
 - utwórz tablicę liczb,
 - napisz metodę `CzyParzysta`,
 - napisz metodę `PoliczParzyste`,
 - wypisz wynik.

Kontrola samodzielna

- Czy wiem, kiedy użyć void?
- Czy wiem, kiedy użyć return?
- Czy umiem odróżnić parametr od argumentu?
- Czy umiem zapisać wynik metody w zmiennej?
- Czy umiem przekazać tablicę do metody?
- Czy Main jest czytelny, czy zawiera zbyt dużo szczegółów?

Podsumowanie

Metody porządkują program i pomagają dzielić go na mniejsze fragmenty.

Parametry pozwalają przekazywać dane do metody, a return pozwala zwrócić wynik. Tablice również można przekazywać do metod, na przykład jako parametr `int[]` liczby.

Dobry program powinien mieć czytelny Main i sensownie nazwane metody.

42. Wprowadzenie

06-zakres-zmiennych/README.md

06 - Zakres zmiennych

W tym dziale uczeń poznaje zakres życia zmiennych, bloki kodu, zmienne lokalne, parametry metod oraz problem przesłaniania nazw.

Planowane lekcje:

- Blok kodu i zakres zmiennej (01-blok-kodu-i-zakres-zmiennej.md)
- Zmienne lokalne (02-zmienne-lokalne.md)
- Parametry metod a zmienne lokalne (03-parametry-a-zmienne-lokalne.md)
- Zmienne w pętlach (04-zmienne-w-petlach.md)
- Przesłanianie zmiennych (05-przeslanianie-zmiennych.md)
- Najczęstsze błędy (06-najczestsze-bledy.md)

43. Blok kodu i zakres zmiennej

06-zakres-zmiennych/01-blok-kodu-i-zakres-zmiennej.md

Blok kodu i zakres zmiennej

Co to jest blok kodu

Blok kodu to fragment programu ograniczony klamrami { }.

Kod: csharp

```
{  
    // instrukcje  
}
```

Blokami kodu są między innymi:

- ciało metody Main,
- ciało instrukcji if,
- ciało pętli for,
- ciało pętli while.

Blok kodu porządkuje instrukcje i wyznacza miejsce, w którym istnieją niektóre zmienne.

Zmienna lokalna

Zmienna lokalna to zmienna zadeklarowana wewnątrz metody albo bloku kodu.

Kod: csharp

```
using System;  
  
class Program  
{  
    static void Main()  
    {  
        int liczba = 10;  
  
        Console.WriteLine(liczba);  
    }  
}
```

Zmienna `liczba` istnieje wewnątrz metody `Main`. Można jej używać w tym bloku od miejsca deklaracji.

Zakres zmiennej

Zakres zmiennej to fragment programu, w którym można używać danej zmiennej.

Najważniejsza zasada:

- zmienna jest dostępna od miejsca deklaracji do końca bloku, w którym została zadeklarowana,
- po wyjściu z bloku zmienna przestaje być dostępna.

Jeżeli zmienna została zadeklarowana wewnątrz klamerek, to zwykle nie można jej użyć poza tymi klamrami.

Diagram: gdzie istnieje zmienna

Diagram Mermaid

```

config:
  flowchart:
    curve: linear
---
flowchart LR
  A([Start])
  B["Wejście do bloku"]
  C["Deklaracja zmiennej"]
  D["Użycie zmiennej"]
  E["Wyjście z bloku"]
  F["Zmienna niedostępna"]
  G([Koniec])

  A --> B
  B --> C
  C --> D
  D --> E
  E --> F
  F --> G

```

Diagram pokazuje, że zmienna istnieje tylko w określonym fragmencie programu. Po zakończeniu bloku nie można już jej używać.

Zmienna w instrukcji if

Przykład poprawny:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int wiek = 18;

        if (wiek >= 18)
        {
            string komunikat = "Osoba pełnoletnia";
            Console.WriteLine(komunikat);
        }
    }
}

```

Zmienna komunikat istnieje tylko wewnątrz bloku if. Dlatego w tym przykładzie jest wypisywana od razu w tym samym bloku.

Przykład błędny:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int wiek = 18;

        if (wiek >= 18)
        {
            string komunikat = "Osoba pełnoletnia";
        }

        Console.WriteLine(komunikat);
    }
}

```

Ten program zawiera błąd. Zmienna komunikat została zadeklarowana wewnątrz if, więc nie istnieje poza tym blokiem.

Zmienna zadeklarowana przed blokiem

Jeżeli zmienna ma być użyta po zakończeniu if, można zadeklarować ją wcześniej.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 18;
        string komunikat = "";

        if (wiek >= 18)
        {
            komunikat = "Osoba pełnoletnia";
        }

        Console.WriteLine(komunikat);
    }
}
```

Zmienna komunikat została zadeklarowana przed instrukcją if, więc można jej użyć także po zakończeniu bloku if.

Zmienna w pętli for

W pętli for często deklarujemy zmienną licznika.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i);
        }
    }
}
```

Zmienna i istnieje w pętli for i jest używana jako licznik.

Przykład błędny:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i);
        }

        Console.WriteLine(i);
    }
}
```

```
}  
}
```

Po zakończeniu pętli zmienna `i` nie jest dostępna. Dlatego ostatnia instrukcja powoduje błąd.

Dlaczego to jest ważne

Zakres zmiennych jest ważny, ponieważ:

- program jest czytelniejszy,
- zmienne nie zaśmiecają całej metody,
- łatwiej unikać przypadkowego użycia niewłaściwej zmiennej,
- kompilator pomaga wykrywać błędy.

Dzięki zakresom zmiennych program ma lepszy porządek.

Najczęstsze błędy

- Użycie zmiennej poza blokiem, w którym została zadeklarowana.
- Deklarowanie zmiennej zbyt głęboko.
- Oczekiwanie, że zmienna z `if` będzie dostępna po `if`.
- Oczekiwanie, że licznik pętli `for` będzie dostępny po pętli.
- Mylenie bloku kodu z wcięciami.

W C# o bloku decydują klamry `{ }`, a nie samo wcięcie. Wcięcia pomagają czytać kod, ale nie wyznaczają zakresu zmiennej.

Ćwiczenia

1. Wskaż, w którym bloku zadeklarowana jest zmienna `liczba`.
2. Napisz program, w którym zmienna `tekst` jest zadeklarowana w `Main` i wypisana na ekranie.
3. Napisz program z `if`, w którym zmienna `komunikat` jest zadeklarowana wewnątrz `if` i wypisana wewnątrz `if`.
4. Popraw program, w którym zmienna zadeklarowana w `if` jest używana poza `if`.
5. Napisz pętlę `for` z licznikiem `i` i wyjaśnij, gdzie dostępna jest zmienna `i`.
6. Sprawdź, co się stanie, gdy spróbujesz użyć zmiennej `i` po zakończeniu pętli `for`.
7. Napisz program, w którym zmienna `wynik` jest zadeklarowana przed `if`, a jej wartość jest ustawiana wewnątrz `if`.

Podsumowanie

Blok kodu w C# wyznaczają klamry `{ }`.

Zmienna lokalna istnieje tylko w określonym zakresie. Jest dostępna od miejsca deklaracji do końca bloku, w którym została zadeklarowana.

Zmienna zadeklarowana w `if` nie jest dostępna poza `if`. Zmienna licznika `for` zwykle nie jest dostępna po zakończeniu pętli.

Dobre rozumienie zakresu zmiennych pomaga pisać poprawne i czytelne programy.

44. Zmienne lokalne

06-zakres-zmiennych/02-zmienne-lokalne.md

Zmienne lokalne

Co to jest zmienna lokalna

Zmienna lokalna to zmienna zadeklarowana wewnątrz metody lub bloku kodu.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 10;
        Console.WriteLine(liczba);
    }
}
```

Zmienna `liczba` jest zmienną lokalną metody `Main`. Można jej używać w tej metodzie od miejsca deklaracji do końca jej zakresu.

Zmienna lokalna przechowuje dane tymczasowe

Zmienna lokalna często służy do przechowania wartości potrzebnej tylko przez pewien fragment programu.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int a = 5;
        int b = 7;
        int suma = a + b;

        Console.WriteLine(suma);
    }
}
```

Zmienna `suma` jest wynikiem pomocniczym. Jest potrzebna po to, aby zapamiętać wynik dodawania przed wypisaniem go na ekranie.

Zmienna lokalna w metodzie pomocniczej

Zmienne lokalne mogą występować nie tylko w `Main`, ale także w innych metodach.

Kod: csharp

```
using System;

class Program
{
    static void PokazWynik()
    {
        int a = 4;
        int b = 6;
    }
}
```

```

        int suma = a + b;

        Console.WriteLine(suma);
    }

    static void Main()
    {
        PokazWynik();
    }
}

```

Zmienne a, b i suma istnieją wewnątrz metody PokazWynik. Nie są dostępne bezpośrednio w metodzie Main.

Diagram: zmienne lokalne w metodzie

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Wejście do metody"]
    C["Utworzenie zmiennych"]
    D["Użycie zmiennych"]
    E["Koniec metody"]
    F["Zmienne niedostępne"]
    G([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G

```

Diagram pokazuje, że zmienne lokalne istnieją podczas wykonywania metody. Po zakończeniu metody nie można już ich używać.

Zmienna lokalna w if

Zmienną lokalną można zadeklarować także wewnątrz bloku if.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        if (true)
        {
            string tekst = "Warunek spełniony";
            Console.WriteLine(tekst);
        }
    }
}

```

Zmienna tekst istnieje tylko w bloku if. Po zamknięciu klamry } tego bloku nie jest już dostępna.

Zmienna lokalna w pętli

Zmienna lokalna może być używana wewnątrz pętli.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            int kwadrat = i * i;
            Console.WriteLine(kwadrat);
        }
    }
}
```

W tym przykładzie:

- `i` jest licznikiem pętli,
- `kwadrat` jest zmienną lokalną tworzoną w każdej iteracji bloku pętli,
- po zakończeniu bloku pętli `kwadrat` nie jest dostępny.

Dobre nazwy zmiennych lokalnych

Nazwa zmiennej lokalnej powinna mówić, do czego zmienna służy.

Dobre nazwy:

- `suma`,
- `srednia`,
- `minimum`,
- `licznik`,
- `komunikat`.

Słabe nazwy:

- `x`,
- `y`,
- `a1`,
- `temp`, jeśli nie wiadomo, co oznacza.

Krótkie nazwy typu `i` są akceptowalne jako licznik prostej pętli.

Zmienna lokalna a parametr metody

Zmienna lokalna jest tworzona wewnątrz metody lub bloku.

Parametr metody także ma nazwę i przechowuje wartość, ale otrzymuje ją z miejsca, w którym metoda została wywołana.

Szczegóły parametrów będą jeszcze wykorzystywane w kolejnych lekcjach. Na tym etapie najważniejsze jest to, że zarówno zmienne lokalne, jak i parametry mają swój zakres.

Najczęstsze błędy

- Użycie zmiennej przed deklaracją.
- Użycie zmiennej poza jej zakresem.

- Deklarowanie zmiennej, która nigdy nie jest używana.
- Nadawanie zmiennym nieczytelnych nazw.
- Mylenie zmiennej lokalnej z parametrem metody.
- Deklarowanie zmiennej zbyt wcześnie albo zbyt późno.

Ćwiczenia

1. Napisz program, w którym w Main zadeklarujesz zmienną lokalną liczba i wypiszesz ją na ekranie.
2. Napisz program, w którym zmienne a i b przechowują dwie liczby, a zmienna suma przechowuje ich sumę.
3. Napisz metodę PokazSume, w której zadeklarujesz zmienne lokalne a, b i suma.
4. Napisz program z if, w którym zmienna komunikat jest zadeklarowana i użyta wewnątrz bloku if.
5. Napisz pętlę for, w której zmienna kwadrat jest obliczana wewnątrz bloku pętli.
6. Sprawdź, co stanie się, gdy spróbujesz użyć zmiennej zadeklarowanej w if poza blokiem if.
7. Popraw nazwy zmiennych w krótkim programie tak, aby były bardziej czytelne.

Podsumowanie

Zmienna lokalna jest zadeklarowana wewnątrz metody lub bloku.

Zmienna lokalna służy do przechowywania danych tymczasowych i istnieje tylko w swoim zakresie.

Zmienna lokalna powinna mieć czytelną nazwę. Dobre rozumienie zmiennych lokalnych ułatwia pisanie metod i większych programów.

45. Parametry metod a zmienne lokalne

06-zakres-zmiennych/03-parametry-a-zmienne-lokalne.md

Parametry metod a zmienne lokalne

Przypomnienie: parametr

Parametr znajduje się w definicji metody i przyjmuje wartość przekazaną podczas wywołania metody.

Kod: csharp

```
using System;

class Program
{
    static void PokazImie(string imie)
    {
        Console.WriteLine(imie);
    }

    static void Main()
    {
        PokazImie("Ala");
    }
}
```

imie jest parametrem metody PokazImie. Metoda może używać tego parametru w swoim ciele.

Przypomnienie: zmienna lokalna

Zmienna lokalna jest zadeklarowana wewnątrz metody lub bloku kodu.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        string imie = "Ala";
        Console.WriteLine(imie);
    }
}
```

imie jest zmienną lokalną metody Main.

Podobieństwo

Parametr i zmienna lokalna są podobne, ponieważ:

- mają nazwę,
- mają typ,
- można ich używać w metodzie,
- przechowują wartość.

W obu przypadkach trzeba zwracać uwagę na zakres, czyli miejsce, w którym dana nazwa jest dostępna.

Różnica

Cecha	Parametr	Zmienna lokalna
-----	-----	-----
Gdzie powstaje?	W nagłówku metody	W ciele metody lub bloku
Skąd ma wartość?	Z argumentu przy wywołaniu	Z przypisania w kodzie
Gdzie jest dostępny?	W metodzie	W swoim bloku
Przykład	`string imie` w nagłówku metody	`int suma = 0;` w ciele metody

Parametr jest częścią nagłówka metody. Zmienna lokalna powstaje dopiero w instrukcji zapisanej w ciele metody albo bloku.

Osobne zakresy metod

Każda metoda ma własny zakres zmiennych.

Kod: csharp

```
using System;

class Program
{
    static void PokazImie(string imie)
    {
        Console.WriteLine(imie);
    }

    static void Main()
    {
        string imie = "Ala";

        PokazImie(imie);
    }
}
```

W tym przykładzie:

- zmienna `imie` w `Main` jest zmienną lokalną metody `Main`,
- parametr `imie` w `PokazImie` jest parametrem tej metody,
- mimo tej samej nazwy są to dwa różne miejsca w kodzie,
- argument z `Main` przekazuje wartość do parametru metody.

Diagram: argument i parametr

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Main: imie = Ala"]
    C["PokazImie(imie)"]
    D["Parametr imie"]
    E["Console.WriteLine"]
    F["Powrót do Main"]
    G([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
```

Diagram pokazuje, że zmienna lokalna z Main jest argumentem przy wywołaniu metody. Jej wartość trafia do parametru metody PokazImie.

Zmienna z Main nie jest automatycznie widoczna w innej metodzie

Przykład błędny:

Kod: csharp

```
using System;

class Program
{
    static void PokazImie()
    {
        Console.WriteLine(imie);
    }

    static void Main()
    {
        string imie = "Ala";

        PokazImie();
    }
}
```

Ten program zawiera błąd. Metoda PokazImie nie widzi zmiennej lokalnej imie z metody Main.

Poprawna wersja:

Kod: csharp

```
using System;

class Program
{
    static void PokazImie(string imie)
    {
        Console.WriteLine(imie);
    }

    static void Main()
    {
        string imie = "Ala";

        PokazImie(imie);
    }
}
```

Wartość trzeba przekazać przez argument do parametru. Dzięki temu metoda PokazImie otrzymuje dane, których potrzebuje.

Czy można użyć innej nazwy parametru

Nazwa argumentu i nazwa parametru nie muszą być takie same.

Kod: csharp

```
using System;

class Program
{
    static void PokazImie(string tekst)
    {
        Console.WriteLine(tekst);
    }
}
```

```
static void Main()
{
    string imie = "Ala";

    PokazImie(imie);
}
```

W Main zmienna nazywa się `imie`, a parametr metody nazywa się `tekst`. Program nadal działa, ponieważ ważna jest przekazana wartość i zgodny typ.

Najczęstsze błędy

- Próba użycia zmiennej lokalnej z Main w innej metodzie bez parametru.
- Mylenie parametru z argumentem.
- Przekonanie, że takie same nazwy oznaczają tę samą zmienną.
- Użycie parametru poza metodą.
- Nieprzekazanie argumentu przy wywołaniu metody z parametrem.

Ćwiczenia

1. Wskaż w przykładzie, co jest parametrem, a co argumentem.
2. Napisz metodę `PokazImie(string imie)` i wywołaj ją z Main.
3. Napisz metodę `PokazLiczbe(int liczba)` i przekaz do niej zmienną lokalną z Main.
4. Zmień nazwę parametru w metodzie i sprawdź, czy program nadal działa.
5. Popraw program, w którym metoda próbuje użyć zmiennej lokalnej z Main bez parametru.
6. Napisz metodę `PokazDane(string imie, int wiek)` i wywołaj ją z Main.
7. Własnymi słowami wyjaśnij różnicę między parametrem a zmienną lokalną.

Podsumowanie

Parametr znajduje się w nagłówku metody. Zmienna lokalna powstaje w ciele metody lub bloku.

Każda metoda ma własny zakres. Zmienna z Main nie jest automatycznie widoczna w innej metodzie.

Dane przekazujemy do metody przez argumenty i parametry.

46. Zmienne w pętlach

06-zakres-zmiennych/04-zmienne-w-petlach.md

Zmienne w pętlach

Dlaczego zakres zmiennych w pętli jest ważny

W pętlach często używamy kilku rodzajów zmiennych:

- licznika pętli,
- akumulatora,
- licznika elementów spełniających warunek,
- zmiennych pomocniczych tworzonych wewnątrz pętli.

Miejsce deklaracji zmiennej decyduje o tym, gdzie można jej używać. Niektóre zmienne są potrzebne tylko w pętli, a inne muszą być dostępne także po jej zakończeniu.

Licznik pętli for

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i);
        }
    }
}
```

W tym przykładzie:

- `i` jest licznikiem pętli,
- `i` jest zadeklarowane w nagłówku pętli `for`,
- `i` jest dostępne w warunku, w kroku pętli i w ciele pętli,
- po zakończeniu pętli `i` zwykle nie jest dostępne.

Przykład błędny:

Kod: **csharp**

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i);
        }

        Console.WriteLine(i);
    }
}
```

To jest błąd, ponieważ `i` nie istnieje poza pętlą `for`.

Zmienna pomocnicza wewnątrz pętli

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            int kwadrat = i * i;
            Console.WriteLine(kwadrat);
        }
    }
}
```

W tym przykładzie:

- kwadrat powstaje wewnątrz bloku pętli,
- można go używać tylko w tym bloku,
- w każdej iteracji tworzona jest nowa wartość pomocnicza,
- po wyjściu z bloku pętli kwadrat nie jest dostępny.

Akumulator przed pętlą

Akumulator musi istnieć przed pętlą, jeżeli chcemy go aktualizować w kolejnych iteracjach i użyć po zakończeniu pętli.

Kod: `csharp`

```
using System;

class Program
{
    static void Main()
    {
        int suma = 0;

        for (int i = 1; i <= 5; i++)
        {
            suma += i;
        }

        Console.WriteLine(suma);
    }
}
```

W tym przykładzie:

- suma jest zadeklarowana przed pętlą,
- w pętli jest aktualizowana,
- po pętli można wypisać jej końcową wartość.

Diagram: zmienna w pętli a zmienna przed pętlą

Diagram Mermaid

```
---
config:
```

```

flowchart:
  curve: linear
---
flowchart TD
  A([Start])
  B["Deklaracja suma"]
  C["Pętla aktualizuje suma"]
  D["suma nadal dostępna"]
  E["Pętla tworzy kwadrat"]
  F["kwadrat niedostępny"]
  G([Koniec])

  A --> B
  B --> C
  C --> D
  D --> G
  A --> E
  E --> F
  F --> G

```

Diagram pokazuje różnicę między zmienną zadeklarowaną przed pętlą a zmienną zadeklarowaną w bloku pętli. suma jest dostępna po pętli, a kwadrat nie.

Licznik spełnionych warunków

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[] liczby = { 3, 8, 12, 5, 10 };
        int licznik = 0;

        foreach (int liczba in liczby)
        {
            if (liczba > 7)
            {
                licznik++;
            }
        }

        Console.WriteLine(licznik);
    }
}

```

W tym przykładzie:

- licznik musi być zadeklarowany przed pętlą,
- pętla zwiększa licznik tylko wtedy, gdy warunek jest spełniony,
- po zakończeniu pętli licznik zawiera wynik.

Błąd: akumulator zadeklarowany w pętli

Przykład błędny:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {

```



```

        for (int i = 1; i <= 5; i++)
        {
            int suma = 0;
            suma += i;
        }

        Console.WriteLine(suma);
    }
}

```

Ten program zawiera błąd:

- suma została zadeklarowana wewnątrz pętli,
- poza pętlą nie jest dostępna,
- dodatkowo w każdej iteracji zaczynałaby od 0,
- dlatego akumulator deklarujemy przed pętlą.

Poprawna wersja:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int suma = 0;

        for (int i = 1; i <= 5; i++)
        {
            suma += i;
        }

        Console.WriteLine(suma);
    }
}

```

Zmienna w pętli while

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int i = 0;

        while (i < 5)
        {
            Console.WriteLine(i);
            i++;
        }
    }
}

```

Przy pętli while licznik często deklaruje się przed pętlą, ponieważ warunek pętli musi go znać.

Najczęstsze błędy

- Użycie licznika for po zakończeniu pętli.
- Deklarowanie akumulatora wewnątrz pętli.
- Zerowanie akumulatora w każdej iteracji.

- Użycie zmiennej pomocniczej poza blokiem pętli.
- Brak aktualizacji licznika w `while`.
- Mylenie licznika pętli z licznikiem spełnionych warunków.

Ćwiczenia

1. Napisz pętlę `for`, która wypisuje liczby od 0 do 4. Wskaż, gdzie istnieje zmienna `i`.
2. Sprawdź, co się stanie, gdy spróbujesz wypisać `i` po zakończeniu pętli `for`.
3. Napisz pętlę, w której wewnątrz bloku tworzona jest zmienna kwadrat.
4. Napisz program, który sumuje liczby od 1 do 10, używając akumulatora `suma`.
5. Popraw program, w którym `suma` jest deklarowana wewnątrz pętli.
6. Napisz program, który liczy, ile elementów tablicy jest większych od 10.
7. Napisz program z `while`, w którym licznik jest zadeklarowany przed pętlą.
8. Wskaż, które zmienne powinny być zadeklarowane przed pętlą, a które wewnątrz pętli.

Podsumowanie

Licznik `for` zwykle istnieje tylko w pętli.

Zmienna pomocnicza zadeklarowana w bloku pętli nie jest dostępna poza tym blokiem.

Akumulator powinien być zadeklarowany przed pętlą. Licznik spełnionych warunków także zwykle deklarujemy przed pętlą.

Miejsce deklaracji zmiennej decyduje o tym, gdzie można jej używać.

47. Przesłanianie zmiennych

06-zakres-zmiennych/05-przeslanianie-zmiennych.md

Przesłanianie zmiennych

O co chodzi z nazwami

Każda zmienna ma nazwę. Nazwa pozwala odwołać się do konkretnej wartości zapisanej w programie. Zakres zmiennej decyduje o tym, gdzie dana nazwa jest widoczna. Ta sama nazwa może pojawić się w różnych miejscach programu, ale nie zawsze oznacza to tę samą zmienną.

Czym jest przesłanianie

Przesłanianie nazwy oznacza sytuację, w której w bliższym zakresie pojawia się nazwa taka sama jak nazwa istniejąca w dalszym zakresie. Wtedy bliższa nazwa może zasłonić dalszą.

Na tym etapie traktujemy to jako temat ostrzegawczy. Lepiej pisać kod tak, aby nie mylić nazw i nie zmuszać czytelnika do zgadywania, o którą zmienną chodzi.

Przykład: pole klasy i zmienna lokalna

Kod: csharp

```
using System;

class Program
{
    static int liczba = 100;

    static void Main()
    {
        int liczba = 5;

        Console.WriteLine(liczba);
    }
}
```

W tym przykładzie:

- `static int liczba = 100;` jest polem klasy `Program`,
- `int liczba = 5;` jest zmienną lokalną metody `Main`,
- w `Main` nazwa `liczba` odnosi się do zmiennej lokalnej,
- lokalna zmienna przesłania pole klasy o tej samej nazwie,
- program wypisze 5.

Pola klasy będą dokładniej omawiane w dziale o obiektach. Tutaj przykład służy tylko do pokazania zjawiska przesłaniania.

Diagram: która nazwa jest bliżej

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
```

```

flowchart LR
    A([Start])
    B["Pole klasy<br/>liczba = 100"]
    C["Wejście do Main"]
    D["Zmienna lokalna<br/>liczba = 5"]
    E["WriteLine używa<br/>lokalnej liczba"]
    F["Wynik: 5"]
    G([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G

```

Diagram pokazuje, że w metodzie Main bliżej jest zmienna lokalna liczba, dlatego to jej wartość zostanie wypisana.

Dlaczego to może być mylące

Taki kod jest legalny, ale może być nieczytelny. Osoba czytająca program musi sprawdzić, czy liczba oznacza pole klasy, czy zmienną lokalną.

Lepsza wersja:

Kod: csharp

```

using System;

class Program
{
    static int domyslnaLiczba = 100;

    static void Main()
    {
        int aktualnaLiczba = 5;

        Console.WriteLine(aktualnaLiczba);
    }
}

```

Różne nazwy lepiej pokazują sens danych. `domyslnaLiczba` i `aktualnaLiczba` nie wyglądają jak przypadkowo powtórzona ta sama nazwa.

C# nie pozwala na wszystko

Nie należy zakładać, że w każdym zagnieżdżonym bloku można utworzyć zmienną o tej samej nazwie. C# chroni przed częścią takich pomyłek.

Przykład błędny:

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int liczba = 10;

        if (liczba > 0)
        {
            int liczba = 5;
            Console.WriteLine(liczba);
        }
    }
}

```

```
    }  
  }  
}
```

W C# taka ponowna deklaracja zmiennej lokalnej o tej samej nazwie w nachodzącym zakresie jest błędem kompilacji.

Kompilator nie pozwala w tym miejscu utworzyć drugiej lokalnej zmiennej `liczba`, ponieważ mogłoby to prowadzić do niejasności.

Parametr i zmienna lokalna o tej samej nazwie

Przykład błędny:

Kod: csharp

```
using System;  
  
class Program  
{  
    static void PokazLiczbe(int liczba)  
    {  
        int liczba = 5;  
        Console.WriteLine(liczba);  
    }  
  
    static void Main()  
    {  
        PokazLiczbe(10);  
    }  
}
```

Parametr `liczba` już istnieje w zakresie metody `PokazLiczbe`, więc nie można w tej samej metodzie zadeklarować drugiej zmiennej lokalnej o tej samej nazwie.

Poprawna wersja:

Kod: csharp

```
using System;  
  
class Program  
{  
    static void PokazLiczbe(int liczba)  
    {  
        int podwojona = liczba * 2;  
        Console.WriteLine(podwojona);  
    }  
  
    static void Main()  
    {  
        PokazLiczbe(10);  
    }  
}
```

Nazwa `podwojona` jasno mówi, że zmienna przechowuje wartość obliczoną na podstawie parametru.

Dobre praktyki

- Nie używaj tej samej nazwy dla różnych rzeczy, jeśli nie musisz.
- Wybieraj nazwy opisujące rolę zmiennej.
- Unikaj nazw typu `x`, `y`, `temp`, jeśli nie wiadomo, co oznaczają.
- Dla pola klasy i zmiennej lokalnej używaj nazw pokazujących różnicę znaczenia.
- Jeżeli kod zaczyna wymagać tłumaczenia, nazwy prawdopodobnie są za słabe.

Najczęstsze błędy

- Przekonanie, że taka sama nazwa zawsze oznacza tę samą zmienną.
- Próba ponownej deklaracji zmiennej lokalnej o tej samej nazwie.
- Mylenie pola klasy ze zmienną lokalną.
- Nadawanie parametrowi i zmiennej lokalnej tej samej nazwy.
- Używanie zbyt ogólnych nazw.

Ćwiczenia

1. Wskaż, która zmienna zostanie użyta w przykładzie z polem klasy `liczba` i zmienną lokalną `liczba`.
2. Zmień nazwy w przykładzie tak, aby kod był czytelniejszy.
3. Sprawdź, czy C# pozwoli zadeklarować zmienną lokalną o tej samej nazwie w bloku `if`.
4. Popraw błędny kod, w którym parametr i zmienna lokalna mają tę samą nazwę.
5. Podaj lepsze nazwy dla zmiennych `x`, `y` i `temp` w krótkim programie sumującym liczby.
6. Wyjaśnij własnymi słowami, dlaczego przesłanianie może utrudniać czytanie programu.
7. Napisz krótki program, w którym nazwy zmiennych jasno pokazują ich przeznaczenie.

Podsumowanie

Przesłanianie oznacza zasłonięcie nazwy z dalszego zakresu przez nazwę z bliższego zakresu.

C# nie pozwala na każdą ponowną deklarację tej samej nazwy. Pole klasy może zostać przesłonięte przez zmienną lokalną, ale zmienna lokalna nie zawsze może zostać ponownie zadeklarowana w zagnieżdżonym bloku.

Taka sama nazwa nie zawsze oznacza tę samą rzecz. Początkujący programista powinien raczej unikać przesłaniania i wybierać czytelne nazwy.

48. Najczęstsze błędy

06-zakres-zmiennych/06-najczestsze-bledy.md

Najczęstsze błędy — zakres zmiennych

Po co analizować błędy

Błędy zakresu zmiennych są częste u początkujących. Programista widzi nazwę w kodzie, ale kompilator sprawdza zakres, czyli miejsce, w którym dana zmienna naprawdę istnieje.

W tej lekcji każdy błąd ma krótki przykład, wyjaśnienie problemu i poprawioną wersję.

Błąd 1: użycie zmiennej poza blokiem if

Przykład błędny:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 18;

        if (wiek >= 18)
        {
            string komunikat = "Osoba pełnoletnia";
        }

        Console.WriteLine(komunikat);
    }
}
```

Problem:

- komunikat został zadeklarowany wewnątrz bloku if,
- poza blokiem if nie istnieje,
- dlatego `Console.WriteLine` nie ma dostępu do tej zmiennej.

Poprawka:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int wiek = 18;
        string komunikat = "";

        if (wiek >= 18)
        {
            komunikat = "Osoba pełnoletnia";
        }

        Console.WriteLine(komunikat);
    }
}
```

Błąd 2: użycie licznika for po pętli

Przykład błędny:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i);
        }

        Console.WriteLine(i);
    }
}
```

Problem:

- `i` zostało zadeklarowane w nagłówku pętli `for`,
- po zakończeniu pętli `i` nie jest dostępne,
- licznik pętli zwykle służy tylko do sterowania pętlą.

Poprawka bez wypisywania `i` po pętli:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 0; i < 5; i++)
        {
            Console.WriteLine(i);
        }
    }
}
```

Poprawka z osobną zmienną, jeśli wartość jest naprawdę potrzebna po pętli:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int ostatniaLiczba = 0;

        for (int i = 0; i < 5; i++)
        {
            ostatniaLiczba = i;
            Console.WriteLine(i);
        }

        Console.WriteLine(ostatniaLiczba);
    }
}
```

Błąd 3: akumulator zadeklarowany wewnątrz pętli

Przykład błędny:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            int suma = 0;
            suma += i;
        }

        Console.WriteLine(suma);
    }
}
```

Problem:

- suma została zadeklarowana wewnątrz pętli,
- poza pętlą nie istnieje,
- dodatkowo w każdej iteracji byłaby tworzona od nowa,
- akumulator powinien być zadeklarowany przed pętlą.

Poprawka:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int suma = 0;

        for (int i = 1; i <= 5; i++)
        {
            suma += i;
        }

        Console.WriteLine(suma);
    }
}
```

Błąd 4: użycie zmiennej przed deklaracją

Przykład błędny:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine(liczba);

        int liczba = 10;
    }
}
```

Problem:

- zmienna jest dostępna dopiero od miejsca deklaracji,
- nie można użyć zmiennej wcześniej, niż została zadeklarowana.

Poprawka:

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int liczba = 10;

        Console.WriteLine(liczba);
    }
}
```

Błąd 5: metoda próbuje użyć zmiennej lokalnej z Main

Przykład błędny:

Kod: csharp

```
using System;

class Program
{
    static void PokazWynik()
    {
        Console.WriteLine(wynik);
    }

    static void Main()
    {
        int wynik = 100;

        PokazWynik();
    }
}
```

Problem:

- wynik jest zmienną lokalną Main,
- metoda PokazWynik ma własny zakres,
- metoda PokazWynik nie widzi automatycznie zmiennych lokalnych z Main,
- dane trzeba przekazać przez parametr.

Poprawka:

Kod: csharp

```
using System;

class Program
{
    static void PokazWynik(int wynik)
    {
        Console.WriteLine(wynik);
    }

    static void Main()
    {
        int wynik = 100;

        PokazWynik(wynik);
    }
}
```

```
}  
}
```

Błąd 6: parametr i zmienna lokalna o tej samej nazwie

Przykład błędny:

Kod: **csharp**

```
using System;  
  
class Program  
{  
    static void PokazLiczbe(int liczba)  
    {  
        int liczba = 5;  
  
        Console.WriteLine(liczba);  
    }  
  
    static void Main()  
    {  
        PokazLiczbe(10);  
    }  
}
```

Problem:

- parametr `liczba` już istnieje w zakresie metody,
- nie można zadeklarować drugiej zmiennej lokalnej o tej samej nazwie w tej samej metodzie,
- należy użyć innej nazwy dla zmiennej pomocniczej.

Poprawka:

Kod: **csharp**

```
using System;  
  
class Program  
{  
    static void PokazLiczbe(int liczba)  
    {  
        int podwojona = liczba * 2;  
  
        Console.WriteLine(podwojona);  
    }  
  
    static void Main()  
    {  
        PokazLiczbe(10);  
    }  
}
```

Błąd 7: mylące przesłanianie pola klasy

Kod legalny, ale mylący:

Kod: **csharp**

```
using System;  
  
class Program  
{  
    static int liczba = 100;  
  
    static void Main()  
    {  
        int liczba = 5;  
  
        Console.WriteLine(liczba);  
    }  
}
```

```

        Console.WriteLine(liczba);
    }
}

```

Problem:

- kod może być legalny,
- lokalna liczba przesłania pole klasy,
- program wypisze 5,
- mimo że kod działa, jest mniej czytelny.

Lepsza wersja z różnymi nazwami:

Kod: csharp

```

using System;

class Program
{
    static int domyslnaLiczba = 100;

    static void Main()
    {
        int aktualnaLiczba = 5;

        Console.WriteLine(aktualnaLiczba);
    }
}

```

Jak unikać błędów zakresu

- Deklaruj zmienną tam, gdzie naprawdę jest potrzebna.
- Jeżeli wynik ma być użyty po pętli, zadeklaruj zmienną przed pętlą.
- Jeżeli metoda potrzebuje danych z Main, prześlij je przez parametry.
- Nie używaj tej samej nazwy dla różnych rzeczy.
- Nie zakładaj, że wcięcia decydują o zakresie. W C# decydują klamry.
- Czytaj komunikaty kompilatora.

Ćwiczenia

1. Popraw program, w którym zmienna zadeklarowana w `if` jest używana poza `if`.
2. Popraw program, w którym licznik `for` jest używany po pętli.
3. Popraw program, w którym akumulator `suma` jest zadeklarowany wewnątrz pętli.
4. Popraw program, w którym zmienna jest użyta przed deklaracją.
5. Popraw program, w którym metoda próbuje użyć zmiennej lokalnej z Main.
6. Popraw program, w którym parametr i zmienna lokalna mają tę samą nazwę.
7. Wyjaśnij, dlaczego przesłanie pola klasy przez zmienną lokalną może być mylące.
8. Napisz własny przykład błędu zakresu zmiennej i jego poprawkę.

Podsumowanie

Zmienna istnieje tylko w swoim zakresie. Zakres w C# wyznaczają klamry i miejsce deklaracji.

Zmienna z `if` nie jest dostępna poza `if`. Licznik `for` zwykle nie jest dostępny po pętli. Akumulator deklarujemy przed pętlą.

Metoda nie widzi automatycznie zmiennych lokalnych z Main. Dobre nazwy zmiennych zmniejszają ryzyko pomyłek.

49. Wprowadzenie

07-tablice-listy-referencje/README.md

□ # 07 - Tablice, listy i przekazywanie danych

Ten dział rozwija pracę z danymi złożonymi w C#. Omawia tablice dwuwymiarowe, podstawy List<T> oraz przekazywanie danych do metod, w tym ref i out.

Lekcje

- Tablice dwuwymiarowe (01-tablice-dwuwymiarowe.md)
- Tablice wielowymiarowe (02-tablice-wielowymiarowe.md)
- List<int> (03-list-int.md)
- List<string> (04-list-string.md)
- List a tablica (05-list-a-tablica.md)
- Przekazywanie parametrów przez wartość (06-przekazywanie-parametrow-przez-wartosc.md)
- ref i out (07-ref-i-out.md)
- Ćwiczenia (08-cwiczenia.md)

50. Tablice dwuwymiarowe

07-tablice-listy-referencje/01-tablice-dwuwymiarowe.md

Tablice dwuwymiarowe

Po co są tablice dwuwymiarowe

Zwykła tablica jednowymiarowa przypomina listę wartości.

Kod: **csharp**

```
int[] liczby = { 4, 7, 2, 9 };
```

Tablica dwuwymiarowa przypomina tabelę, czyli dane ułożone w wierszach i kolumnach.

Przykłady zastosowań:

- tabela ocen,
- plansza gry,
- macierz liczb,
- dane pomiarowe w kilku kolumnach,
- arkusz podobny do prostego Excela.

Tablica jako tabela

	Kolumna 0	Kolumna 1	Kolumna 2
Wiersz 0	1	2	3
Wiersz 1	4	5	6

Elementy tablicy dwuwymiarowej mają dwa indeksy:

- pierwszy indeks oznacza wiersz,
- drugi indeks oznacza kolumnę.

Przykład:

Kod: **csharp**

```
tablica[0, 1]
```

Ten zapis oznacza element z wiersza 0 i kolumny 1.

Deklaracja tablicy dwuwymiarowej

Kod: **csharp**

```
int[,] tablica = new int[2, 3];
```

Wyjaśnienie:

- `int[,]` oznacza tablicę dwuwymiarową liczb całkowitych,
- 2 oznacza liczbę wierszy,
- 3 oznacza liczbę kolumn,
- indeksy zaczynają się od 0.

Inicjalizacja tablicy wartościami

Kod: **csharp**

```
int[,] liczby =
{
    { 1, 2, 3 },
    { 4, 5, 6 }
};
```

Ta tablica ma 2 wiersze i 3 kolumny.

Dostęp do elementu tablicy

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[,] liczby =
        {
            { 1, 2, 3 },
            { 4, 5, 6 }
        };

        Console.WriteLine(liczby[0, 0]);
        Console.WriteLine(liczby[0, 1]);
        Console.WriteLine(liczby[1, 2]);
    }
}
```

Program wypisze:

- 1, bo `liczby[0, 0]` oznacza wiersz 0, kolumnę 0,
- 2, bo `liczby[0, 1]` oznacza wiersz 0, kolumnę 1,
- 6, bo `liczby[1, 2]` oznacza wiersz 1, kolumnę 2.

Diagram: wiersz i kolumna

Diagram Mermaid

```
---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Tablica dwuwymiarowa"]
    C["Wybór wiersza"]
    D["Wybór kolumny"]
    E["Odczyt elementu"]
    F([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
```

Do odczytania elementu tablicy dwuwymiarowej potrzebne są dwa indeksy: indeks wiersza i indeks kolumny.

Liczba wierszy i kolumn

W tablicy dwuwymiarowej można sprawdzić liczbę wierszy i kolumn za pomocą `GetLength`.

- `GetLength(0)` zwraca liczbę wierszy,
- `GetLength(1)` zwraca liczbę kolumn.

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[,] liczby =
        {
            { 1, 2, 3 },
            { 4, 5, 6 }
        };

        int wiersze = liczby.GetLength(0);
        int kolumny = liczby.GetLength(1);

        Console.WriteLine(wiersze);
        Console.WriteLine(kolumny);
    }
}
```

Program wypisze 2, a potem 3.

Przejęcie po tablicy dwiema pętlami

Kod: csharp

```
using System;

class Program
{
    static void Main()
    {
        int[,] liczby =
        {
            { 1, 2, 3 },
            { 4, 5, 6 }
        };

        for (int wiersz = 0; wiersz < liczby.GetLength(0); wiersz++)
        {
            for (int kolumna = 0; kolumna < liczby.GetLength(1); kolumna++)
            {
                Console.Write(liczby[wiersz, kolumna] + " ");
            }

            Console.WriteLine();
        }
    }
}
```

Wyjaśnienie:

- zewnętrzna pętla przechodzi po wierszach,
- wewnętrzna pętla przechodzi po kolumnach,
- `Console.Write` wypisuje element bez przejścia do nowej linii,
- `Console.WriteLine` po wewnętrznej pętli przechodzi do następnego wiersza.

Suma wszystkich elementów

Kod: csharp

```
int suma = 0;

for (int wiersz = 0; wiersz < liczby.GetLength(0); wiersz++)
{
    for (int kolumna = 0; kolumna < liczby.GetLength(1); kolumna++)
    {
        suma += liczby[wiersz, kolumna];
    }
}

Console.WriteLine(suma);
```

Zmienna suma musi być zadeklarowana przed pętlami, ponieważ jest aktualizowana w wielu obrotach pętli i używana po ich zakończeniu.

Suma jednego wiersza

Przykład dla wiersza o indeksie 0:

Kod: csharp

```
int sumaWiersza = 0;
int wybranyWiersz = 0;

for (int kolumna = 0; kolumna < liczby.GetLength(1); kolumna++)
{
    sumaWiersza += liczby[wybranyWiersz, kolumna];
}

Console.WriteLine(sumaWiersza);
```

Wiersz jest stały, a zmienia się kolumna.

Suma jednej kolumny

Przykład dla kolumny o indeksie 1:

Kod: csharp

```
int sumaKolumny = 0;
int wybranaKolumna = 1;

for (int wiersz = 0; wiersz < liczby.GetLength(0); wiersz++)
{
    sumaKolumny += liczby[wiersz, wybranaKolumna];
}

Console.WriteLine(sumaKolumny);
```

Kolumna jest stała, a zmienia się wiersz.

Najczęstsze błędy

- Pomylenie `int[]` z `int[,]`.
- Użycie jednego indeksu zamiast dwóch.
- Pomylenie wiersza z kolumną.
- Zapomnienie, że indeksy zaczynają się od 0.
- Użycie `GetLength(0)` tam, gdzie trzeba `GetLength(1)`.
- Wyjście poza zakres tablicy.
- Użycie `Console.WriteLine` w miejscu, gdzie chcemy wypisać wiersz tabeli.

Ćwiczenia

1. Utwórz tablicę dwuwymiarową 2x3 i wpisz do niej dowolne liczby.
2. Wypisz jeden wybrany element tablicy, używając dwóch indeksów.
3. Wypisz wszystkie elementy tablicy za pomocą dwóch pętli for.
4. Wypisz tablicę w formie tabeli.
5. Oblicz sumę wszystkich elementów tablicy.
6. Oblicz sumę pierwszego wiersza.
7. Oblicz sumę drugiej kolumny.
8. Znajdź największy element w tablicy dwuwymiarowej.

Podsumowanie

Tablica dwuwymiarowa przypomina tabelę.

Pierwszy indeks oznacza wiersz, a drugi indeks oznacza kolumnę.

`int[, ]` oznacza tablicę dwuwymiarową. `GetLength(0)` zwraca liczbę wierszy, a `GetLength(1)` zwraca liczbę kolumn.

Do przejścia po tablicy dwuwymiarowej najczęściej używamy dwóch pętli.

51. Tablice wielowymiarowe

07-tablice-listy-referencje/02-tablice-wielowymiarowe.md

Tablice wielowymiarowe

Od tablicy jednowymiarowej do wielowymiarowej

Tablica jednowymiarowa przypomina listę:

Kod: **csharp**

```
int[] liczby = { 1, 2, 3 };
```

Tablica dwuwymiarowa przypomina tabelę:

Kod: **csharp**

```
int[,] tabela =  
{  
    { 1, 2, 3 },  
    { 4, 5, 6 }  
};
```

Tablica wielowymiarowa ma więcej niż jeden wymiar. Tablica dwuwymiarowa też jest tablicą wielowymiarową, ale najczęściej osobno mówimy o tablicach 2D, bo są najważniejsze w praktyce.

Wymiar tablicy

Liczba wymiarów mówi, ilu indeksów potrzeba, aby odczytać jeden element tablicy.

- Jeden wymiar: jeden indeks, np. `liczby[0]`.
- Dwa wymiary: dwa indeksy, np. `tabela[0, 1]`.
- Trzy wymiary: trzy indeksy, np. `dane[0, 1, 2]`.

Rodzaj tablicy	Składnia typu	Przykład dostępu
Jednowymiarowa	<code>int[]</code>	<code>liczby[0]</code>
Dwuwymiarowa	<code>int[,]</code>	<code>tabela[0, 1]</code>
Trójwymiarowa	<code>int[,,,]</code>	<code>dane[0, 1, 2]</code>

Tablica trójwymiarowa

Tablicę trójwymiarową można wyobrazić sobie jako kilka tabel ułożonych jedna za drugą.

Kod: **csharp**

```
int[,,,] dane = new int[2, 3, 4];
```

Wyjaśnienie:

- pierwszy wymiar ma rozmiar 2,
- drugi wymiar ma rozmiar 3,
- trzeci wymiar ma rozmiar 4,
- do elementu trzeba podać trzy indeksy.

Dostęp do elementu

Kod: **csharp**

```
using System;
```

```

class Program
{
    static void Main()
    {
        int[, ,] dane = new int[2, 3, 4];

        dane[0, 0, 0] = 10;
        dane[0, 1, 2] = 25;
        dane[1, 2, 3] = 99;

        Console.WriteLine(dane[0, 0, 0]);
        Console.WriteLine(dane[0, 1, 2]);
        Console.WriteLine(dane[1, 2, 3]);
    }
}

```

Każdy element tablicy trójwymiarowej ma trzy indeksy. W przykładzie program zapisuje i odczytuje trzy różne elementy.

Diagram: trzy indeksy

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Tablica trójwymiarowa"]
    C["Indeks 1"]
    D["Indeks 2"]
    E["Indeks 3"]
    F["Odczyt elementu"]
    G([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G

```

Diagram pokazuje, że do elementu tablicy trójwymiarowej potrzebne są trzy indeksy.

GetLength w tablicy trójwymiarowej

Metoda `GetLength` pozwala sprawdzić rozmiar wybranego wymiaru.

- `GetLength(0)` zwraca rozmiar pierwszego wymiaru,
- `GetLength(1)` zwraca rozmiar drugiego wymiaru,
- `GetLength(2)` zwraca rozmiar trzeciego wymiaru.

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[, ,] dane = new int[2, 3, 4];

        Console.WriteLine(dane.GetLength(0));
        Console.WriteLine(dane.GetLength(1));
    }
}

```

```

        Console.WriteLine(dane.GetLength(2));
    }
}

```

Program wypisze 2, potem 3, a potem 4.

Przechodzenie po tablicy trójwymiarowej

Kod: csharp

```

using System;

class Program
{
    static void Main()
    {
        int[, ,] dane = new int[2, 2, 3];

        for (int i = 0; i < dane.GetLength(0); i++)
        {
            for (int j = 0; j < dane.GetLength(1); j++)
            {
                for (int k = 0; k < dane.GetLength(2); k++)
                {
                    dane[i, j, k] = i + j + k;
                    Console.WriteLine(dane[i, j, k]);
                }
            }
        }
    }
}

```

Wyjaśnienie:

- pierwsza pętla przechodzi po pierwszym wymiarze,
- druga pętla przechodzi po drugim wymiarze,
- trzecia pętla przechodzi po trzecim wymiarze,
- im więcej wymiarów, tym trudniej czytać kod.

Kiedy to może mieć sens

Tablice wielowymiarowe mogą mieć sens na przykład dla:

- danych pomiarowych: dzień, godzina, numer czujnika,
- kilku plansz gry,
- prostych danych przestrzennych,
- zestawów tabel.

Na poziomie początkującym najważniejsze są tablice jednowymiarowe i dwuwymiarowe.

Dlaczego trzeba uważać

Tablice wielowymiarowe szybko robią się trudne do czytania. Jeżeli program ma dużo wymiarów, łatwo pomylić indeksy.

Dlatego zwykle warto bardzo dobrze opanować tablice 1D i 2D, a tablice 3D traktować jako temat dodatkowy.

Najczęstsze błędy

- Pomylenie `int[,]` z `int[, ,]`.
- Użycie za małej liczby indeksów.
- Użycie za dużej liczby indeksów.
- Pomylenie kolejności indeksów.
- Użycie `GetLength(1)` zamiast `GetLength(2)`.
- Wyjście poza zakres tablicy.
- Tworzenie tablicy wielowymiarowej tam, gdzie wystarczyłaby zwykła tablica.

Ćwiczenia

1. Wskaż, ile indeksów potrzeba do odczytu elementu z tablicy `int[]`.
2. Wskaż, ile indeksów potrzeba do odczytu elementu z tablicy `int[,]`.
3. Wskaż, ile indeksów potrzeba do odczytu elementu z tablicy `int[, ,]`.
4. Utwórz tablicę trójwymiarową `int[, ,]` o rozmiarze 2x2x2.
5. Przypisz wartość do elementu `dane[0, 1, 1]`.
6. Wypisz rozmiary wszystkich trzech wymiarów za pomocą `GetLength`.
7. Napisz trzy pętle `for` przechodzące po wszystkich elementach tablicy trójwymiarowej.

Podsumowanie

Tablica wielowymiarowa ma więcej niż jeden wymiar.

Tablica dwuwymiarowa jest najważniejszym praktycznym przykładem tablicy wielowymiarowej.

Tablica trójwymiarowa w C# ma typ na przykład `int[, ,]`. Do elementu tablicy trójwymiarowej potrzeba trzech indeksów.

`GetLength` pozwala sprawdzić rozmiar konkretnego wymiaru. Z dużą liczbą wymiarów trzeba uważać, bo kod staje się trudniejszy do czytania.

52. List<int>

07-tablice-listy-referencje/03-list-int.md

List<int>

Po co jest List<int>

Tablica ma stały rozmiar, a lista może rosnąć w trakcie działania programu.

Kod: csharp

```
int[] liczby = new int[3];
```

W tym przykładzie tablica ma miejsce na 3 liczby.

Kod: csharp

```
List<int> liczby = new List<int>();
```

W tym przykładzie tworzymy pustą listę liczb całkowitych. Do listy można później dodawać kolejne liczby.

Najważniejsze różnice:

- tablica ma z góry ustaloną liczbę elementów,
- do List<int> można dodawać kolejne liczby,
- List<int> jest wygodna, gdy nie wiemy od razu, ile elementów będzie potrzebnych.

Potrzebny using

Aby używać List<T>, trzeba dodać:

Kod: csharp

```
using System.Collections.Generic;
```

Minimalny szkielet programu:

Kod: csharp

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> liczby = new List<int>();
    }
}
```

List<int> oznacza listę przechowującą liczby całkowite.

Utworzenie listy i dodawanie elementów

Kod: csharp

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
```



```

        List<int> liczby = new List<int>();

        liczby.Add(5);
        liczby.Add(8);
        liczby.Add(12);

        Console.WriteLine(liczby.Count);
    }
}

```

Wyjaśnienie:

- Add dodaje nowy element na koniec listy,
- Count zwraca liczbę elementów listy.

Program wypisze 3, ponieważ do listy dodano trzy liczby.

Diagram: lista rośnie

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Pusta lista"]
    C["Add(5)"]
    D["1 element"]
    E["Add(8)"]
    F["2 elementy"]
    G["Add(12)"]
    H["3 elementy"]
    I([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
    G --> H
    H --> I

```

Lista może rosnąć po dodaniu kolejnych elementów.

Odczyt elementu przez indeks

Kod: csharp

```

Console.WriteLine(liczby[0]);
Console.WriteLine(liczby[1]);

```

Wyjaśnienie:

- indeksy zaczynają się od 0,
- liczby[0] oznacza pierwszy element,
- liczby[1] oznacza drugi element,
- tak jak w tablicach, wyjście poza zakres jest błędem.

Przejsie po liście pętlą for

Kod: csharp

```
for (int i = 0; i < liczby.Count; i++)
{
    Console.WriteLine(liczby[i]);
}
```

W liście używamy Count, a w tablicy używamy Length.

Pętla for jest dobra, gdy potrzebujemy indeksu elementu.

Przejsięcie po liście pętlą foreach

Kod: csharp

```
foreach (int liczba in liczby)
{
    Console.WriteLine(liczba);
}
```

Pętla foreach jest wygodna, gdy chcemy przejść po wszystkich elementach i nie potrzebujemy indeksu.

Usuwanie elementów

Kod: csharp

```
liczby.Remove(8);
```

Remove usuwa pierwsze wystąpienie konkretnej wartości.

Kod: csharp

```
liczby.RemoveAt(0);
```

RemoveAt usuwa element z podanego indeksu.

Po usunięciu elementu lista ma mniejszy Count.

Sprawdzenie, czy lista zawiera element

Kod: csharp

```
if (liczby.Contains(12))
{
    Console.WriteLine("Lista zawiera 12");
}
```

Contains zwraca true albo false. Dzięki temu można sprawdzić, czy na liście znajduje się wybrana wartość.

Suma elementów listy

Kod: csharp

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<int> liczby = new List<int>();

        liczby.Add(5);
        liczby.Add(8);
        liczby.Add(12);

        int suma = 0;
```

```

        foreach (int liczba in liczby)
        {
            suma += liczba;
        }

        Console.WriteLine(suma);
    }
}

```

Sumowanie działa podobnie jak przy tablicy. Różnica polega na tym, że elementy przechowuje lista.

Tablica a lista

Cecha	<code>int[]</code>	<code>List<int></code>
--- --- ---		
Rozmiar	Zwykle ustalony przy tworzeniu	Może rosnąć
Liczba elementów	<code>Length</code>	<code>Count</code>
Dodawanie elementu	Niewygodne	<code>Add</code>
Odczyt po indeksie	<code>liczby[0]</code>	<code>liczby[0]</code>
Dobre zastosowanie	Gdy znamy rozmiar	Gdy rozmiar może się zmieniać

Najczęstsze błędy

- Brak `using System.Collections.Generic`.
- Mylenie `Length` z `Count`.
- Próba odczytu elementu, którego nie ma.
- Pomylenie `Remove` z `RemoveAt`.
- Zapomnienie, że indeksy zaczynają się od 0.
- Używanie `List<int>`, gdy zwykła tablica w prostym zadaniu wystarcza.

Ćwiczenia

1. Utwórz pustą listę `List<int>`.
2. Dodaj do listy trzy liczby metodą `Add`.
3. Wypisz liczbę elementów listy za pomocą `Count`.
4. Wypisz pierwszy element listy.
5. Wypisz wszystkie elementy listy pętlą `for`.
6. Wypisz wszystkie elementy listy pętlą `foreach`.
7. Oblicz sumę elementów listy.
8. Sprawdź, czy lista zawiera wybraną liczbę za pomocą `Contains`.

Podsumowanie

`List<int>` przechowuje liczby całkowite.

Lista może rosnąć w trakcie działania programu. `Add` dodaje element, a `Count` zwraca liczbę elementów.

Elementy listy można odczytywać przez indeks. Pętla `foreach` dobrze nadaje się do przechodzenia po wszystkich elementach.

`List<int>` jest wygodna, gdy liczba danych nie jest znana z góry.

53. List<string>

07-tablice-listy-referencje/04-list-string.md

List<string>

Lista tekstów

List<int> przechowuje liczby całkowite, a List<string> przechowuje teksty.

Kod: csharp

```
List<int> liczby = new List<int>();
```

Kod: csharp

```
List<string> imiona = new List<string>();
```

Wyjaśnienie:

- string oznacza tekst,
- List<string> oznacza listę tekstów,
- lista tekstów może przechowywać np. imiona, nazwy przedmiotów, komunikaty albo hasła menu.

Potrzebny using

Do list potrzebny jest:

Kod: csharp

```
using System.Collections.Generic;
```

Minimalny przykład:

Kod: csharp

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<string> imiona = new List<string>();
    }
}
```

Dodawanie tekstów do listy

Kod: csharp

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<string> imiona = new List<string>();

        imiona.Add("Ala");
        imiona.Add("Bartek");
        imiona.Add("Celina");

        Console.WriteLine(imiona.Count);
    }
}
```

```
    }  
}
```

Wyjaśnienie:

- Add dodaje nowy tekst na koniec listy,
- Count zwraca liczbę elementów,
- teksty zapisujemy w cudzysłowie.

Program wypisze 3, ponieważ lista zawiera trzy teksty.

Diagram: dodawanie tekstów

Diagram Mermaid

```
---  
config:  
  flowchart:  
    curve: linear  
---  
flowchart LR  
  A([Start])  
  B["Pusta lista imiona"]  
  C["Add: Ala"]  
  D["Add: Bartek"]  
  E["Add: Celina"]  
  F["3 elementy"]  
  G([Koniec])  
  
  A --> B  
  B --> C  
  C --> D  
  D --> E  
  E --> F  
  F --> G
```

Diagram pokazuje, że lista tekstów rośnie po dodaniu kolejnych elementów.

Odczyt elementu przez indeks

Kod: csharp

```
Console.WriteLine(imiona[0]);  
Console.WriteLine(imiona[1]);
```

Wyjaśnienie:

- indeksy zaczynają się od 0,
- imiona[0] oznacza pierwszy tekst,
- imiona[1] oznacza drugi tekst,
- próba odczytu nieistniejącego indeksu jest błędem.

Wypisywanie listy pętlą for

Kod: csharp

```
for (int i = 0; i < imiona.Count; i++)  
{  
    Console.WriteLine(imiona[i]);  
}
```

i jest indeksem. Count mówi, ile elementów ma lista. Przez indeks można odczytać konkretny tekst.

Wypisywanie listy pętlą foreach

Kod: csharp

```
foreach (string imie in imiona)
{
    Console.WriteLine(imie);
}
```

foreach jest wygodny, gdy chcemy wypisać wszystkie teksty i nie potrzebujemy indeksu.

Sprawdzanie, czy lista zawiera tekst

Kod: csharp

```
if (imiona.Contains("Ala"))
{
    Console.WriteLine("Ala jest na liście");
}
```

Wyjaśnienie:

- Contains sprawdza, czy lista zawiera podany tekst,
- wynik jest typu bool,
- wielkość liter ma znaczenie, więc "Ala" i "ala" to różne teksty.

Usuwanie tekstów z listy

Kod: csharp

```
imiona.Remove("Bartek");
```

Remove usuwa pierwsze wystąpienie podanego tekstu.

Kod: csharp

```
imiona.RemoveAt(0);
```

RemoveAt usuwa element z podanego indeksu.

Po usunięciu elementu zmienia się Count. Po RemoveAt zmieniają się indeksy kolejnych elementów.

Przykład: prosta lista obecności

Kod: csharp

```
using System;
using System.Collections.Generic;

class Program
{
    static void Main()
    {
        List<string> obecni = new List<string>();

        obecni.Add("Ala");
        obecni.Add("Bartek");
        obecni.Add("Celina");

        Console.WriteLine("Lista obecnych:");

        foreach (string osoba in obecni)
        {
            Console.WriteLine(osoba);
        }

        if (obecni.Contains("Ala"))
```

```
        {  
            Console.WriteLine("Ala jest obecna");  
        }  
    }  
}
```

Lista tekstów dobrze nadaje się do przechowywania prostych zbiorów nazw.

Najczęstsze błędy

- Brak `using System.Collections.Generic`.
- Zapis tekstu bez cudzysłowu.
- Mylenie `Count` z `Length`.
- Próba odczytu elementu spoza zakresu.
- Pomylenie `Remove` z `RemoveAt`.
- Oczekiwanie, że `Contains` ignoruje wielkość liter.
- Używanie indeksu po usunięciu elementu bez sprawdzenia aktualnej długości listy.

Ćwiczenia

1. Utwórz pustą listę `List<string>` o nazwie `imiona`.
2. Dodaj do listy trzy imiona.
3. Wypisz liczbę elementów listy za pomocą `Count`.
4. Wypisz pierwszy element listy.
5. Wypisz wszystkie elementy listy pętlą `for`.
6. Wypisz wszystkie elementy listy pętlą `foreach`.
7. Sprawdź, czy lista zawiera wybrane imię za pomocą `Contains`.
8. Usuń jedno imię z listy i ponownie wypisz wszystkie elementy.

Podsumowanie

`List<string>` przechowuje teksty. Teksty zapisujemy w cudzysłowie.

`Add` dodaje tekst do listy, a `Count` zwraca liczbę elementów.

Elementy listy można odczytywać przez indeks. `foreach` jest wygodny do wypisywania wszystkich tekstów.

`Contains` sprawdza, czy lista zawiera podany tekst. `Remove` i `RemoveAt` usuwają elementy na różne sposoby.

54. List a tablica

07-tablice-listy-referencje/05-list-a-tablica.md

List a tablica

Dwa sposoby przechowywania wielu wartości

W C# poznaliśmy już dwa sposoby przechowywania wielu wartości tego samego typu:

- tablicę, np. `int[]`,
- listę, np. `List<int>`.

Przykład tablicy:

Kod: csharp

```
int[] tablica = { 4, 7, 2 };
```

Przykład listy:

Kod: csharp

```
List<int> lista = new List<int>();  
lista.Add(4);  
lista.Add(7);  
lista.Add(2);
```

Obie struktury pozwalają przechowywać kilka liczb, ale różnią się sposobem pracy.

Tablica ma ustalony rozmiar

Kod: csharp

```
int[] liczby = new int[3];  
  
liczby[0] = 10;  
liczby[1] = 20;  
liczby[2] = 30;
```

Wyjaśnienie:

- tablica ma rozmiar ustalony podczas tworzenia,
- w tym przykładzie tablica ma 3 miejsca,
- nie można po prostu dodać czwartego elementu metodą `Add`,
- liczba elementów tablicy to `Length`.

Kod: csharp

```
Console.WriteLine(liczby.Length);
```

Lista może rosnąć

Kod: csharp

```
using System;  
using System.Collections.Generic;  
  
class Program  
{  
    static void Main()  
    {  
        List<int> liczby = new List<int>();  
  
        liczby.Add(10);  
    }  
}
```



```

        liczby.Add(20);
        liczby.Add(30);
        liczby.Add(40);

        Console.WriteLine(liczby.Count);
    }
}

```

Wyjaśnienie:

- lista na początku może być pusta,
- Add dodaje kolejne elementy,
- lista może zwiększać liczbę elementów,
- liczba elementów listy to Count.

Diagram: stały rozmiar i rosnąca lista

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Tablica: stały rozmiar"]
    C["Length"]
    D["Lista może rosnąć"]
    E["Count"]
    F([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F

```

Diagram pokazuje główną różnicę: tablica ma ustalony rozmiar, a lista może rosnąć. Dlatego tablica używa Length, a lista Count.

Dostęp przez indeks

W obu przypadkach można odczytać element przez indeks.

Kod: csharp

```

Console.WriteLine(tablica[0]);
Console.WriteLine(lista[0]);

```

Wyjaśnienie:

- indeksy zaczynają się od 0,
- pierwszy element ma indeks 0,
- drugi element ma indeks 1,
- wyjście poza zakres jest błędem zarówno w tablicy, jak i w liście.

Przejdźcie pętlą for

Przejdźcie po tablicy:

Kod: csharp

```
for (int i = 0; i < tablica.Length; i++)
{
    Console.WriteLine(tablica[i]);
}
```

Przejdźcie po liście:

Kod: csharp

```
for (int i = 0; i < lista.Count; i++)
{
    Console.WriteLine(lista[i]);
}
```

Przy tablicy używamy Length, a przy liście Count.

Przejdźcie pętlą foreach

Kod: csharp

```
foreach (int liczba in tablica)
{
    Console.WriteLine(liczba);
}
```

Kod: csharp

```
foreach (int liczba in lista)
{
    Console.WriteLine(liczba);
}
```

foreach działa wygodnie w obu przypadkach, gdy chcemy przejść po wszystkich elementach.

Kiedy użyć tablicy

Tablica jest dobra, gdy:

- znamy liczbę elementów,
- rozmiar nie będzie się zmieniał,
- dane mają prostą, ustaloną strukturę,
- pracujemy np. z tablicą 2D albo prostym zestawem danych.

Przykład:

Kod: csharp

```
int[] oceny = new int[5];
```

Kiedy użyć listy

Lista jest dobra, gdy:

- nie wiemy z góry, ile będzie elementów,
- będziemy dodawać elementy w trakcie działania programu,
- będziemy usuwać elementy,
- chcemy wygodnie korzystać z Add, Remove, Contains.

Przykład:

Kod: csharp

```
List<int> punkty = new List<int>();
```

Tabela porównawcza

Cecha	Tablica <code>int[]</code>	Lista <code>List<int></code>	
--- --- ---			
Rozmiar	Ustalony przy tworzeniu	Może się zmieniać	
Liczba elementów	<code>Length</code>	<code>Count</code>	
Dodawanie elementów	Niewygodne	<code>Add</code>	
Usuwanie elementów	Niewygodne	<code>Remove</code> , <code>RemoveAt</code>	
Dostęp przez indeks	Tak	Tak	
<code>foreach</code>	Tak	Tak	
Dobre zastosowanie	Stała liczba danych	Zmienna liczba danych	

Najczęstsze błędy

- Użycie Count przy tablicy.
- Użycie Length przy liście.
- Próba użycia Add na tablicy.
- Zapomnienie using System.Collections.Generic przy List<int>.
- Wyjście poza zakres indeksu.
- Używanie listy tam, gdzie prosta tablica wystarczy.
- Używanie tablicy tam, gdzie liczba elementów będzie się zmieniać.

Ćwiczenia

1. Utwórz tablicę `int[]` o rozmiarze 5 i wpisz do niej liczby.
2. Wypisz długość tablicy za pomocą `Length`.
3. Utwórz pustą listę `List<int>`.
4. Dodaj do listy kilka liczb metodą `Add`.
5. Wypisz liczbę elementów listy za pomocą `Count`.
6. Wypisz wszystkie elementy tablicy pętlą `for`.
7. Wypisz wszystkie elementy listy pętlą `for`.
8. Dla kilku opisów zadań zdecyduj, czy lepsza będzie tablica, czy lista.

Podsumowanie

Tablica i lista służą do przechowywania wielu wartości.

Tablica ma zwykle ustalony rozmiar, a lista może rosnąć i maleć.

Tablica używa `Length`, a lista używa `Count`.

Obie struktury pozwalają odczytywać elementy przez indeks. Wybór zależy od tego, czy liczba elementów jest stała, czy zmienna.

55. Przekazywanie parametrów przez wartość

07-tablice-listy-referencje/06-przekazywanie-parametrow-przez-wartosc.md

Przekazywanie parametrów przez wartość

Przypomnienie: parametr i argument

Parametr znajduje się w definicji metody. Argument przekazujemy podczas wywołania metody.

Kod: csharp

```
using System;

class Program
{
    static void PokazLiczbe(int liczba)
    {
        Console.WriteLine(liczba);
    }

    static void Main()
    {
        int x = 10;
        PokazLiczbe(x);
    }
}
```

W tym przykładzie x jest argumentem, a liczba jest parametrem metody PokazLiczbe.

Co znaczy „przez wartość”

Domyślnie C# przekazuje argumenty do metod przez wartość. Oznacza to, że metoda dostaje kopię wartości.

Jeżeli w Main mamy:

Kod: csharp

```
int x = 10;
```

a potem wywołujemy metodę:

Kod: csharp

```
ZmienLiczbe(x);
```

to metoda otrzymuje wartość 10, ale nie dostaje bezpośredniego dostępu do zmiennej x z Main.

Przykład: zmiana parametru nie zmienia zmiennej w Main

Kod: csharp

```
using System;

class Program
{
    static void ZmienLiczbe(int liczba)
    {
        liczba = 99;
        Console.WriteLine("W metodzie: " + liczba);
    }

    static void Main()
    {

```

```

        int x = 10;

        ZmienLiczbe(x);

        Console.WriteLine("W Main: " + x);
    }
}

```

Wynik programu:

- w metodzie zostanie wypisane 99,
- w Main nadal zostanie wypisane 10,
- ponieważ liczba jest kopią wartości x.

Diagram: kopia wartości

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Main: x = 10"]
    C["ZmienLiczbe(x)"]
    D["liczba dostaje kopię 10"]
    E["liczba = 99"]
    F["Powrót do Main"]
    G["x nadal 10"]
    H([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
    G --> H

```

Diagram pokazuje, że metoda pracuje na kopii wartości, a nie bezpośrednio na zmiennej x z Main.

Dlaczego tak się dzieje

Parametr metody jest osobną zmienną dostępną wewnątrz metody. Może mieć taką samą wartość jak argument, ale nie jest tą samą zmienną.

Porównanie:

- x istnieje w Main,
- liczba istnieje w metodzie ZmienLiczbe,
- zmiana liczba nie zmienia x.

Jak poprawnie zwrócić zmienioną wartość

Jeśli metoda ma obliczyć nową wartość, powinna ją zwrócić przez return.

Kod: csharp

```

using System;

class Program

```

```

{
    static int Zwiększ(int liczba)
    {
        liczba = liczba + 1;
        return liczba;
    }

    static void Main()
    {
        int x = 10;

        x = Zwiększ(x);

        Console.WriteLine(x);
    }
}

```

Wyjaśnienie:

- metoda otrzymuje kopię wartości x,
- oblicza nową wartość,
- zwraca ją przez return,
- w Main przypisujemy wynik z powrotem do x.

Zmiana parametru a zwrócenie wyniku

Sytuacja	Czy zmienia zmienną w Main?
--- ---	
Zmiana parametru w metodzie	Nie
Zwrócenie wyniku przez `return` i przypisanie go w `Main`	Tak, bo sami przypisujemy nową wartość
Samo wywołanie metody bez przypisania wyniku	Nie

Częsty błąd:

Kod: csharp

```

Zwiększ(x);
Console.WriteLine(x);

```

Jeśli nie przypiszemy wyniku, x pozostanie bez zmian.

Poprawnie:

Kod: csharp

```

x = Zwiększ(x);
Console.WriteLine(x);

```

Przykład z dwoma liczbami

Kod: csharp

```

using System;

class Program
{
    static int Dodaj(int a, int b)
    {
        return a + b;
    }

    static void Main()
    {
        int pierwsza = 4;
        int druga = 6;
    }
}

```

```
        int suma = Dodaj(pierwsza, druga);  
  
        Console.WriteLine(suma);  
    }  
}
```

Metoda otrzymuje wartości zmiennych pierwsza i druga, oblicza wynik i zwraca go.

Uwaga o tablicach i listach

Przy typach takich jak `int`, `double`, `bool` czy `char` łatwo zobaczyć przekazywanie przez wartość: metoda dostaje kopię wartości.

Przy tablicach i listach sprawa jest trudniejsza, bo są to typy referencyjne. Oznacza to, że można zmienić elementy tablicy lub listy przekazanej do metody.

Na tym etapie wystarczy zapamiętać, że zwykle liczby zachowują się inaczej niż tablice i listy. Ten temat będzie wracał przy dalszej nauce.

Przygotowanie do ref i out

C# ma specjalne słowa `ref` i `out`, które pozwalają przekazywać dane do metod inaczej niż zwykle: przekazywanie przez wartość.

Zanim jednak ich użyjemy, trzeba dobrze rozumieć zwykle przekazywanie przez wartość.

Najczęstsze błędy

- Oczekiwanie, że zmiana parametru `int` w metodzie zmieni zmienną w `Main`.
- Zapomnienie o przypisaniu wyniku metody zwracającej wartość.
- Mylenie parametru z argumentem.
- Przekonanie, że taka sama nazwa zmiennej oznacza tę samą zmienną.
- Zbyt szybkie używanie `ref` bez potrzeby.
- Mylenie zwykłych typów liczbowych z tablicami i listami.

Ćwiczenia

1. Wskaż w przykładzie, co jest argumentem, a co parametrem.
2. Napisz metodę `PokazLiczbe(int liczba)` i wywołaj ją z `Main`.
3. Napisz metodę `ZmienLiczbe(int liczba)`, która ustawia parametr na 100 i wypisuje go w metodzie.
4. Sprawdź, czy zmienna przekazana z `Main` zmieniła się po wywołaniu metody.
5. Napisz metodę `int Zwieksz(int liczba)`, która zwraca liczbę większą o 1.
6. Wywołaj `Zwieksz(x)` bez przypisania wyniku i sprawdź, co stanie się z `x`.
7. Wywołaj `x = Zwieksz(x)` i sprawdź, co stanie się z `x`.
8. Własnymi słowami wyjaśnij, co znaczy, że parametr jest przekazywany przez wartość.

Podsumowanie

Domyślnie argumenty są przekazywane do metod przez wartość.

Metoda otrzymuje kopię wartości. Zmiana parametru `int` w metodzie nie zmienia zmiennej z `Main`.

Jeśli metoda ma dać nową wartość, powinna ją zwrócić przez `return`. Wynik metody trzeba przypisać, jeśli chcemy go dalej używać.

`ref` i `out` są specjalnymi mechanizmami i będą omówione osobno.

56. ref i out

07-tablice-listy-referencje/07-ref-i-out.md

ref i out

Przypomnienie: zwykle przekazywanie przez wartość

Zwykły parametr typu `int` dostaje kopię wartości.

Kod: `csharp`

```
using System;

class Program
{
    static void ZmienLiczbe(int liczba)
    {
        liczba = 99;
    }

    static void Main()
    {
        int x = 10;

        ZmienLiczbe(x);

        Console.WriteLine(x);
    }
}
```

Program wypisze 10, bo zmiana parametru `liczba` nie zmienia zmiennej `x` w `Main`.

Po co jest ref

`ref` pozwala przekazać zmienną tak, aby metoda mogła zmienić jej wartość w miejscu wywołania.

Ważne zasady:

- `ref` zapisujemy w definicji metody,
- `ref` zapisujemy także przy wywołaniu metody,
- zmienna przekazana przez `ref` musi być wcześniej zainicjalizowana.

Przykład ref

Kod: `csharp`

```
using System;

class Program
{
    static void ZmienLiczbe(ref int liczba)
    {
        liczba = 99;
    }

    static void Main()
    {
        int x = 10;

        ZmienLiczbe(ref x);

        Console.WriteLine(x);
    }
}
```

```
    }
}
```

Wyjaśnienie:

- metoda `ZmienLiczbe` otrzymuje dostęp do zmiennej `x`,
- przypisanie `liczba = 99` zmienia `x`,
- program wypisze 99.

Diagram: ref zmienia zmienną z Main

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
  A([Start])
  B["Main: x = 10"]
  C["ZmienLiczbe(ref x)"]
  D["Metoda ustawia 99"]
  E["x w Main ma 99"]
  F([Koniec])

  A --> B
  B --> C
  C --> D
  D --> E
  E --> F

```

Diagram pokazuje, że `ref` pozwala metodzie zmienić zmienną z miejsca wywołania.

Zamiana wartości dwóch zmiennych

Kod: `csharp`

```

using System;

class Program
{
    static void Zamien(ref int a, ref int b)
    {
        int pomocnicza = a;
        a = b;
        b = pomocnicza;
    }

    static void Main()
    {
        int pierwsza = 5;
        int druga = 10;

        Zamien(ref pierwsza, ref druga);

        Console.WriteLine(pierwsza);
        Console.WriteLine(druga);
    }
}

```

Wyjaśnienie:

- `ref` pozwala metodzie zmienić obie zmienne z `Main`,
- zmienna pomocnicza jest potrzebna, aby nie stracić jednej wartości,

- po wywołaniu pierwsza ma 10, a druga ma 5.

Po co jest out

out służy do przekazywania wartości z metody na zewnątrz.

Parametr out nie musi mieć wartości przed wywołaniem, ale metoda musi przypisać mu wartość przed zakończeniem.

Ważne zasady:

- out zapisujemy w definicji metody,
- out zapisujemy także przy wywołaniu metody,
- metoda musi ustawić wartość parametru out,
- out często służy do zwrócenia dodatkowej wartości obok zwykłego return.

Przykład out

Kod: csharp

```
using System;

class Program
{
    static void UstawLiczbe(out int liczba)
    {
        liczba = 50;
    }

    static void Main()
    {
        int x;

        UstawLiczbe(out x);

        Console.WriteLine(x);
    }
}
```

Wyjaśnienie:

- x nie musi mieć wartości przed wywołaniem,
- metoda UstawLiczbe musi przypisać wartość do liczba,
- po wywołaniu x ma wartość 50.

out i wynik dodatkowy

Kod: csharp

```
using System;

class Program
{
    static bool SprobujPodzielic(int a, int b, out double wynik)
    {
        if (b == 0)
        {
            wynik = 0;
            return false;
        }

        wynik = (double)a / b;
        return true;
    }
}
```

```

    }

    static void Main()
    {
        double wynik;

        bool czySieUdalo = SprobujPodzielic(10, 2, out wynik);

        if (czySieUdalo)
        {
            Console.WriteLine(wynik);
        }
        else
        {
            Console.WriteLine("Nie można dzielić przez zero.");
        }
    }
}

```

Wyjaśnienie:

- metoda zwraca bool, czyli informację, czy operacja się udała,
- parametr out wynik zwraca obliczoną wartość,
- jeśli dzielenie nie jest możliwe, metoda też musi przypisać coś do wynik.

ref a out

Cecha	`ref`	`out`
Czy zmienna musi mieć wartość przed wywołaniem?	Tak	Nie
Czy metoda może zmienić zmienną z Main?	Tak	Tak
Czy metoda musi przypisać wartość?	Nie zawsze	Tak
Typowe użycie	Zmiana istniejącej wartości	Zwrócenie dodatkowego wyniku
Słowo przy wywołaniu	`ref`	`out`

Kiedy używać ref i out

ref i out są przydatne, ale nie powinny być pierwszym wyborem w prostych programach.

Najczęściej lepiej:

- zwrócić wynik przez return,
- użyć czytelnej metody zwracającej wartość,
- stosować ref i out dopiero wtedy, gdy faktycznie trzeba zmienić zmienną z miejsca wywołania albo zwrócić dodatkowy wynik.

Najczęstsze błędy

- Wpisanie ref tylko w definicji metody, ale nie przy wywołaniu.
- Wpisanie ref przy wywołaniu, ale nie w definicji metody.
- Przekazanie przez ref zmiennej, która nie ma wartości.
- Brak przypisania wartości do parametru out w każdej możliwej ścieżce metody.
- Nadużywanie ref zamiast return.
- Mylenie ref i out.
- Próba przekazania bezpośredniej liczby, np. ref 5, zamiast zmiennej.

Ćwiczenia

1. Napisz metodę `ZmienLiczbe(ref int liczba)`, która ustawia liczbę na 100.
2. Napisz program, który pokazuje różnicę między zwykłym parametrem `int` a parametrem `ref int`.
3. Napisz metodę `Zamien(ref int a, ref int b)`, która zamienia wartości dwóch zmiennych.
4. Napisz metodę `UstawZero(out int liczba)`, która przypisuje do parametru wartość 0.
5. Napisz metodę `SpróbujPodzielic(int a, int b, out double wynik)`, która zwraca false przy dzieleniu przez zero.
6. Wskaż, czy w danym przykładzie lepiej użyć `return`, `ref` czy `out`.
7. Popraw metodę z `out`, w której nie każda ścieżka przypisuje wartość do parametru.
8. Wyjaśnij własnymi słowami różnicę między `ref` i `out`.

Podsumowanie

Zwykle przekazywanie parametru typu `int` daje metodzie kopię wartości.

`ref` pozwala metodzie zmienić istniejącą zmienną z miejsca wywołania.

`out` pozwala metodzie zwrócić wartość przez parametr.

Zmienna `ref` musi mieć wartość przed wywołaniem. Parametr `out` musi otrzymać wartość w metodzie.

`ref` i `out` trzeba zapisać zarówno w definicji metody, jak i przy wywołaniu. W prostych przypadkach często wystarczy `return`.

57. Ćwiczenia

07-tablice-listy-referencje/08-cwiczenia.md

Ćwiczenia — tablice, listy i przekazywanie danych

Jak pracować z ćwiczeniami

- Najpierw przeczytaj dokładnie, jakich danych używa zadanie.
- Zdecyduj, czy potrzebujesz tablicy, listy czy metody.
- Pisz program małymi fragmentami.
- Po każdej części uruchom program.
- Sprawdzaj indeksy i zakresy.
- Nie używaj LINQ ani gotowych skrótów.

Ćwiczenia: tablice dwuwymiarowe

1. Utwórz tablicę `int[,]` o rozmiarze 2x3 i wpisz do niej liczby.
2. Wypisz element z wiersza 0 i kolumny 1.
3. Wypisz wszystkie elementy tablicy 2D za pomocą dwóch pętli `for`.
4. Wypisz tablicę 2D w formie tabeli.
5. Oblicz sumę wszystkich elementów tablicy 2D.
6. Oblicz sumę pierwszego wiersza.
7. Oblicz sumę wybranej kolumny.
8. Znajdź największy element w tablicy 2D.
9. Znajdź najmniejszy element w tablicy 2D.
10. Policz, ile elementów tablicy 2D jest parzystych.

Ćwiczenia: tablice wielowymiarowe

1. Utwórz tablicę `int[, ,]` o rozmiarze 2x2x2.
2. Przypisz wartość do elementu `dane[0, 1, 1]`.
3. Wypisz rozmiary wszystkich trzech wymiarów za pomocą `GetLength`.
4. Przejdź po wszystkich elementach tablicy 3D trzema pętlami `for`.
5. Wyjaśnij, dlaczego tablice 3D są trudniejsze do czytania niż tablice 2D.

Ćwiczenia: `List<int>`

1. Utwórz pustą listę `List<int>`.
2. Dodaj do listy pięć liczb metodą `Add`.
3. Wypisz liczbę elementów listy za pomocą `Count`.
4. Wypisz pierwszy i ostatni element listy.
5. Wypisz wszystkie elementy listy pętlą `for`.
6. Wypisz wszystkie elementy listy pętlą `foreach`.
7. Oblicz sumę elementów listy.
8. Policz, ile elementów listy jest większych od 10.

Ćwiczenia: List<string>

1. Utwórz pustą listę List<string> o nazwie imiona.
2. Dodaj do listy pięć imion.
3. Wypisz liczbę elementów listy.
4. Wypisz wszystkie imiona pętlą foreach.
5. Sprawdź, czy lista zawiera wybrane imię.
6. Usuń jedno imię metodą Remove.
7. Usuń pierwszy element listy metodą RemoveAt.
8. Po każdej operacji usuwania wypisz aktualną zawartość listy.

Ćwiczenia: tablica czy lista

Dla każdego przypadku wskaż, czy lepsza będzie tablica, czy lista, i krótko uzasadnij:

1. Znamy dokładnie 5 ocen ucznia.
2. Użytkownik będzie dodawał dowolną liczbę wyników.
3. Chcemy przechować planszę gry 3x3.
4. Chcemy przechowywać listę uczestników, która może się zmieniać.
5. Chcemy przechować tabelę temperatur: 7 dni i 4 pomiary dziennie.
6. Chcemy dodawać i usuwać zadania z prostej listy zadań.

Ćwiczenia: przekazywanie przez wartość

1. Napisz metodę PokazLiczbe(int liczba), która wypisuje przekazaną liczbę.
2. Napisz metodę ZmienLiczbe(int liczba), która ustawia parametr na 100 i wypisuje go w metodzie.
3. Sprawdź, czy zmienna z Main zmieniła się po wywołaniu ZmienLiczbe.
4. Napisz metodę int Zwieksz(int liczba), która zwraca liczbę większą o 1.
5. Wywołaj Zwieksz(x) bez przypisania wyniku i sprawdź wartość x.
6. Wywołaj x = Zwieksz(x) i sprawdź wartość x.

Ćwiczenia: ref

1. Napisz metodę ZmienNaZero(ref int liczba), która ustawia przekazaną zmienną na 0.
2. Napisz metodę Podwoj(ref int liczba), która podwaja przekazaną zmienną.
3. Napisz metodę Zamien(ref int a, ref int b), która zamienia wartości dwóch zmiennych.
4. Pokaż różnicę między metodą Podwoj(int liczba) a Podwoj(ref int liczba).
5. Wyjaśnij, dlaczego przy ref słowo ref musi pojawić się i w definicji metody, i przy wywołaniu.

Ćwiczenia: out

1. Napisz metodę UstawLiczbe(out int liczba), która ustawia wartość 50.
2. Napisz metodę PobierzDane(out string imie, out int wiek), która ustawia przykładowe dane.
3. Napisz metodę SprobujPodzielic(int a, int b, out double wynik), która zwraca false przy dzieleniu przez zero.
4. Popraw metodę z out, w której nie każda ścieżka przypisuje wartość do parametru.
5. Wyjaśnij różnicę między return a out.

Zadania przekrojowe

1. Program „Analiza tabeli”

- utwórz tablicę `int[, ] 3x3`,
- wypisz ją jako tabelę,
- oblicz sumę wszystkich elementów,
- znajdź minimum i maksimum.

2. Program „Oceny ucznia”

- utwórz `List<int> ocen`,
- dodaj kilka ocen,
- oblicz średnią bez LINQ,
- policz oceny pozytywne,
- sprawdź, czy lista zawiera ocenę 6.

3. Program „Lista obecności”

- utwórz `List<string> imion`,
- dodaj kilka imion,
- wypisz listę,
- sprawdź obecność wybranego ucznia,
- usuń jedno imię i wypisz listę ponownie.

4. Program „Zamiana liczb”

- utwórz dwie zmienne `int`,
- napisz metodę `Zamien(ref int a, ref int b)`,
- pokaż wartości przed i po zamianie.

5. Program „Bezpieczne dzielenie”

- napisz metodę `SpróbujPodzielic` z parametrem `out`,
- w `Main` sprawdź kilka przypadków,
- wypisz wynik albo komunikat o błędzie.

6. Program „Tablica czy lista”

- napisz krótki opis problemu,
- wybierz tablicę albo listę,
- uzasadnij wybór w komentarzu,
- napisz prostą implementację.

Kontrola samodzielna

- Czy wiem, czym różni się `int[]` od `int[, ]`?
- Czy wiem, co oznaczają `GetLength(0)` i `GetLength(1)`?
- Czy umiem przejść po tablicy 2D dwiema pętlami?
- Czy wiem, kiedy użyć `List<int>`?
- Czy wiem, kiedy użyć `List<string>`?
- Czy pamiętam różnicę między `Length` i `Count`?
- Czy wiem, co oznacza przekazywanie przez wartość?
- Czy wiem, kiedy `ref` zmienia zmienną z `Main`?

- Czy wiem, czym out różni się od ref?
- Czy umiem zdecydować, kiedy wystarczy return?

Podsumowanie

Tablice 2D służą do danych tabelarycznych, a tablice wielowymiarowe wymagają wielu indeksów.

`List<int>` i `List<string>` pozwalają przechowywać zmienną liczbę elementów.

Tablica ma `Length`, a lista ma `Count`.

Zwykły parametr typu `int` jest przekazywany przez wartość. `ref` pozwala zmienić istniejącą zmienną, a `out` pozwala zwrócić wartość przez parametr.

W prostych programach najpierw warto rozważyć zwykły `return`.

58. Wprowadzenie

08-objekty/README.md

08 - Wstęp do obiektowości

W tym dziale uczeń poznaje podstawowe pojęcia programowania obiektowego w C#: klasę, obiekt, pola, metody obiektu i konstruktor.

Planowane lekcje:

- Po co są obiekty (01-po-co-sa-objekty.md)
- Klasa i obiekt (02-klasa-i-objekt.md)
- Pola obiektu (03-pola-objektu.md)
- Metody obiektu
- Konstruktor
- Kilka obiektów
- Podsumowanie

59. Po co są obiekty

08-obiekty/01-po-co-sa-obiekty.md

Po co są obiekty

Problem: wiele zmiennych opisuje jedną rzecz

Do tej pory przechowywaliśmy dane w osobnych zmiennych.

Kod: csharp

```
string imie = "Ala";  
int wiek = 16;  
double srednia = 4.75;
```

Te trzy zmienne opisują jednego ucznia:

- imie przechowuje imię,
- wiek przechowuje wiek,
- srednia przechowuje średnią ocen.

Na początku taki zapis jest prosty i zrozumiały. Problem pojawia się wtedy, gdy danych jest więcej.

Gdy danych jest więcej

Przykład z dwoma uczniami bez obiektów:

Kod: csharp

```
string imie1 = "Ala";  
int wiek1 = 16;  
double srednia1 = 4.75;  
  
string imie2 = "Bartek";  
int wiek2 = 17;  
double srednia2 = 3.90;
```

Taki zapis zaczyna być niewygodny:

- nazwy zmiennych robią się sztuczne,
- łatwo pomylić dane,
- trudno dodać trzeciego ucznia,
- trudno przekazać dane ucznia do metody,
- program robi się mniej czytelny.

W większych programach takie rozproszone dane szybko prowadzą do bałaganu.

Pomysł: potraktuj ucznia jako jedną całość

W rzeczywistości uczeń nie jest trzema osobnymi rzeczami. Uczeń jest jedną osobą, która ma różne informacje, na przykład:

- imię,
- wiek,
- średnią ocen.

W programie chcemy móc podobnie potraktować dane ucznia jako jedną całość.

Zamiast myśleć o osobnych zmiennych imię, wiek i średnia, możemy myśleć o jednym uczniu, który ma te dane.

Obiekt jako jedna całość

Obiekt to sposób przedstawienia w programie konkretnej rzeczy, osoby albo pojęcia.

Przykłady obiektów:

- uczeń,
- książka,
- produkt,
- samochód,
- konto użytkownika,
- postać w grze.

Obiekt może mieć dane i może mieć czynności z nim związane.

Dane i czynności

Obiekt	Dane	Czynności
Uczeń	imię, wiek, średnia	pokaż dane, sprawdź promocję
Produkt	nazwa, cena, ilość	oblicz wartość
Postać w grze	imię, punkty życia	atakuj, lecz się

Dane opisują obiekt. Czynności mówią, co można z obiektem zrobić.

W C# dane i czynności można połączyć w klasie.

Klasa i obiekt — bardzo wstępnie

Klasa to opis, jakiego rodzaju dane i metody może mieć obiekt.

Obiekt to konkretny egzemplarz utworzony na podstawie klasy.

Prosta analogia:

- klasa jest jak formularz ucznia,
- obiekt jest jak wypełniony formularz konkretnego ucznia.

Nie musisz jeszcze znać całej składni klas. Szczegóły pojawią się w kolejnych lekcjach.

Pierwszy bardzo prosty przykład

Poniższy kod jest zapowiedzią dalszych lekcji. Na razie chodzi głównie o zobaczenie, że dane ucznia można zebrać w jednym obiekcie.

Kod: csharp

```
using System;

class Uczeń
{
    public string imię;
    public int wiek;
    public double średnia;
}

class Program
{
```

```

static void Main()
{
    Uczeń uczen1 = new Uczeń();

    uczen1.imie = "Ala";
    uczen1.wiek = 16;
    uczen1.srednia = 4.75;

    Console.WriteLine(uczen1.imie);
    Console.WriteLine(uczen1.wiek);
    Console.WriteLine(uczen1.srednia);
}
}

```

Wyjaśnienie:

- `class Uczeń` opisuje, jakie dane ma uczeń,
- `Uczeń uczen1` tworzy zmienną przechowującą obiekt ucznia,
- `new Uczeń()` tworzy nowy obiekt,
- `uczen1.imie` oznacza pole imię konkretnego ucznia,
- szczegóły składni będą omawiane w następnych lekcjach.

W tym przykładzie używamy pól publicznych tylko dydaktycznie, jako najprostszej wersji do nauki. Właściwości `get` i `set` będą omówione później.

Diagram: od wielu zmiennych do obiektu

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
    A([Start])
    B["Osobne zmienne:<br/>imie, wiek, srednia"]
    C["Dane jednego ucznia"]
    D["Obiekt Uczeń"]
    E["Dane są razem"]
    F([Koniec])

    A --> B
    B --> C
    C --> D
    D --> E
    E --> F

```

Diagram pokazuje, że obiekt pomaga zebrać dane jednej rzeczy w jedną całość.

Dlaczego obiekty pomagają

Obiekty pomagają, bo:

- grupują dane jednej rzeczy,
- ułatwiają przekazywanie danych do metod,
- zmniejszają liczbę luźnych zmiennych,
- ułatwiają tworzenie wielu podobnych elementów,
- porządkują większe programy,
- przygotowują do budowania bardziej realistycznych aplikacji.

Obiekty nie są potrzebne w każdym bardzo małym programie. Stają się ważne wtedy, gdy program ma więcej danych i więcej powiązanych działań.

Czego jeszcze nie robimy

W tej lekcji nie omawiamy jeszcze:

- dziedziczenia,
- interfejsów,
- polimorfizmu,
- hermetyzacji,
- właściwości get i set,
- zaawansowanego projektowania klas.

Na razie najważniejsze jest zrozumienie, że obiekt grupuje dane i zachowania.

Najczęstsze błędy na początku

- Zaczynanie od trudnych definicji zamiast od problemu.
- Mylenie klasy z obiektem.
- Mylenie jednej zmiennej z całym obiektem.
- Próba uczenia się dziedziczenia przed zrozumieniem klasy i obiektu.
- Używanie obiektów tam, gdzie wystarczy jedna prosta zmienna.
- Oczekiwanie, że obiekty od razu będą łatwe.

Ćwiczenia

1. Wypisz trzy zmienne opisujące jednego ucznia: imię, wiek, średnia.
2. Wypisz sześć zmiennych opisujących dwóch uczniów i zauważ problem z nazwami.
3. Wskaż, jakie dane może mieć obiekt Książka.
4. Wskaż, jakie dane może mieć obiekt Produkt.
5. Wskaż, jakie czynności mogłyby dotyczyć obiektu Uczeń.
6. Wyjaśnij własnymi słowami różnicę między klasą a obiektem.
7. Przepisz przykład z klasą Uczeń i zmień dane ucznia na własne.

Podsumowanie

Obiekt pozwala traktować dane jednej rzeczy jako całość.

Klasa opisuje, jakiego rodzaju dane i metody może mieć obiekt.

Obiekt jest konkretnym egzemplarzem klasy.

Obiekty pomagają porządkować większe programy. Na początku najważniejsze jest zrozumienie problemu, który obiekty rozwiązują.

60. Klasa i obiekt

08-objekty/02-klasa-i-objekt.md

Klasa i obiekt

Przypomnienie z poprzedniej lekcji

W poprzedniej lekcji pojawiły się trzy ważne myśli:

- obiekt pozwala traktować dane jednej rzeczy jako całość,
- klasa opisuje, jakie dane i metody może mieć obiekt,
- obiekt jest konkretnym egzemplarzem klasy.

Teraz spokojnie rozdzielimy te dwa pojęcia: klasę i obiekt.

Klasa jako opis

Klasa jest opisem rodzaju obiektu. Sama klasa nie jest jeszcze konkretną rzeczą.

Proste analogie:

- klasa to formularz, a obiekt to wypełniony formularz,
- klasa to przepis, a obiekt to konkretne danie,
- klasa to projekt domu, a obiekt to konkretny dom.

W programie klasa mówi, jakie dane i działania mogą należeć do obiektu danego typu.

Prosta klasa Uczeń

Kod: **csharp**

```
class Uczeń
{
    public string imie;
    public int wiek;
    public double srednia;
}
```

Wyjaśnienie:

- `class Uczeń` tworzy nowy typ danych,
- `imie`, `wiek` i `srednia` są polami,
- pola przechowują dane obiektu,
- `public` oznacza, że na tym etapie możemy odwołać się do pola z `Main`,
- to jest uproszczony zapis dydaktyczny; później poznamy lepsze sposoby kontroli dostępu.

Obiekt jako egzemplarz klasy

Kod: **csharp**

```
Uczen uczen1 = new Ucen();
```

Wyjaśnienie:

- `Uczen` to typ,
- `uczen1` to zmienna przechowująca obiekt,

- `new Uczeń()` tworzy nowy obiekt,
- obiekt ma własne pola `imie`, `wiek` i `srednia`.

Klasa `Uczeń` jest opisem, a `uczen1` jest konkretnym obiektem utworzonym na podstawie tej klasy.

Dostęp do pól przez kropkę

Kod: `csharp`

```
uczen1.imie = "Ala";
uczen1.wiek = 16;
uczen1.srednia = 4.75;

Console.WriteLine(uczen1.imie);
```

Kropka pozwala wejść do pola konkretnego obiektu.

`uczen1.imie` oznacza pole `imie` obiektu `uczen1`. To nie jest osobna zmienna `imie` w `Main`, tylko pole obiektu.

Kompletny przykład

Kod: `csharp`

```
using System;

class Uczeń
{
    public string imie;
    public int wiek;
    public double srednia;
}

class Program
{
    static void Main()
    {
        Uczeń uczen1 = new Uczeń();

        uczen1.imie = "Ala";
        uczen1.wiek = 16;
        uczen1.srednia = 4.75;

        Console.WriteLine(uczen1.imie);
        Console.WriteLine(uczen1.wiek);
        Console.WriteLine(uczen1.srednia);
    }
}
```

Co dzieje się w programie:

1. `class Uczeń` opisuje dane ucznia.
2. W `Main` instrukcja `new Uczeń()` tworzy nowy obiekt.
3. Zmienna `uczen1` przechowuje ten obiekt.
4. Do pól obiektu przypisywane są wartości.
5. Program wypisuje wartości pól na ekranie.

Diagram: klasa i obiekt

Diagram Mermaid

```
---
config:
  flowchart:
```



```

        curve: linear
    ---
    flowchart LR
        A([Start])
        B["Klasa Uczeń<br/>opis"]
        C["new Uczeń()"]
        D["Obiekt uczeń1"]
        E["Pola obiektu"]
        F([Koniec])

        A --> B
        B --> C
        C --> D
        D --> E
        E --> F

```

Diagram pokazuje, że klasa jest opisem, a obiekt powstaje dopiero po użyciu new.

Kilka obiektów tej samej klasy

Kod: csharp

```

Uczen uczen1 = new Uczeń();
Uczen uczen2 = new Uczeń();

uczen1.imie = "Ala";
uczen1.wiek = 16;
uczen1.srednia = 4.75;

uczen2.imie = "Bartek";
uczen2.wiek = 17;
uczen2.srednia = 3.90;

Console.WriteLine(uczen1.imie);
Console.WriteLine(uczen2.imie);

```

Wyjaśnienie:

- oba obiekty powstały na podstawie tej samej klasy,
- uczen1 i uczen2 mają takie same pola,
- wartości pól mogą być różne,
- zmiana pola jednego obiektu nie zmienia pola drugiego obiektu.

Klasa to nie obiekt

Sama klasa Uczeń jest tylko opisem. Dopiero new Uczeń() tworzy konkretny obiekt.

Ważne:

- klasa Uczeń nie jest jeszcze konkretnym uczniem,
- konkretnym uczniem jest dopiero obiekt, np. uczen1,
- z jednej klasy można utworzyć wiele obiektów.

To podobne do formularza: pusty formularz jest wzorem, a wypełniony formularz opisuje konkretną osobę.

Najczęstsze błędy

- Mylenie klasy z obiektem.
- Zapomnienie o new Uczeń().
- Próba odwołania się do pola bez obiektu.

- Pomylenie `uczen1.imie` z osobną zmienną `imie`.
- Oczekiwanie, że zmiana `uczen1` zmieni też `uczen2`.
- Zbyt szybkie przechodzenie do zaawansowanej teorii OOP.

Ćwiczenia

1. Napisz klasę `Ksiazka` z polami `tytul`, `autor` i `liczbaStron`.
2. Utwórz obiekt klasy `Ksiazka`.
3. Przypisz wartości do pól obiektu `Ksiazka`.
4. Wypisz dane książki na ekranie.
5. Napisz klasę `Produkt` z polami `nazwa`, `cena` i `ilosc`.
6. Utwórz dwa obiekty klasy `Produkt` i nadaj im różne wartości pól.
7. Wyjaśnij własnymi słowami różnicę między klasą a obiektem.
8. Sprawdź, co się stanie, jeśli spróbujesz użyć pola obiektu przed utworzeniem obiektu przez `new`.

Podsumowanie

Klasa jest opisem typu obiektu.

Obiekt jest konkretnym egzemplarzem klasy.

`new` tworzy nowy obiekt. Pola przechowują dane obiektu.

Do pól obiektu odwołujemy się przez kropkę. Z jednej klasy można utworzyć wiele obiektów, a każdy obiekt ma własne wartości pól.

61. Pola obiektu

08-objekty/03-pola-obiektu.md

Pola obiektu

Cel lekcji

Po tej lekcji rozumiesz, że pola przechowują dane obiektu. Potrafisz zdefiniować proste pola w klasie, przypisać im wartości dla konkretnego obiektu i odczytać te wartości za pomocą kropki.

Co to jest pole

Pole to zmienna należąca do obiektu. Przechowuje jedną informację o obiekcie.

Jeżeli tworzymy klasę opisującą ucznia, pola mogą przechowywać na przykład imię, wiek i średnią ocen.

Kod: csharp

```
class Uczeń
{
    public string imie;
    public int wiek;
    public double srednia;
}
```

W tym przykładzie:

- imie jest polem typu string,
- wiek jest polem typu int,
- srednia jest polem typu double.

Każde pole ma typ i nazwę. Pola opisują dane, które może przechowywać obiekt danej klasy.

Pole a zmienna lokalna

Zmienna lokalna jest zadeklarowana wewnątrz metody. Istnieje tylko w tej metodzie lub w jej bloku kodu.

Pole jest zadeklarowane w klasie. Należy do obiektu utworzonego na podstawie tej klasy.

Kod: csharp

```
class Uczeń
{
    public string imie;
}

class Program
{
    static void Main()
    {
        string tekst = "Ala";
    }
}
```

W tym przykładzie:

- imie jest polem klasy Uczeń,
- tekst jest zmienną lokalną metody Main.

To ważna różnica. Pole opisuje dane obiektu, a zmienna lokalna jest pomocniczą wartością używaną w określonym miejscu programu.

Dostęp do pola przez kropkę

Aby dostać się do pola konkretnego obiektu, używamy kropki.

Kod: csharp

```
Uczen uczen1 = new Uczen();

uczen1.imie = "Ala";
uczen1.wiek = 16;
uczen1.srednia = 4.75;

Console.WriteLine(uczen1.imie);
```

Zapis `uczen1.imie` oznacza pole `imie` obiektu `uczen1`.

Schemat jest prosty:

Kod: text

```
nazwaObiektu.nazwaPola
```

Najpierw podajemy nazwę obiektu, potem kropkę, a na końcu nazwę pola.

Kompletny przykład z polami

Kod: csharp

```
using System;

class Uczen
{
    public string imie;
    public int wiek;
    public double srednia;
    public bool czyAktywny;
}

class Program
{
    static void Main()
    {
        Uczen uczen1 = new Uczen();

        uczen1.imie = "Ala";
        uczen1.wiek = 16;
        uczen1.srednia = 4.75;
        uczen1.czyAktywny = true;

        Console.WriteLine(uczen1.imie);
        Console.WriteLine(uczen1.wiek);
        Console.WriteLine(uczen1.srednia);
        Console.WriteLine(uczen1.czyAktywny);
    }
}
```

Pola mogą mieć różne typy danych. W przykładzie użyto:

- `string` do tekstu,
- `int` do liczby całkowitej,
- `double` do liczby rzeczywistej,
- `bool` do wartości `true` albo `false`.

Diagram: obiekt i jego pola

Diagram Mermaid

```

---
config:
  flowchart:
    curve: linear
---
flowchart LR
  A([Start])
  B["Obiekt uczen1"]
  C["Pole imie"]
  D["Pole wiek"]
  E["Pole srednia"]
  F["Odczyt pól"]
  G([Koniec])

  A --> B
  B --> C
  B --> D
  B --> E
  C --> F
  D --> F
  E --> F
  F --> G

```

Obiekt może mieć wiele pól. Każde pole przechowuje osobną informację o tym samym obiekcie.

Każdy obiekt ma własne pola

Jeżeli utworzymy dwa obiekty tej samej klasy, każdy z nich ma własne wartości pól.

Kod: csharp

```

using System;

class Uczeń
{
    public string imie;
    public int wiek;
}

class Program
{
    static void Main()
    {
        Uczeń uczen1 = new Uczeń();
        Uczeń uczen2 = new Uczeń();

        uczen1.imie = "Ala";
        uczen1.wiek = 16;

        uczen2.imie = "Bartek";
        uczen2.wiek = 17;

        Console.WriteLine(uczen1.imie);
        Console.WriteLine(uczen2.imie);
    }
}

```

Obiekty uczen1 i uczen2 mają takie same pola, bo powstały z tej samej klasy. Ich wartości mogą być jednak różne.

Zmiana pola jednego obiektu nie zmienia pola drugiego obiektu.

Zmiana wartości pola

Wartość pola można zmienić podobnie jak wartość zmiennej. Trzeba jednak wskazać, którego obiektu dotyczy zmiana.

Kod: csharp

```
uczen1.srednia = 4.75;  
Console.WriteLine(uczen1.srednia);  
  
uczen1.srednia = 5.10;  
Console.WriteLine(uczen1.srednia);
```

W tym przykładzie zmieniamy wartość pola `srednia` obiektu `uczen1`.

public jako uproszczenie

W tej lekcji używamy pól zapisanych z wyrazem `public`, aby wyraźnie zobaczyć mechanizm działania pól.

Kod: csharp

```
public string imie;
```

Oznacza to, że do pola można dostać się bezpośrednio z zewnątrz klasy.

W większych programach zwykle nie udostępnia się pól w taki sposób. Później poznasz bezpieczniejsze sposoby pracy z danymi obiektu, na przykład właściwości. Na tym etapie najważniejsze jest zrozumienie, że pole przechowuje dane obiektu.

Najczęstsze błędy

- Użycie pola bez podania nazwy obiektu.
- Mylenie pola ze zmienną lokalną.
- Pominięcie kropki przy dostępie do pola.
- Oczekiwanie, że zmiana pola jednego obiektu zmieni pole drugiego obiektu.
- Nadawanie polom nieczytelnych nazw, na przykład `x`, `a`, `dane`.
- Traktowanie pól `public` jako docelowego stylu pisanie większych programów.

Przykład błędu:

Kod: csharp

```
imie = "Ala";
```

Jeżeli `imie` jest polem obiektu, trzeba wskazać obiekt:

Kod: csharp

```
uczen1.imie = "Ala";
```

Ćwiczenia

1. Utwórz klasę `Ksiazka` z polami `tytul`, `autor` i `liczbaStron`.
2. Utwórz obiekt klasy `Ksiazka` i przypisz wartości do jego pól.
3. Wypisz w konsoli wartości pól obiektu `Ksiazka`.
4. Utwórz klasę `Produkt` z polami `nazwa`, `cena` i `ilosc`.
5. Utwórz dwa obiekty klasy `Produkt` i nadaj im różne wartości pól.
6. Zmień wartość pola `cena` jednego produktu i wypisz ją ponownie.
7. Wyjaśnij własnymi słowami różnicę między polem a zmienną lokalną.
8. W przykładowym programie wskaż, które nazwy są polami, a które zmiennymi lokalnymi.

Podsumowanie

- Pole przechowuje dane obiektu.
- Każde pole ma typ i nazwę.
- Do pola konkretnego obiektu odwołujemy się przez kropkę.
- Każdy obiekt ma własne wartości pól.
- Pole różni się od zmiennej lokalnej miejscem deklaracji i zakresem użycia.
- Pola `public` są na początku kursu uproszczeniem dydaktycznym.

62. Wprowadzenie

09-projekty/README.md

09 - Projekty

Ten dział będzie zawierał proste projekty konsolowe utrwalające materiał z kursu.

Na razie jest to tylko zapowiedź. Rozbudowane opisy projektów i większe zadania powstaną w osobnym etapie.

Planowane projekty

- kalkulator,
- parzysta / nieparzysta,
- zgadywanie liczby,
- średnia ocen,
- menu konsolowe.